Learn Harness Engineering

简体中文 coursebook PDF · generated 2026-10-30

55 个章节 · 目录见下一页

目录

Learn Harness Engineering · 简体中文

	章节	PDF 页码
01	欢迎来到 Learn Harness Engineering	4
02	第一讲. 模型能力强，不等于执行可靠	7
03	第二讲. Harness 到底是什么	12
04	第三讲. 让代码仓库成为唯一的事实来源	17
05	第四讲. 把指令拆分到不同文件里	23
06	第五讲. 让跨会话的任务保持上下文连续	29
07	第六讲. 让 agent 每次工作前先初始化	36
08	第七讲. 给 agent 划清每次任务的边界	42
09	第八讲. 用功能清单约束 agent 该做什么	48
10	第九讲. 防止 agent 提前宣告完成	54
11	第十讲. 跑通完整流程才算真正验证	60
12	第十一讲. 让 agent 的运行过程可观测	66
13	第十二讲. 每次会话结束前都做好交接	73
14	第十三讲. 从手动驱动到自动循环	82
15	第十四讲. 从单循环到图工程	102
16	欢迎来到项目实战	118
17	Project 01. 只写提示词让 agent 做，和定好规则再让它做，差多少	120
18	Project 02. 让 agent 看懂项目、接住上次的工作	124
19	Project 03. 让 agent 关掉再打开还能接着干	128
20	Project 04. 用运行反馈修正 agent 的行为	132
21	Project 05. 让 agent 自己检查自己做的对不对	136
22	Project 06. 搭建一套完整的 agent 工作环境	140
23	Project 07. 搭建你的第一个自动循环	145
24	Project 08. 把你的工作流画成一张图	151
25	中文资料库	156
26	模板使用指南	158
27	编码代理开工流程	165
28	中文参考	167
29	初始化代理操作手册	171
30	方法对照表	173
31	Prompt 校准	174
32	OpenAI 高级资源包	175

	章节	PDF 页码
33	AGENTS.md	**178**
34	ARCHITECTURE.md	**180**
35	核心信念	**183**
36	设计文档索引	**184**
37	DESIGN.md	**185**
38	当前执行计划	**186**
39	已完成计划	**187**
40	技术债跟踪	**188**
41	FRONTEND.md	**189**
42	数据库结构	**190**
43	PLANS.md	**191**
44	PRODUCT_SENSE.md	**193**
45	产品规格索引	**194**
46	新用户引导	**195**
47	QUALITY_SCORE.md	**196**
48	RELIABILITY.md	**198**
49	SECURITY.md	**200**
50	高级仓库模板	**201**
51	SOP：Chrome DevTools 验证闭环	**202**
52	SOP：把不可见知识编码进仓库	**204**
53	OpenAI 高级 SOP	**206**
54	SOP：分层领域架构	**207**
55	SOP：可观测性反馈闭环	**209**

页码为合并后 PDF 的绝对页码；页脚数字仍按各章节内部重新计数。

欢迎来到 Learn Harness Engineering

Learn Harness Engineering 是一门专注于 AI 编程智能体工程化落地的课程。本课程深度研究并总结了业内最前沿的 Harness Engineering（工具马具/脚手架工程）理论与实践，参考资料包括：

- OpenAI: Harness engineering: leveraging Codex in an agent-first world
- Anthropic: Effective harnesses for long-running agents
- Anthropic: Harness design for long-running application development
- Awesome Harness Engineering

通过系统的环境设计、状态管理、验证与控制机制，本课程旨在帮助你让 Codex 和 Claude Code 等 AI Agent 能够真正可靠地完成真实工程任务。它通过明确的规则和边界约束你的 AI 编程助手，帮助你更可靠地构建功能、修复 Bug 并自动化开发任务。

开始学习

选择适合你的学习路径。本课程分为理论讲义、实战项目和开箱即用的资料库。

讲义

理解为什么强大的模型依然会失败，掌握构建有效 Harness 的理论基础。

项目

动手实践，从零开始搭建一个可靠的 Agent 工作环境。

资料库

开箱即用的模板（AGENTS.md、feature_list.json 等），可直接复制到你自己的代码仓库中。

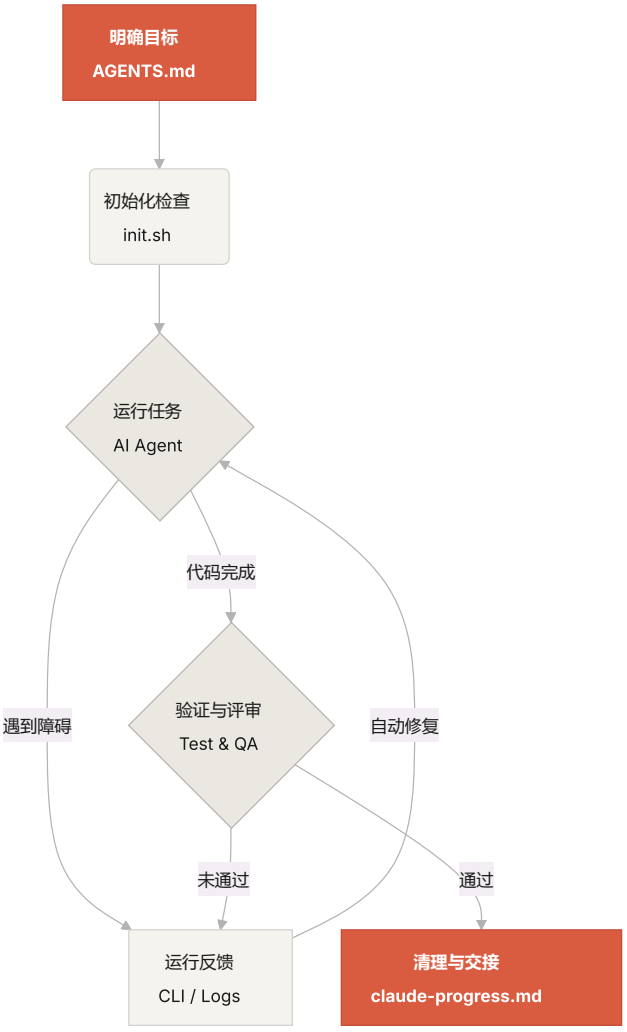
前沿 Harness 拆解

用本课程的理论框架，逐层拆解 Pi、Claude Code、Codex、DeepSeek 的真实 harness 设计。

Harness 的核心机制

Harness 的本质不是“让模型变聪明”，而是给模型建立一套闭环的**工作系统**。你可以通过下面的简单图示理解它的核心运作流：

点击放大



你将学到什么

你将在本课程中掌握以下核心概念：

- 用明确的规则和边界**约束 Agent 的行为**。
- 在跨会话的长时任务中**保持上下文连续性**。
- **防止 Agent 提前宣告**任务完成。
- 让 Agent 学会通过完整的流水线测试来**验证自己的工作**。
- 让 Agent 的运行过程**可观测、可调试**。

下一步

了解核心概念后，可以通过以下内容深入学习：

- L01. 模型能力强，不等于执行可靠：从理论开始。
- P01. 提示词 vs 规则驱动：完成你的第一个对比实战任务。
- 中文模板：获取最小 Harness 模板包（AGENTS.md、feature_list.json 等），直接用于你的项目。

English Version →

> 本篇代码示例：code/ 实战练习：Project 01. 只写提示词让 agent 做，和定好规则再让它做，差多少

第一讲. 模型能力强，不等于执行可靠

2025 年 9 月 29 日，Anthropic 公布 Claude Sonnet 4.5 在 SWE-bench Verified 上取得 77.2%。这个数字听起来还不错，但别急着高兴——那些都是精心挑选过的任务，有明确的 issue 描述，有现成的测试用例。等你把自己日常的需求丢过去，需求模糊、没有现成测试、隐含的业务规则散落在各处，通过率只会更低。你信心满满地交代一个任务，agent 跑了 20 分钟后告诉你"做完了"，你一看代码，加了功能但测试挂了，改了 bug 但引入了新 bug，根本不是你要的东西。

遇到这种情况，大多数人的第一反应是"这模型不行，换一个更贵的"。先别急着掏钱包。问题可能不在模型身上。

同一匹马，两种命运

Anthropic 比较了同一个 prompt、"做一个 2D 复古游戏编辑器"、同一个模型 Opus 4.5 在两种 harness 下的表现。第一次裸跑，20 分钟花了 $9：应用能打开，但实体出现在屏幕上、对输入没有反应，实体定义和游戏运行时的接线断了。第二次配上了完整的 harness——planner、generator、evaluator 三 agent 架构——6 小时花了 $200：能移动实体，但物理有粗糙之处，还会被墙挡住，核心玩法能跑，应用仍不完整。

Opus 4.5 还是那个 Opus 4.5，模型没有换。换的是马具。

OpenAI 在 2025 年的 harness engineering 文章里把这件事说得更直白。他们说 Codex 在一个 harness 搭得好的仓库里，表现能从"不可靠"直接跳到"可靠"。注意这个用词——不是"好了一点"，是质变。harness 这个词在这里的意思就是**模型权重之外的一切工程基础设施。**

---

agent 到底卡在哪

具体的失败模式其实就那么几种：

- **需求描述模糊，agent 只能猜。** "加个搜索功能"，这话说了等于没说。搜索的对象是什么？全文本还是结构化查询？结果要不要分页、要不要高亮？你没说明白，agent 就只好自己猜。猜对了算运气好，猜错了你再改，来回一折腾，比一开始说清楚多花好几倍的时间。
- **隐性约定没写下来，agent 无从遵守。** 你们全组都用 SQLAlchemy 2.0 的新语法，但 agent 默认写了 1.x 的代码；所有 API 端点必须走 OAuth 2.0 认证，可这条规矩只存在于你脑子里和三个月前一条 Slack 消息里。Agent 压根不知道有这么回事，不是不想遵守，是真没见过。
- **环境配置有缺口，agent 把精力花在修环境上。** 开发环境配置不完整、依赖缺了、工具版本不对，agent 把宝贵的上下文窗口花在了 `pip install` 报错、Node 版本冲突这些事上，真正该干的活反而没精力做。
- **缺少验证手段，agent 自己觉得做完了就算完成。** 没有测试、没有 lint、或者验证命令根本没告诉 agent。Agent 写完代码，自己看了看觉得没问题，就说完成了。Anthropic 还观察到一个有意思的现象：当 agent 感觉上下文快满了，它们会匆忙结束当前工作，跳过验证步骤，选一个简单的方案而不是最优方案。他们把这叫"上下文焦虑"。
- **跨会话状态丢失，每个新会话都要重新探索。** 上次会话的发现全丢了，每个新会话都得重新探索项目结构、理解代码组织。缺乏持久化状态的 agent 在超过 30 分钟的任务中失败率急剧上升。

关键名词解释

理解了上面的场景，这些概念就不再是一堆术语了：

- **能力鸿沟（Capability Gap）**：模型在基准测试上的表现和真实任务上的表现之间的巨大落差。基准任务的 issue 描述、测试和环境是给定的；真实任务里，这些往往要你自己补。
- **Harness**：模型之外的一切——指令、工具、环境、状态管理、验证反馈。不是模型权重的部分，全是 harness。也就是我们说的"马具"。
- **Harness 诱导失败**：模型本身能力足够，但因为执行环境有结构性缺陷而失败。上面的复古游戏对比就是一个例子。
- **验证缺口**：agent 对自己输出的信心评估和实际正确性之间的偏差。agent 说"我做完了"但实际没做完——这是最常见的失败模式。
- **诊断循环**：执行 → 观察失败 → 定位到 harness 的哪一层出了问题 → 修补那一层 → 重新执行。这是 harness 工程的核心方法论。
- **完成定义（Definition of Done）**：一组可以用命令验证的条件——测试通过、lint 没报错、类型检查通过。没有显式的完成定义，agent 就会自己编一个。

遇到失败，先修 harness

核心原则只有一条：**遇到失败，先别换模型，先检查 harness。** 如果同一个模型在类似的结构良好的任务中能成功，那优先假设是 harness 的问题。

具体怎么做？每次失败都归因到具体层。不要笼统地说"模型不行"，问自己：是任务没说清楚？是上下文不够？是没有验证手段？把每次失败归到五层防御里——任务规范、上下文供给、执行环境、验证反馈、状态管理。养成这个习惯，你会发现"模型不行"这个结论在日志里出现得越来越少。

然后，给每个任务写显式的完成定义。不要说"加个搜索功能"，要说清楚：

```
完成标准：
- 新增 GET /api/search?q=xxx 端点
- 支持分页，默认 20 条
- 返回结果包含高亮片段
- 所有新代码通过 pytest
- 类型检查通过（mypy --strict）
```

在仓库根目录放一个 `AGENTS.md` 文件，告诉 agent 这个项目的技术栈、架构约定、验证命令。这是 harness 工程的第一步，也是投入产出比最高的一步。一个 `AGENTS.md` 文件可能比你换一个更贵的模型更有效——这话不是开玩笑。

再往后，建立诊断循环。不要把失败当成"模型又犯傻了"，把它当作"harness 又暴露了一个缺陷"的信号。每次失败，定位到某一层，修补，下次不再犯。几轮下来，你的 harness 会越来越强，agent 的表现也会稳定提升。顺手记个简单的日志就行——每个任务成功了没有，失败了是哪一层的问题。跑几轮之后你就能看出来哪个层是瓶颈，集中火力修那个层。

一百万行代码的实验

2025 年 8 月，OpenAI 的三个工程师开始了一项实验。规则很简单：他们不写代码，只让 Codex 写。从一个空的 git 仓库起步，五个月下来，仓库里有了大约 100 万行代码。应用逻辑、基础设施、工具、文档——全是 agent 生成的。团队后来扩展到七人，一共合并了约 1,500 个 PR，平均每人每天 3.5 个。

起初的进展出乎意料地慢。Codex 并不差，但它缺少足够完整的工具和结构去推进那些高层次的目标。三个工程师慢慢摸到了门道：把大的目标拆成小的积木块——设计、编码、审查、测试——让 agent 逐个搭建，再用这些积木去组合更复杂的任务。每当某件事做砸了，问题几乎从来不是"不够努力"，而是 agent 还缺什么——缺的能力能不能用一种既可理解又可执行的方式补上去。

这个实验直接印证了本讲的核心论点：**同一个模型，在空白环境里和在有完整 harness 的环境里，产出有本质差异。** 模型没变，变的是环境。

> 来源：OpenAI: Harness engineering: leveraging Codex in an agent-first world

---

一个更接地气的例子

一个团队用 Claude Sonnet 给一个中等规模的 Python Web 应用（FastAPI + PostgreSQL + Redis，约 15,000 行代码）添加新的 API 端点。

起初他们只给了一句话："在 `/api/v2/users` 下添加用户偏好设置端点"。结果呢？agent 把大量时间花在探索仓库结构上，产出了看似合理的代码但没遵循项目的错误处理模式，用了旧版 SQLAlchemy 语法，宣称完成但端点实际有运行时错误。下一个会话还得重新做发现工作。

后来他们加了 `AGENTS.md`（描述项目架构和技术栈版本）、显式的验证命令（`pytest tests/api/v2/ && python -m mypy src/`）、和架构决策记录。同一个模型开始遵循项目约定，并用能执行的命令检查工作。

模型没变。变的还是 harness。

核心要点

- 模型能力和执行可靠性是两回事，千里马也得配上好马具。
- 失败的时候先看 harness，再看模型。换模型是成本最高的选择，而且很多时候根本不是模型的问题。
- 每次失败都是一个信号：你的 harness 有结构性缺陷。把它找出来、修掉。
- 遇到失败不要笼统地说"模型不行"。任务没交代清楚、上下文不够、环境没配好、验证手段缺失、上一个会话做到哪了新会话接不上，五个地方逐个排查，问题十有八九出在其中一层。

- 一个 `AGENTS.md` 文件可能比你换一个更贵的模型更有效。

延伸阅读

- OpenAI: Harness Engineering — Leveraging Codex in an Agent-First World
- Anthropic: Effective Harnesses for Long-Running Agents
- HumanLayer: Skill Issue — Harness Engineering for Coding Agents
- SWE-bench Leaderboard
- Thoughtworks Technology Radar: Harness Engineering

练习

1. **对比实验**：选一个你熟悉的代码仓库和一项非平凡的修改任务。先不给任何 harness 支持，让 agent 跑一次，记录失败。然后加一个 `AGENTS.md` 和显式的验证命令，让同一个 agent 再跑一次。对比两次的结果，把失败归因到五层防御中的某一层。
2. **验证缺口测量**：选 5 个编码任务，在每个任务完成后记录 agent 是否声称完成，然后用独立测试验证实际正确性。计算 agent 在实际没完成时声称完成的比例——这就是你的验证缺口。然后想想：加什么验证命令能把这个比例降下来？
3. **诊断循环实践**：找一个 agent 在你的项目中反复失败的任务。跑一次，记录失败。归因到五层中的某一层。修那一层。再跑。重复三到五轮，记录每一轮的改善。

English Version →

> 本篇代码示例：code/ 实战练习：Project 01. 只写提示词让 agent 做，和定好规则再让它做，差多少

第二讲. Harness 到底是什么

"harness"这个词在 AI coding agent 的圈子里被用得越来越多了，但大部分人说 harness 的时候，其实指的是一个 prompt 文件。一个 prompt 文件不是 harness。

这节课要给 harness 一个精确的、可操作的定义。不是学术论文里的抽象概念，而是你今天就能拿去用的框架。harness 由五个子系统组成：指令、工具、环境、状态、反馈。每个子系统都有明确的职责和评判标准。

先讲一个类比

想象你是一个刚入职的工程师，被丢进一个没有任何文档的项目里。没有 README，代码里没有注释，没有人告诉你怎么跑测试，CI 配置文件藏在某个角落里。你能写出好代码吗？也许能——如果你足够聪明又足够有耐心。但你会花大量时间在"搞清楚这个项目是怎么回事"上，而不是在"解决问题"上。

AI agent 面对的困境一模一样，甚至更糟。你至少可以问同事，agent 只能看到你放在它面前的文件和它能执行的命令。

OpenAI 在他们的 harness engineering 文章里把 harness 的核心原则表述为"仓库即规范"，所有必要的上下文都应该在仓库里，通过结构化的指令文件、明确的验证命令和清晰的目录组织来呈现。Anthropic 的 long-running agents 文档则更侧重状态持久化、显式恢复路径和结构化的进度跟踪。两家公司的侧重点不同，但说的是同一件事：**模型之外的一切工程基础设施，决定了模型能力能被发挥多少。**

看看几个你熟悉的工具：

Claude Code 的设计就体现了 harness 思想。它会读你仓库里的 `CLAUDE.md`，能用 shell 跑命令，在你的本地环境里执行，有会话历史，能跑测试看结果。但如果你不告诉它怎么跑测试，它就没法验证自己做得对不对。

Cursor 也是类似的逻辑。`.cursorrules` 文件是它的指令来源，终端是它的工具，它能读你的项目结构和 lint 配置。不过 Cursor 的状态管理相对弱，你关掉 IDE 再打开，上次的上下文就没了。

Codex（OpenAI 的 coding agent）用 git worktree 隔离每个任务的运行环境，配合本地的可观测性栈（日志、指标、追踪），让每个变更都在独立的环境中验证。它在有 `AGENTS.md` 和清晰验证命令的仓库里，表现远超在"裸"仓库里。

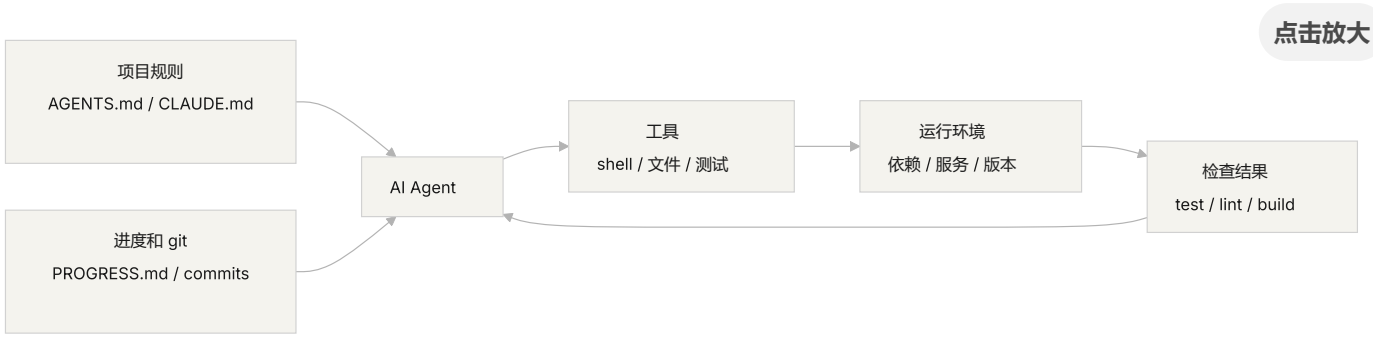
AutoGPT 则是反面教材。缺乏结构化的状态管理导致长任务中上下文不断累积，缺乏精确的反馈机制导致 agent 陷入循环。很多人说 AutoGPT "不行"，但其实是它的 harness 不行。

核心概念

- **什么是 harness**：模型权重之外的一切工程基础设施。OpenAI 把工程师的核心工作概括为三件事：设计环境、表达意图、构建反馈循环。Anthropic 直接把 Claude Agent SDK 称为"通用 agent harness"。
- **仓库是唯一事实来源**：agent 看不到的东西，对它来说就不存在。OpenAI 把仓库当作"记录系统"，所有必要的上下文都必须在仓库里，通过结构化的文件和清晰的目录组织来呈现。
- **给地图，不给说明书**：OpenAI 的经验是 `AGENTS.md` 应该是目录页，不是百科全书。100 行左右就够了，放不下就拆分到 `docs/` 目录里，让 agent 按需去读。
- **约束而非微操**：好的 harness 用可执行的规则来约束 agent，而不是在指令里逐条叮嘱。OpenAI 说"执行不变量，不要微管实现"；Anthropic 发现 agent 会自信地夸赞自己的工作，解决方案是把"干活的人"和"检查的人"分开。
- **逐个移除看效果**：想量化 harness 各组件的边际贡献，就逐个移除，看哪个移除后性能下降最多。它能告诉你哪些组件当前最有价值，也能暴露哪些组件暂时贡献不明显。Anthropic 用这个方法发现：随着模型变强，某些组件不再关键，但总会有新的关键组件出现。

Harness 五子系统模型

回到类比。harness 有五个子系统：



指令子系统：创建 `AGENTS.md`（或 `CLAUDE.md`），内容包括项目概览和目的、技术栈和版本、首次运行命令、不可违反的硬约束、指向更详细文档的链接。

工具子系统：确保 agent 有足够的工具访问权限。不要因为"安全考虑"把 shell 给禁了，agent 连 `pip install` 都跑不了，还怎么干活？但也别什么都开放，按最小权限原则来。

环境子系统：让环境状态自描述。用 `pyproject.toml` 或 `package.json` 锁定依赖，用 `.nvmrc` 或 `.python-version` 指定运行时版本，用 Docker 或 devcontainer 让环境可重现。

状态子系统：长任务必须有进度跟踪。用一个简单的 `PROGRESS.md` 文件记录：哪些做完了，哪些在做，哪些被阻塞。每个会话结束前更新，下一个会话开始时读取。

反馈子系统：这是投入产出比最高的子系统。在 `AGENTS.md` 里显式列出验证命令：

```
验证命令：
- 测试：pytest tests/ -x
- 类型检查：mypy src/ --strict
- Lint：ruff check src/
- 完整验证：make check（包含以上全部）
```

五个子系统缺一个，harness 就不完整，agent 用起来总会别扭。

量化 harness 组件价值的方法：用"控制变量排除法"。保持模型不变，逐个移除五个子系统，看哪个子系统缺失时性能下降最多。下降最多的组件，说明它在当前任务里边际贡献最大，值得优先保留；是否要加强它，取决于失败归因，而不是只看下降幅度。几乎没有影响的组件，也不能直接判定为无用：它可能是冗余、设计失效，或还没有被这个任务充分触发。这个实验回答的是"当前哪个组件最有价值"，不能单独证明"瓶颈在哪里"。真正定位瓶颈，要先看失败记录和归因：任务没说清楚、上下文不足、环境不可复现、验证反馈缺失，还是状态管理断裂；组件拆除结果只能作为辅助证据。

一个团队的真实经历

一个团队用 GPT-4o 开发一个 TypeScript + React 前端应用（约 20,000 行代码）。他们经历了四个阶段，其实就是在一件一件地添置 harness 的组件：

阶段 1：只有 README 里的基本项目描述。5 次运行成功 1 次（20%）。主要失败：选错了包管理器（npm vs yarn）、没遵循组件命名约定、跑不了测试。

阶段 2：添加 `AGENTS.md`，写明技术栈版本、命名约定、关键架构决策。成功率升到 60%。剩余失败主要来自环境问题和验证缺失。

阶段 3：在 `AGENTS.md` 里列出验证命令 `yarn test && yarn lint && yarn build`。成功率升到 80%。

阶段 4：引入进度文件模板，agent 在每次运行中记录完成和未完成的工作。成功率稳定在 80-100%。

四次迭代，模型一个字没改，成功率从 20% 到接近 100%。你没有换更好的模型，变的只是 harness。

核心要点

- Harness = 指令 + 工具 + 环境 + 状态 + 反馈，五个子系统缺一不可。
- 不是模型权重的部分全是 harness，你的 harness 决定了模型能力能被发挥多少。
- 五个子系统中，反馈子系统通常是投入最少、回报最高的。先把验证命令写清楚。
- 用"控制变量排除法"量化各子系统的边际贡献；定位真正瓶颈要靠失败记录和归因，不能只靠拆除实验。
- Harness 和代码一样会腐化，定期审计，像还技术债一样还 harness 债。

延伸阅读

- OpenAI: Harness Engineering

- Anthropic: Effective Harnesses for Long-Running Agents
- HumanLayer: Harness Engineering for Coding Agents
- SWE-agent: Agent-Computer Interfaces
- Thoughtworks: Harness Engineering on Technology Radar

练习

1. **Harness 五元组审计**：拿你正在用 AI agent 的项目，按五元组框架做一个完整审计。每个子系统打 1-5 分。找出最低分的那个子系统，花 30 分钟改进它，然后观察 agent 的表现变化。

2. **等模型对照下的组件价值实验**：选一个模型和一个有挑战性的任务。依次移除指令（删掉 AGENTS.md）、移除反馈（不给验证命令）、移除状态（不提供进度文件），每次只移除一个，测量性能下降幅度。基于结果，排出各子系统在当前任务里的边际价值；如果要找瓶颈，还必须同时记录失败日志并做原因归因。

3. **可供性分析**：找一个 agent 在你的项目中"想做但做不了"的场景（比如知道要用参数化查询但不知道你项目的 ORM 怎么写）。分析这是执行鸿沟（不知道怎么操作）还是评估鸿沟（不知道做得对不对），然后设计 harness 改进来弥补。

English Version →

> 本篇代码示例：code/ 实战练习：Project 02. 让 agent 看懂项目、接住上次的工作

第三讲. 让代码仓库成为唯一的事实来源

你团队的架构决策散落在 Confluence、Slack、Jira、和几个资深工程师的脑子里。对人类来说这勉强够用，你可以问同事、搜聊天记录、翻文档，实在不行还能去茶水间堵人。但对 AI agent 来说，不在仓库里的信息等于不存在。

这不是夸张。Agent 的输入只有三样东西：系统提示和任务描述、仓库里的文件内容、工具执行的输出。你的 Slack 历史、Jira 工单、Confluence 页面、和周五下午跟同事聊的架构决定，agent 全都看不到。它不能"去问一下"，也不能"搜一下聊天记录"。它的整个工作世界就是仓库本身，仓库外面的事它一概不知。

所以问题变成了：你给不给它一张足够好的地图？

地图上该画什么

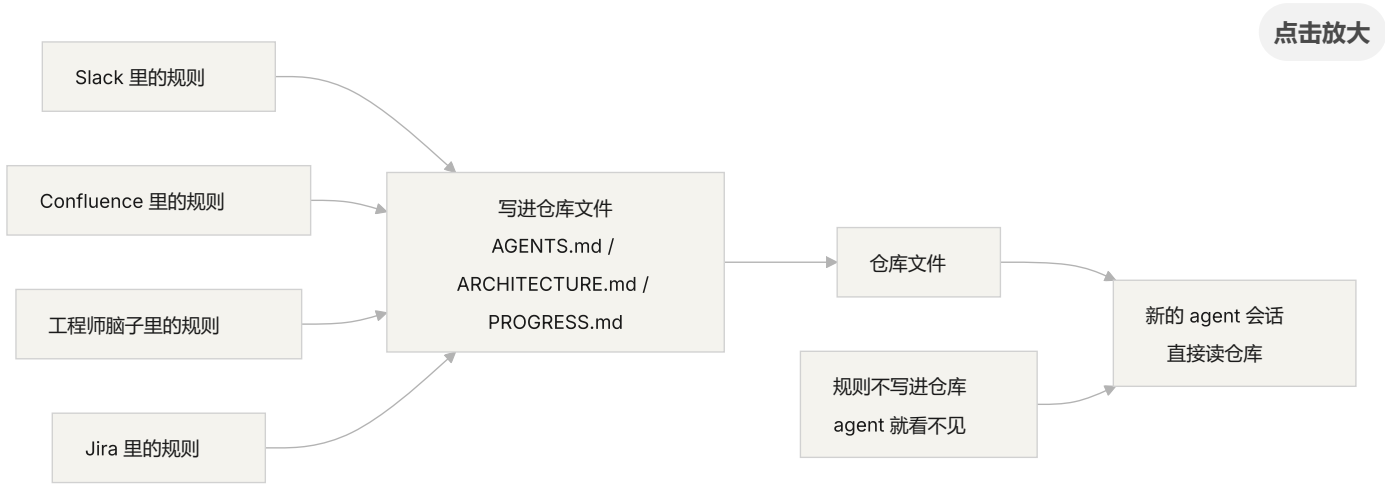
OpenAI 在他们的 harness engineering 文章里把这个问题说得非常直白：**仓库里不存在的信息，对 agent 来说等于不存在。** 他们把这称为"仓库即规范"原则，即仓库本身就是最高权威的规范文档。

Anthropic 的 long-running agents 文档也强调了类似的观点：持久化状态是长任务连续性的必要条件，跨会话的知识可恢复性直接决定了任务成功率。而这些状态必须存在于仓库中，因为那是 agent 唯一稳定可访问的存储。

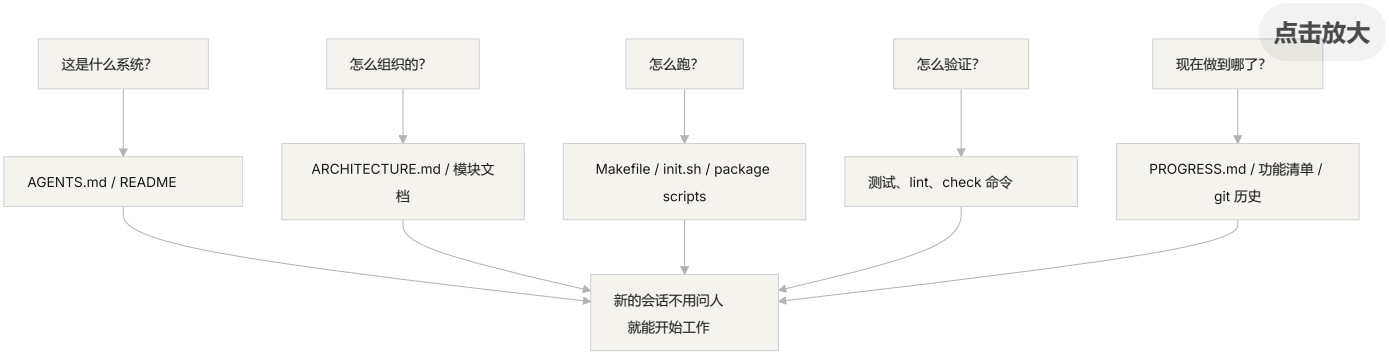
你可能会想："我们团队人少，知识都在大家脑子里，这不也工作得好好的吗？"没错，对人类来说确实可以。但你要用 agent，就得接受一个事实：agent 不能问人。所有它需要知道的东西，都必须写下来，放在它能找到的地方。

这不是"写更多文档"的问题，而是"把决策信息放到正确的位置"的问题。一份在 `src/api/` 目录下、50 行的 `ARCHITECTURE.md`，比一份在 Confluence 里、500 页但没人维护的设计文档有用得多。就近放置的价值远大于篇幅长短，因为信息只有在你需要的时候就在手边，才能真正发挥作用。

知识可见性



怎么检验你的地图画得够不够好？做一个"全新会话测试"：开一个全新的 agent 会话，只让它看仓库内容，看它能不能回答五个基本问题。



如果它答不上来，说明地图上有空白。空白的地方，agent 就得自己猜，猜错了就是 bug，猜多了就浪费上下文。每个新会话都要猜一遍，猜的成本远高于一开始就把地图画好。

核心概念

- **知识可见性缺口**：项目总知识中不在仓库里的比例。缺口越大，agent 失败的概率越高。你可以这样估算：把脑子里关于这个项目的隐性知识全算上，再看有多少写进了仓库，两者的差距就是可见性缺口。
- **系统记录（System of Record）**：代码仓库作为项目决策、架构约束、执行状态和验证标准的权威信息源。仓库说了算，别的地方说了不算。如果"此路不通"这个信息只在老张的脑子里，那每次都得问老张。写进仓库，谁都不用问。

- **全新会话测试**：上一节说的五个问题。能回答几个，你的地图就画了几分。
- **发现成本**：agent 为了在仓库里找到一条关键信息需要消耗多少上下文。信息放得越隐蔽，发现成本越高，留给实际任务的预算越少。关键信息应该放在 agent 最先看到的位置，而不是藏在十层目录深处。
- **知识衰减率**：仓库中单位时间内变得过时的知识条目比例。文档和代码脱节是最大的敌人，比没有文档更危险的是过时的文档。
- **ACID 类比**：把数据库的事务管理原则（原子性、一致性、隔离性、持久性）用到 agent 的状态管理上。后面会展开讲。

怎么画好这张地图

原则 1：知识靠近代码。 一条关于 API 端点认证的规则，应该放在 API 代码旁边，而不是藏在一个巨大的全局文档里。每个模块目录下放一个简短的文档，说清楚这个模块的职责、接口和特殊约束。模块目录本身就是天然的索引，agent 读到代码就能读到约束，不用到处翻找。

原则 2：用标准化的入口文件。 `AGENTS.md`（或 `CLAUDE.md`）是 agent 的"着陆页"。它不需要包含所有信息，但必须能让 agent 快速回答"这是什么项目"、"怎么跑"、"怎么验证"这三个问题。50-100 行就够了。

原则 3：最小但完备。 每条知识都应该有明确的使用场景。如果你删掉某条规则不影响 agent 的决策质量，那这条规则就不应该存在。但全新会话测试中的每个问题都必须有答案。这是一个需要持续调整的平衡，不多不少，刚好够用。

原则 4：和代码一起更新。 把知识更新跟代码变更绑定在一起。最简单的方法：把架构文档放在对应的模块目录里。改代码的时候自然会看到文档，改代码之后 CI 提醒你检查文档是否需要更新。

具体的仓库结构：

```
project/
├── AGENTS.md                # 入口：项目概览、运行命令、硬约束
├── src/
│   ├── api/
│   │   ├── ARCHITECTURE.md  # API 层的架构决策
│   │   └── ...
│   ├── db/
│   │   ├── CONSTRAINTS.md   # 数据库操作的硬约束
│   │   └── ...
│   └── ...
├── PROGRESS.md              # 当前进度：做了什么、在做什么、被什么阻塞
└── Makefile                 # 标准化的操作命令：setup、test、lint、check
```

用 ACID 原则管理 agent 状态

这个类比来自数据库的事务管理。你可能会觉得这是在把简单的事情搞复杂，但它确实提供了一个非常实用的框架：

- **原子性**：每次"逻辑操作"（比如"添加新端点并更新测试"）用一个 git commit 原子化。中途挂了就 `git stash` 回滚。要么全做，要么不做，没有"做了一半"。
- **一致性**：定义"一致状态"的验证谓词，比如所有测试通过、lint 无报错。Agent 每次操作后跑验证，不一致的中间状态不要 commit。操作完系统应该处于可验证的正确状态。
- **隔离性**：多个 agent 并发工作时，状态文件要避免竞争条件。简单方案：每个 agent 用独立的进度文件，或者用 git 分支隔离。并发写入同一文件是出问题的常见原因。
- **持久性**：关键的项目知识用 git 跟踪的文件持久化。临时状态可以只在会话内存里，但跨会话必须的知识必须写到文件里。脑子里的不算，写在纸上的才算。

OpenAI：让仓库成为知识的事实来源

OpenAI 在用 Codex 开发内部产品时，把架构说明、设计决策、执行计划和进度保存在版本化的仓库文件中。聊天和外部文档里的知识，只有变成 agent 工作上下文中可访问的材料，才能参与它的决策。团队用结构化的 docs 目录组织这些知识，并检查文档是否过时、链接是否完整。

这是原文记录的工程实践，没有给出仓库文档提升任务成功率的对照实验比例。

OpenAI

OpenAI 报告：一个由 3 名工程师驱动 Codex 的小团队，在 5 个月内开出并合入约 1,500 个 PR，平均每名工程师每天产出 3.5 个 PR；团队后来增至 7 人。这是整套工程流程的实际产出，原文没有单独测量仓库文档的贡献。 OpenAI: Harness engineering

---

核心要点

- 不在仓库里的知识对 agent 来说等于不存在。把关键决策信息放进仓库是最基本的 harness 投资，画好地图才不会迷路。
- 用"全新会话测试"检验仓库质量：全新会话能不能只看仓库回答五个基本问题。
- 知识要靠近代码、最小但完备、跟代码一起更新。不是写更多文档，是把信息放到正确的位置。
- 用 ACID 原则管理 agent 状态：原子提交、一致性验证、隔离并发、持久化关键知识。
- 知识衰减是最大敌人。过时的文档比没有文档更危险，它会让 agent 走错方向还以为自己是对的。

延伸阅读

- OpenAI: Harness Engineering
- Anthropic: Effective Harnesses for Long-Running Agents
- Infrastructure as Code — Martin Fowler
- ADR: Architecture Decision Records
- The Twelve-Factor App

练习

1. **全新会话测试**：在你的项目里开一个全新的 agent 会话（不提供任何口头上下文），只让它看仓库内容，然后问它五个问题：这是什么系统？怎么组织的？怎么运行？怎么验证？现在进度如

何？记录它答不上来的问题，然后改进仓库让它能答上来。

2. **知识外置化量化**：列出你的项目中所有对开发工作重要的决策和约束。标注每个条目是在仓库内还是仓库外。算一下你的知识可见性缺口有多大（不在仓库里的占总数的比例）。制定计划把缺口降到 10% 以下。

3. **ACID 准则评估**：用本讲的 ACID 类比评估你的项目状态管理。原子性：agent 的操作能不能干净地回滚？一致性：仓库有没有"一致状态"的验证？隔离性：多 agent 并发时会不会互相踩脚？持久性：跨会话的知识是不是都持久化了？

English Version →

> 本篇代码示例：code/ 实战练习：Project 02. 让 agent 看懂项目、接住上次的工作

第四讲. 把指令拆分到不同文件里

你开始认真对待 harness 了，这很好。你建了个 `AGENTS.md`，把能想到的所有规则、约束、历史教训都塞了进去。一个月后这个文件膨胀到了 300 行，两个月 450 行，三个月 600 行。然后你发现 agent 的表现反而变差了：改一个小 bug，agent 花大量上下文处理无关的部署指令；关键的安全约束埋在第 300 行，被直接忽略了；文件里有三条互相矛盾的代码风格规则，agent 每次随机选一条。

这就是"巨型指令文件"陷阱。觉得什么都有用，什么都往里装，结果想找一条具体规则得把整个文件翻一遍。写了 600 行，但真正跟当前任务相关的可能只有三分之一。

问题的根源：一个恶性循环

最常见的恶性循环是这样的：agent 犯了个错，你说"加条规则防止这个"，加到 AGENTS.md，暂时管用。然后 agent 又犯了另一个错，再加一条。重复下去，文件膨胀到不可控。

这其实是很自然的反应，每次出问题就"加条规则"感觉很合理。但累积效应是灾难性的。让我们看看具体出了什么问题。

上下文预算被吃掉了。 Agent 的上下文窗口是有限的。假设你的 agent 有 200K tokens 的窗口（Claude 的标准），一个膨胀的指令文件可能占掉 10-20K。看起来还有不少余量？但一个复杂的任务可能需要读几十个源文件，工具执行的输出也占上下文，对话历史也在累积。到真正需要理解代码的时候，预算已经不够了。

中间迷失。 "Lost in the Middle"这篇论文（Liu et al., 2023）清楚地证明了：LLM 对长文本中间部分的信息利用效率显著低于两端。你的 AGENTS.md 有 600 行，第 300 行写的是"所有数据库查询必须用参数化查询"，这是安全硬约束。但它被埋在中间，agent 几乎一定会忽略它。

优先级冲突。 文件里混合了不可违反的硬约束（"不得使用 eval()"）、重要的设计指导（"优先使用函数式风格"）、和某个特定场景的历史教训（"上周修了一个 WebSocket 内存泄漏，注意类似的模式"）。这三条规则的重要性完全不同，但在文件里看起来一模一样。Agent 没有可靠的信号来区分哪个是红线，哪个只是建议。

维护衰减。 大文件天生难维护。指令过时了没人删，因为删除的后果不确定（"也许别的地方依赖这条规则？"），但加新指令是无成本的。结果文件只增不减，信噪比持续下降。这和软件里的技术债务积累是同一个问题。

矛盾累积。 不同时期加的指令之间开始出现矛盾：一条说"用 TypeScript 严格模式"，另一条说"某些遗留文件允许用 any"。Agent 每次随机选一条遵循。

核心概念

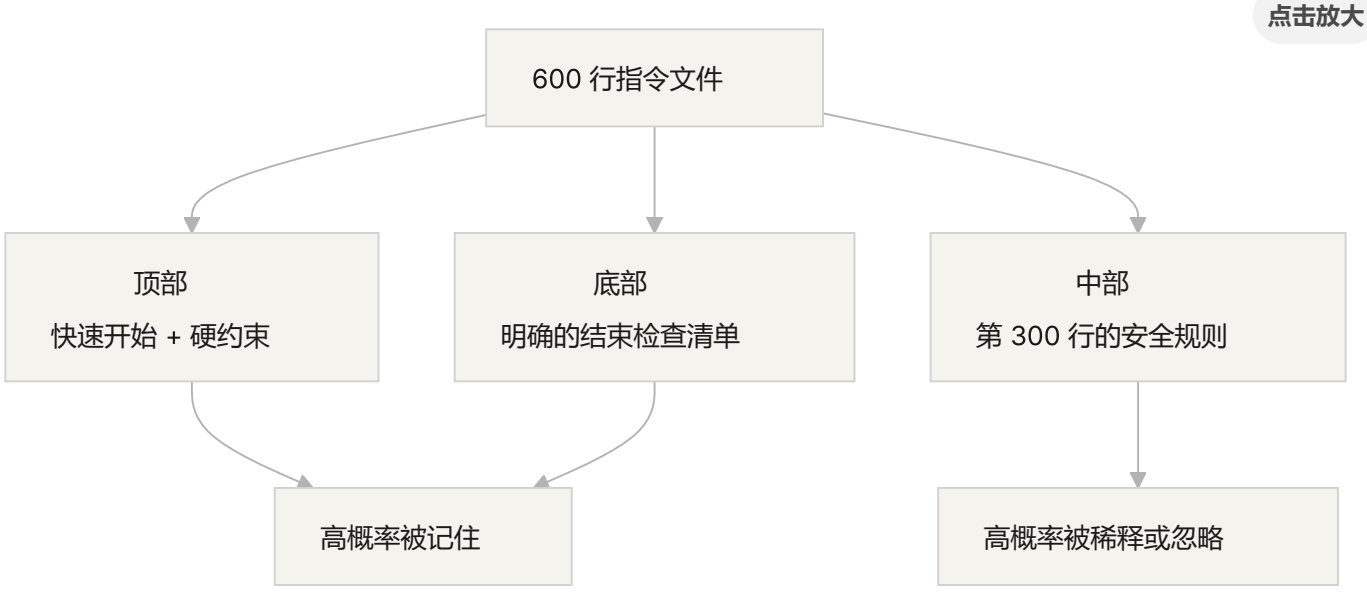
- **指令膨胀**：指令文件一旦占到上下文窗口的 10-15%，就开始挤占代码阅读和任务推理的预算。一个 600 行的 `AGENTS.md` 可能占用 10,000-20,000 tokens，对 128K 的窗口来说就是 8-15%。
- **长文本中间信息容易被忽略**：Liu 等人 2023 年的研究表明，LLM 对长文本中间部分的信息利用效率明显低于两端。埋在 600 行文件第 300 行的关键约束，被忽略的概率非常高。
- **指令信噪比（SNR）**：文件中与当前任务相关的指令占总指令的比例。做 bug 修复时被要求读 50 行部署指令，SNR 就很低。
- **入口文件**：短小的入口文件，作用是引导 agent 去找更详细的文档，而不是自己包含所有内容。50-200 行就够了。
- **按需展开**：先给概要信息，需要的时候再给详细信息。好的 harness 设计和好的 UI 设计一样，不把所有选项一次性砸到用户脸上。
- **分不清轻重**：当所有指令以相同格式和位置呈现时，agent 分不清哪些是不可违反的硬约束，哪些只是建议性的软约束。

指令文件架构

点击放大



点击放大



拆分思路

核心原则：常用信息放手边，偶尔用的收起来，用不上的别带。

入口文件 `AGENTS.md` 控制在 50-200 行，只放最常用的东西：项目概览（一两句话说清楚这是什么）、首次运行命令（`make setup && make test`）、全局硬约束（不超过 15 条不可违反的规则）、指向专题文档的链接（一行描述加适用条件）。

```markdown
# AGENTS.md

## 项目概览
Python 3.11 FastAPI 后端，PostgreSQL 15 数据库。

## 快速开始
- 安装：`make setup`
- 测试：`make test`
- 完整验证：`make check`

## 硬约束
- 所有 API 必须走 OAuth 2.0 认证
- 所有数据库查询必须用 SQLAlchemy 2.0 语法
- 所有 PR 必须通过 pytest + mypy --strict + ruff check

## 专题文档
- API 设计规范（`docs/api-patterns.md`）— 添加新端点时必读
- 数据库操作约束（`docs/database-rules.md`）— 涉及数据库修改时必读
- 测试标准（`docs/testing-standards.md`）— 编写测试时参考
```

每个专题文档 50-150 行，按主题放在 `docs/` 目录下或对应模块目录旁。Agent 只在需要时才去读。用收纳袋整理行李的思路：内衣一个袋，洗漱一个袋，充电器一个袋，找东西不用翻整个箱子。

还有些信息直接放在代码里更合适，比如类型定义、接口注释、配置文件里的说明。Agent 读代码的时候自然能看到，不用再在指令里重复一遍。

每条指令都应该标明来源（"为什么加这条规则？"）、适用条件（"这条规则在什么时候需要？"）、过期条件（"什么情况下可以删掉这条规则？"）。定期审计，删掉过时的、冗余的、矛盾的条目。像管理代码依赖一样管理你的指令，用不上的依赖就该删掉，不然它们只会拖慢系统。

如果某条指令必须在入口文件里，放顶部或底部，不要放中间。"中间迷失"效应告诉我们，LLM 对长文本中间部分的信息利用效率显著低于两端。但更好的做法是把指令放到专题文档里，让 agent 按需加载。

OpenAI 和 Anthropic 都隐性支持拆分的做法。OpenAI 说入口文件应"短小且以路由为导向"，Anthropic 说长运行 agent 的控制信息应"简洁且高优先级"。两家都在说同一件事：别把什么都塞进一个文件里。

OpenAI：简短入口加文档导航

OpenAI 尝试过一个巨型 AGENTS.md，发现它挤占任务上下文、混淆优先级、积累过时规则，也难以检查。后来团队用约 100 行的入口文件作为地图，把详细知识放进结构化的 docs 目录，并用专门的 lint 和 CI 检查维护知识库。原文没有报告这次调整前后的任务成功率或安全约束遵循率。
OpenAI: Harness engineering

效果取决于内容和任务。ETH Zurich 的研究发现，在其评估设置中，仓库上下文文件并未普遍提高任务成功率，却使推理成本增加了 20% 以上，因而建议人工编写的要求保持精简。文件变短不等于效果必然变好，需要在实际任务上验证。 ETH Zurich: Evaluating AGENTS.md

一项配对研究使用 gpt-5.2-codex，在 10 个仓库的 124 个 PR 派生任务上，对比相同任务和仓库快照有无 AGENTS.md。表 1 中，中位耗时从 98.57 秒降到 70.34 秒，降低 28.64%；输出 token 中位数从 2,925 降到 2,440，降低 16.58%。任务最多改动 100 行、5 个文件。这测量的是效率，而非拆分巨型文件的效果；完整功能正确性验证不在研究范围内。 Lulla et al., Table 1

核心要点

- "加条规则"是短期的止痛药，长期的毒药。每次加规则前想想，这条规则放专题文档是不是更合适。
- 入口文件是路由器，不是百科全书。50-200 行，只放概览、硬约束和链接。
- 利用"中间迷失"效应：重要信息放文件顶部或底部，不重要的移到专题文档。
- 像管理技术债一样管理指令膨胀。定期审计，每条指令要有来源、适用条件和过期条件。
- 拆分之后信噪比提升，agent 把更多上下文预算花在实际任务上，而不是处理无关指令。

延伸阅读

- OpenAI: Harness Engineering
- Anthropic: Effective Harnesses for Long-Running Agents
- Lost in the Middle: How Language Models Use Long Contexts

- HumanLayer: Harness Engineering for Coding Agents
- Nielsen Norman Group: Progressive Disclosure
- ETH Zurich: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?: 仓库上下文文件的实证研究：任务成功率、推理成本，以及保持要求精简的建议。见摘要和结论。
- Lulla et al., Table 1

练习

1. **信噪比审计**：拿你当前的入口指令文件，列出所有指令条目。选 5 个不同的常见任务类型，标注每条指令是否跟该任务相关。计算每个任务类型的 SNR。那些对大多数任务都是噪声的指令，移到专题文档里。
2. **按需展开重构**：如果你有一个超过 300 行的指令文件，把它拆成：(a) 不超过 100 行的入口文件，(b) 3-5 个专题文档。重构前后各跑同一组任务（至少 5 个），对比成功率。
3. **中间迷失验证**：在一个长指令文件里，把一条关键约束分别放在顶部、中间、底部各跑一组任务（每组至少 5 次），看遵循率有没有差别。你可能会惊讶于位置效应有多大。

English Version →

> 本篇代码示例：code/ 实战练习：Project 03. 让 agent 关掉再打开还能接着干

第五讲. 让跨会话的任务保持上下文连续

你让 Claude Code 帮你实现一个完整的功能，它跑了 30 分钟，做了大部分工作，但上下文快满了。你开个新会话继续，然后发现：它不记得上次做了什么决策、为什么选了方案 A 而不是方案 B、哪些文件已经改过、测试跑到什么状态了。它得花 15 分钟重新探索一遍项目，而且可能跟上次的做法不一致。

这是 AI coding agent 在跨会话任务中面对的真实困境。本节课讲为什么 agent 会"断片"，以及如何通过结构化的状态持久化，让新会话能快速接上上次的工作。

上下文窗口：不是无限的

上下文窗口是有限的。这不是一个可以通过模型升级解决的问题，即使窗口大小增长到 1M tokens，复杂任务依然会用完。因为 agent 不只是在生成代码，它还要理解代码库、跟踪自己的决策历史、处理工具输出、维护对话上下文。这些信息加起来增长得比窗口扩容快得多。

更深层的问题在于，agent 产生的信息不是均匀重要的。中间推理步骤包含决策的"为什么"，比如为什么选了方案 A 而不是方案 B，为什么用了这个库而不是那个库，为什么跳过了某个优化。最终输出只包含"是什么"，即代码本身。压缩策略通常保留后者但丢了前者。下一个会话看到代码却不知道为什么这么写，可能会"优化"掉一个有意为之的设计决策。

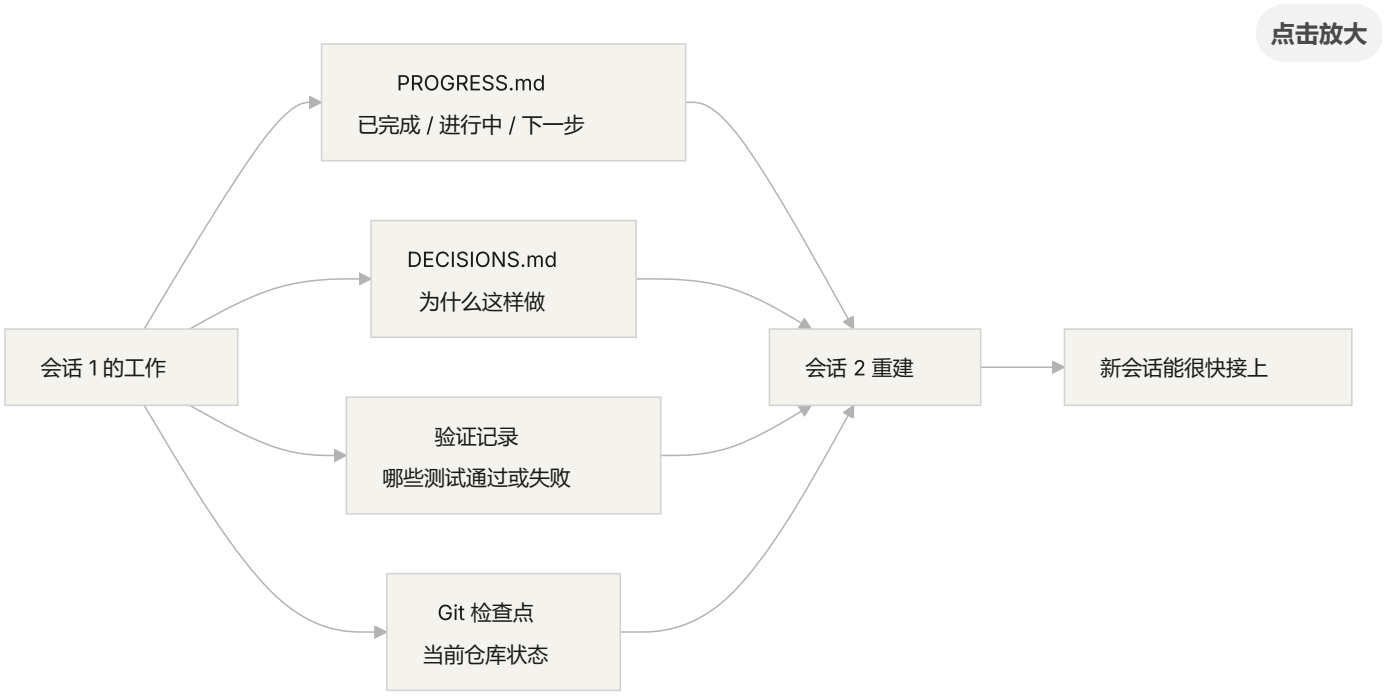
Anthropic 在长运行 agent 研究中观察到一个有意思的现象：当 agent 感觉上下文快满了，它们会表现出"赶工收尾"的行为，匆忙结束当前工作，跳过验证步骤，或者选一个简单的方案而不是最优方案。Anthropic 把这叫"上下文焦虑"。

会话连续性流程

没有状态持久化文件的时候，每个新会话都要重新摸索：



有状态持久化文件的时候，新会话能快速接上：



核心概念

- **上下文窗口是有限的**：不管模型宣称多大的窗口（128K、200K、1M），长任务总会用完。用完之后要么压缩（丢信息），要么重置（开新会话），两种方式都会丢东西。
- **状态持久化文件**：持久化的状态文件，让新会话能无歧义地恢复到上次离开的地方。最基本的形式包括进度日志、验证记录和下一步行动。
- **重建成本**：新会话恢复到可执行状态所需的时间。好的 harness 会把进度、验证结果和下一步写进文件，让新会话从写下来的状态开始。
- **漂移（Drift）**：agent 的理解跟代码仓库实际状态之间的偏差。每次会话边界都会引入漂移，不加控制会越漂越远。
- **上下文焦虑**：Anthropic 观察到的现象，agent 在接近上下文限制时表现异常，过早结束任务以避免信息丢失，本质上是一种非理性的资源焦虑。
- **压缩 vs 重置**：压缩是在同一个会话里把上下文摘要化，保留"是什么"但可能丢了"为什么"；重置是开新会话从持久化状态重建，状态干净但依赖工件的完备性。

连续性断了以后会发生什么

上个会话花了很多上下文预算分析了三种方案的优劣，最终选了方案 B。这个会话的 agent 不知道这个分析过程，可能基于不完整的信息重新做了决策，而且可能选了方案 A。同样的信息，不同的结论，因为做决策的上下文丢了。

更要命的是重复劳动。Agent 不确定某项工作是否已完成，重新做了一遍。或者更糟，做了一半发现跟已有的实现冲突，需要返工。在没有进度记录的情况下，新会话完全不知道哪些工作已经有人做过了。

几个会话累积下来，实现方向可能已经悄悄偏离了原始需求。每个新会话对项目目标的理解都略有偏差，偏差一层层叠加，最终的结果可能跟最初的意图相去甚远。

还有验证缺口。上个会话的验证结果（哪些测试通过、哪些失败、为什么失败）没有记录，新会话得重新跑一遍验证才能了解当前状态。每次都重新诊断，每次都浪费宝贵的上下文。

OpenAI 和 Anthropic 都在他们的文档里强调了结构化状态持久化的重要性。OpenAI 的 harness engineering 文章把仓库当作"操作记录"，每次操作的结果都应该在仓库里留下可追溯的痕迹。Anthropic 的 long-running agents 文档则更具体地建议使用"交接文件"，包含当前状态、已知问题和下一步行动的结构化文档。

状态持久化的实践方法

核心思路：**把 agent 当成一个每次会话都会清空短期记忆的工程师来管理。** 每次它要"下班"之前，必须把关键信息写下来，让下一个"接班"的 agent 能快速上手。

工具 1：进度文件（PROGRESS.md）。这是最基本的状态持久化文件：

```markdown
# 项目进度

## 当前状态
- 最新 commit: abc1234 (feat: add user preferences endpoint)
- 测试状态: 42/43 通过 (test_pagination_edge_case 失败)
- Lint: 通过

## 已完成
- [x] 用户模型和数据库迁移
- [x] 基础 CRUD 端点
- [x] 认证中间件集成

## 进行中
- [ ] 分页功能 (90% - 边界条件测试失败)

## 已知问题
- test_pagination_edge_case 在空结果集时返回 500
- 需要确认是否要在列表中包含已删除用户

## 下一步
1. 修复分页边界条件 bug
2. 添加"是否包含已删除用户"的查询参数
3. 更新 API 文档
```

工具 2：决策日志（DECISIONS.md）。记录重要的设计决策和原因。不需要详细的设计文档，只需要"什么决策、为什么、什么时候做的"：

```markdown
# 设计决策

## 2024-01-15: 使用 Redis 缓存用户偏好
- 原因: 读取频率高（每次 API 调用都需要），数据量小
- 否决方案: 用 PostgreSQL 物化视图（变更频率高，物化视图维护成本不划算）
- 约束: 缓存 TTL 设为 5 分钟，写入时主动失效
```

工具 3：git 提交作为检查点。 每完成一个原子工作单元就提交，commit message 要说清楚做了什么和为什么。这是免费的、自动版本化的状态快照。

工具 4：init.sh 或 harness 的初始化流程。 在 `AGENTS.md` 里写明每次"上班"和"下班"的流程：

```markdown
## 每次会话开始时（上班）
1. 读 PROGRESS.md 了解当前状态
2. 读 DECISIONS.md 了解重要决策
3. 跑 make check 确认仓库处于一致状态
4. 从 PROGRESS.md 的"下一步"部分继续工作

## 每次会话结束前（下班）
1. 更新 PROGRESS.md
2. 跑 make check 确认一致状态
3. 提交所有已完成的工作
```

混合策略：不需要每次都重置上下文。短任务（30 分钟以内）可以在同一个会话里完成，长任务（跨会话）必须用进度文件和决策日志来维持连续性。判断标准：如果任务需要的上下文超过窗口的 60%，就开始准备交接。

上下文焦虑的深层分析

Anthropic 在 2026 年 3 月发布的研究进一步揭示了上下文焦虑的具体表现：在 Sonnet 4.5 上，当上下文接近窗口限制时，agent 会表现出强烈的"赶工收尾"行为。

针对这个现象，有两种策略：

压缩（Compaction）：在同一个会话里把早期对话摘要化。优点是保留连续性，agent 能看到"是什么"。缺点是"为什么"经常在摘要中丢失，比如为什么选了方案 B 而非 A，为什么跳过了某个优化。更关键的是，压缩并不能消除上下文焦虑，agent 知道上下文曾经很大，心理上仍然倾向于加速收尾。

重置（Context Reset）：完全清空上下文，开一个新会话，从持久化工件重建。优点是干净的心理状态，新会话没有"我快没时间了"的焦虑。缺点是依赖交接工件的完备性，如果进度文件里漏了关键信息，新会话可能在错误方向上浪费时间。

Anthropic 的实际数据：对于 Sonnet 4.5，上下文焦虑足够严重，以至于压缩单独不够用，上下文重置成为 harness 设计的关键组件。但对于 Opus 4.5，这种行为大幅减弱，可以不依赖重置而靠压缩管理上下文。这意味着：**harness 设计需要对目标模型有具体的理解，而不是套用通用模板。**

> 来源：Anthropic: Harness design for long-running application development

Anthropic：跨编码会话的交接

Anthropic 介绍了一个 Claude 网站克隆案例：初始化 agent 将需求展开为 200 多项功能，初始状态都标为未通过，同时创建 init.sh、进度文件和初始 Git 提交。后续编码会话读取进度和 Git 历史，逐步实现功能、验证行为，再为下一次会话留下更新。

这个案例记录的是连续性机制，没有报告此前这段文字中的功能完成率、隐藏缺陷率和上下文重建耗时比例。配套代码用于教学模拟，不是模型性能实测。

Anthropic

LangChain 报告了一组真实的 harness 改进结果：固定使用 gpt-5.2-codex，在包含 89 个任务的 Terminal Bench 2.0 上，得分从 52.8% 提升到 66.5%，提高 13.7 个百分点。改动包括验证指导、中间件和上下文管理。这是多项 harness 调整的整体结果，不能单独归因于进度文件。 LangChain: Improving Deep Agents

核心要点

- 上下文窗口是有限的资源。长任务一定会跨会话，跨会话一定会丢信息，这是客观现实。
- 解决方案不是更大的窗口，而是更好的状态持久化。进度文件、决策日志、git 检查点，三者配合让新会话能接上之前的工作。
- 把 agent 当成每次会话都会清空短期记忆的工程师来管理：每次"下班"前写清楚做了什么、为什么、下一步做什么。
- 重建成本是关键指标。好的 harness 会把进度、验证结果和下一步写进文件，让新会话快速恢复。
- 混合策略：短任务在会话内完成，长任务用结构化工件维持连续性。

延伸阅读

- Anthropic: Effective Harnesses for Long-Running Agents
- OpenAI: Harness Engineering

- Lost in the Middle: How Language Models Use Long Contexts
- Claude Code Documentation
- HumanLayer: Harness Engineering for Coding Agents
- LangChain: Improving Deep Agents

练习

1. **连续性损耗度量**：选一个需要至少 3 个会话的开发任务。不提供任何状态持久化文件，在每个会话开始时记录 agent 花了多少上下文来"搞清楚上次做了什么"。会话结束后，创建进度文件，让下一个会话从进度文件开始。对比有进度文件和没有时的重建成本。
2. **交接模板设计**：设计一个最小化的交接模板，包含四个字段：仓库状态（commit hash）、运行时状态（测试通过率）、阻塞项、下一步行动。让一个全新的 agent 会话只凭这个模板恢复项目状态，记录恢复过程中出现的歧义点，迭代改进模板。
3. **混合策略实验**：在一个包含 5 个会话的开发任务中，对比三种策略：(a) 每次都开全新会话 + 进度文件，(b) 在同一个会话里尽可能多做（上下文压缩），(c) 混合策略（短任务在会话内，长任务跨会话 + 进度文件）。对比重建时间、功能完成率和决策一致性。

English Version →

> 本篇代码示例：code/ 实战练习：Project 03. 让 agent 关掉再打开还能接着干

第六讲. 让 agent 每次工作前先初始化

在使用 AI 编码 agent 时，一个常见的低效模式是：让 agent 直接开始做功能，它上来就写代码，但很快会发现测试框架没配好、环境有问题、项目结构不清晰，大量时间花在了"搞清楚这个项目怎么运作"上面，真正用于写功能的时间反而很少。

更好的做法是，在让 agent 开始干活之前，先用一个独立的阶段把基础环境搭好、验证命令跑通、项目结构搞清楚。初始化工作应该和功能实现分开，它们是两种性质完全不同的任务。

这节课要讨论的，就是为什么初始化必须是独立的阶段，不能跟实现混在一起。

两种不同的工作

初始化和实现的优化目标完全不同。实现阶段的目标是最大化已验证功能的数量和质量，初始化阶段的目标则是最大化后续所有实现的可靠性和效率。

当你把初始化和实现混在一起的时候，agent 面临一个多目标优化问题：它要同时搭基础设施和写功能代码。在没有显式优先级设定的情况下，agent 自然倾向于写代码（因为那是直接可见的产出），而牺牲基础设施（因为它的价值只能在后续会话中体现）。结果就是基础设施没搭牢，功能代码的可靠性也打了折扣。

初始化生命周期



点击放大

初始化与实现同时进行的问题

最直接的问题是基础设施搭不牢。Agent 把大部分精力花在功能代码上，基础设施只是顺手搭一点。测试框架配了但没验证过，lint 规则设了但太宽松，进度文件没创建。这些缺陷在第一个会话里不明显（因为 agent 还记得它做了什么），但到第二个会话就暴露了：新 agent 不知道项目怎么跑、怎么测、做到哪了。

更隐蔽的代价是"未验证的累积"。在测试框架配好之前写的功能代码，等回头补测试的时候可能发现设计上就有问题，早知道的话应该用不同的方式实现。前面写的代码越多，后面需要推翻重来的就越多。

上下文预算也在被浪费。初始化工作（配环境、配测试、理解项目结构）消耗了大量预算，留给实际功能实现的反而不够了。结果第一个会话只完成了一半的功能，第二个会话还得从头理解项目。预算花在了初始化上，但初始化也没做好，两头都没占着。

最容易被忽略的是隐式假设埋下的雷。Agent 在初始化过程中做的决策（用什么测试框架、目录怎么组织、依赖怎么管理）如果不显式记录下来，后续会话就可能做出矛盾的选择。第一个会话选了 Vitest 做测试框架，第二个会话的 agent 不知道，又引入了 Jest，两套测试框架共存，维护成本翻倍。

Anthropic 的长运行 agent 工作把初始化放在独立的第一个会话里：initializer 写 `init.sh`、进度文件、功能清单和初始 commit；后续 coding 会话读取这些状态，一次做一个功能，端到端测试，提交并更新进度。

OpenAI 的 Codex harness engineering 指南也强调"仓库作为操作记录"的原则：第一次运行就要建立清晰的操作结构，否则每次新会话都得重新推断项目约定。

核心概念

- **初始化阶段**：agent 生命周期中的第一个阶段，只建立后续实现所需的执行前提，不做功能开发。它的产出是基础设施，而不是业务代码。
- **启动就绪清单**：一个项目能被全新 agent 会话无歧义操作的条件：能启动、能测试、能看进度、能接手下一步。四个条件缺一不可。
- **从零开始 vs 从模板开始**：从零开始意味着 agent 需要自行推断项目结构，效果差；从模板开始意味着基础设施已经就位，效果好得多。能用模板就用模板。

- **随时可接手**：项目在任何时刻都处于"可以被全新 agent 接手"的状态。不需要口头解释，只看仓库内容就能接着干。
- **从开始到第一次测试通过**：衡量初始化效率的核心指标。时间越短，初始化越高效。
- **后续会话的成功率**：后续会话不需要依赖隐式知识就能成功执行任务的比例，这是初始化质量的最佳衡量标准。

初始化的正确做法

把初始化当作一个独立的阶段来执行。 第一个会话只做初始化，不写任何业务功能代码。初始化的产出是：

1. 可运行的环境。 项目能启动、依赖都装好、没有环境问题。

2. 可验证的测试框架。 至少有一个示例测试能通过，证明测试框架本身是配好的。

3. 启动就绪清单文档。 一个明确的文档告诉后续会话：

```markdown
# 初始化契约

## 启动命令
- 安装依赖：`make setup`
- 启动开发服务器：`make dev`
- 运行测试：`make test`
- 完整验证：`make check`

## 当前状态
- 所有依赖已安装并锁定
- 测试框架已配置（Vitest + React Testing Library）
- 示例测试通过（1/1）
- Lint 规则已配置（ESLint + Prettier）

## 项目结构
- src/ — 源代码
- src/components/ — React 组件
- src/api/ — API 客户端
- tests/ — 测试文件
```

4. 任务分解。 把整个项目拆成有序的任务列表，每个任务有明确的验收标准：

```markdown
# 任务分解

## Task 1: 用户认证基础
- 实现 JWT 认证中间件
- 添加登录/注册端点
- 验收标准：pytest tests/test_auth.py 全部通过

## Task 2: 用户资料页面
- 实现用户资料 CRUD
- 添加资料编辑表单
- 验收标准：pytest tests/test_profile.py 全部通过

## Task 3: 搜索功能
- ...
```

5. Git 提交作为检查点。 初始化完成后提交一个干净的 checkpoint。后续所有工作都从这个 checkpoint 开始。

热启动策略：不要从空目录开始。用一个项目模板（create-react-app、fastapi-template 等）预置好标准的目录结构、依赖配置和测试框架。把通用的初始化步骤预置到模板里，只留下项目特有的初始化工作。

初始化的完成条件：启动就绪清单的四个条件全部满足——能启动、能测试、能看进度、能接手下一步。用这个检查清单验收初始化：

```markdown
## 初始化验收清单
- [ ] `make setup` 从零开始能成功
- [ ] `make test` 至少有一个测试通过
- [ ] 新的 agent 会话能只看仓库回答"怎么跑"和"怎么测"
- [ ] 任务分解文件存在且有至少 3 个任务
- [ ] 所有内容已提交到 git
```

实际案例

一个 React 前端项目的两种初始化方式对比：

混合方式：agent 在第一个会话中同时做了项目脚手架创建和首个功能实现。会话结束时，仓库有可运行的代码，但没有显式的启动和测试命令文档、没有进度跟踪文件、没有任务分解。第二个会话花了约 20 分钟推断项目结构、测试框架和构建流程。

独立初始化：第一个会话只做初始化，用项目模板创建目录结构、配置测试框架（Vitest + React Testing Library）、写一个示例测试并验证通过、创建启动就绪清单文档和任务分解文件、提交初始检查点。第二个会话的重建时间不到 3 分钟，直接从任务列表开始工作。

整个项目周期对比：混合方式的总重建时间（跨所有会话）比独立初始化多约 60%。独立初始化多花的那 20 分钟在后续会话中被成倍收回。前期多投入一点时间把初始化做扎实，后续的效率反而更高。

核心要点

- 初始化和实现的优化目标不同，混在一起只会互相拖后腿。
- 初始化的产出是基础设施：可运行的环境、可验证的测试、启动就绪清单、任务分解。
- 用"启动就绪清单"的四个条件验收初始化：能启动、能测试、能看进度、能接手下一步。
- 热启动优于冷启动，用项目模板预置标准化的基础设施。
- 初始化投入的时间会在后续 3-4 个会话中完全收回，这是前期投资，不是额外成本。

延伸阅读

- Anthropic: Effective Harnesses for Long-Running Agents
- OpenAI: Harness Engineering
- HumanLayer: Harness Engineering for Coding Agents
- Infrastructure as Code — Martin Fowler
- SWE-agent: Agent-Computer Interfaces

练习

1. **启动就绪清单设计**：为你正在开发的项目写一个完整的启动就绪清单。然后开一个全新的 agent 会话，只给它看仓库内容（不给任何口头上下文），让它尝试启动项目、跑测试、了解当前进度。记录它遇到的问题，每个问题都对应启动就绪清单中缺失的一个条款。

2. **对比实验**：选一个中等复杂度的新项目。方式 A 让 agent 初始化和首次实现同时做，方式 B 先花一个会话做独立初始化，第二个会话再开始实现。在 4 个会话后对比首次验证时间、重建成本和功能完成率。

3. **初始化验收清单**：为你的项目设计一个初始化验收清单，让一个全新的 agent 会话逐项执行，记录哪些项通过了、哪些没通过。没通过的项就是 harness 需要补强的地方。

English Version →

> 本篇代码示例：code/ 实战练习：Project 04. 用运行反馈修正 agent 的行为

第七讲. 给 agent 划清每次任务的边界

Agent 经常会越界，同时做太多事情。在一个典型场景中，你让它给项目加上用户认证功能，结果它同时开始改数据库 schema、写路由、改前端组件，还顺手重构了错误处理中间件。两个小时后一看，12 个文件被修改，800 行新代码，但没有一个功能是端到端跑通的。

Agent 天生就有"多做一点"的冲动：看到相关的事情就顺手一起做了。问题是，同时做太多事情，结果往往每一件都做不好。

Anthropic 在 "Effective harnesses for long-running agents" 工程博客中明确指出：当提示太宽泛时，agent 倾向于"同时启动多件事"而非"先做完一件事"。OpenAI 在 Codex 工程实践中也发现，没有显式范围控制的任务，完成率会暴跌。这本质上是一个 harness 设计问题：没有给 agent 划清边界。

注意力是有限的资源

这本质上是一个数学问题。假设 agent 的上下文容量为 C，同时激活 k 个任务，每个任务平均获得 C/k 的推理资源。当 C/k 低于完成单个任务所需的最小阈值时，所有任务都做不完。

Claude Code 的真实行为很说明问题。你让它"添加用户注册功能"，它很可能这样做：

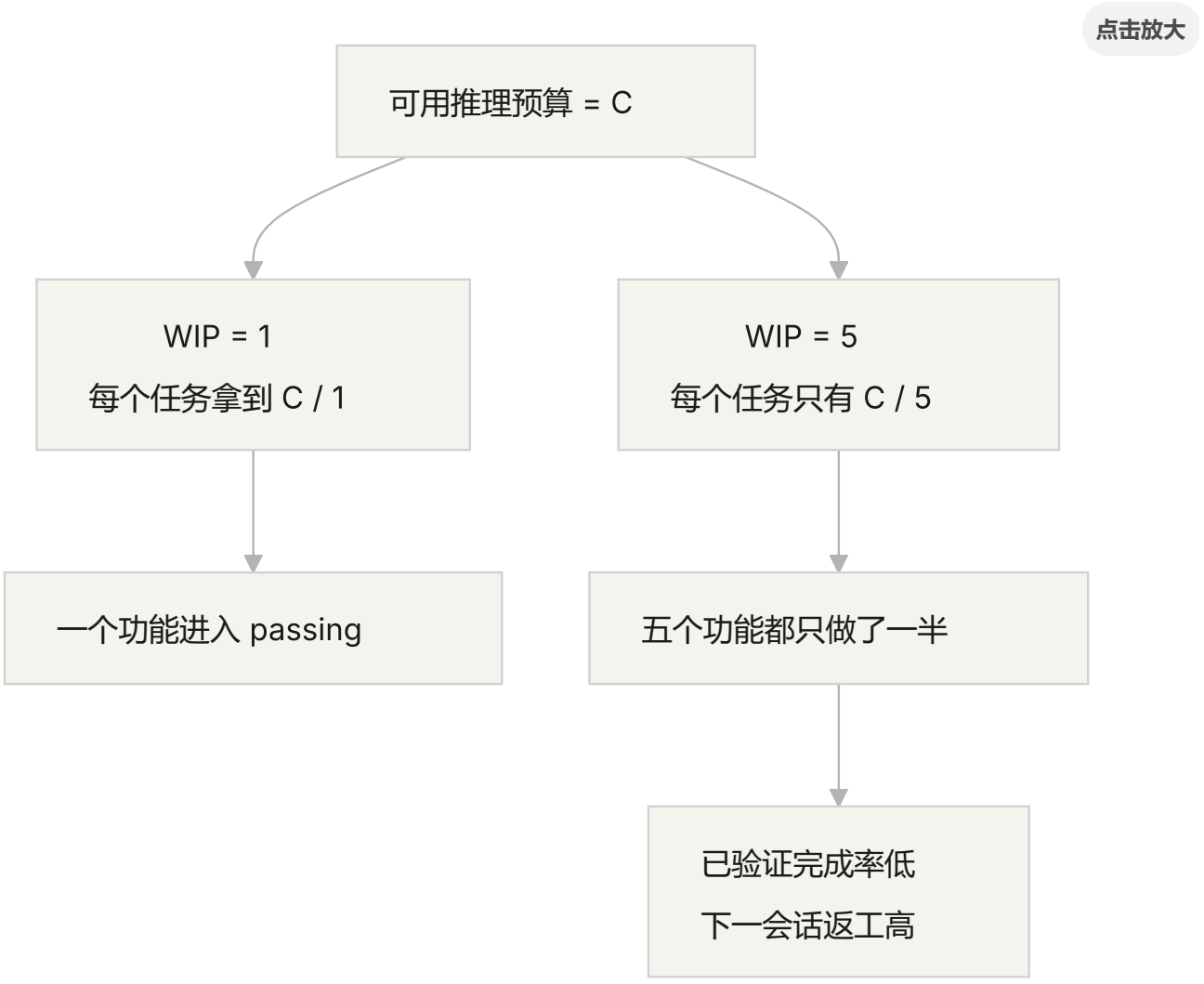
1. 创建 User model
2. 写注册路由
3. 发现需要邮箱验证，于是加邮件服务
4. 看到密码需要加密，于是引入 bcrypt
5. 注意到错误处理不统一，于是重构全局错误中间件
6. 看到测试文件结构不清晰，于是重组目录结构

6 步之后，每一个都是半成品。没有端到端验证，代码之间耦合复杂，下一个会话来接手时会一脸懵。

Anthropic 的长运行 agent 指南要求 coding agent 一次只做一个功能，验证、提交、更新进度，再继续下一个。宽泛的 prompt 会让 agent 同时开几条线，每条线都在争抢同一份上下文和验证预算。

WIP=1 工作流





核心概念

- **过度延伸（Overreach）**：agent 在一次会话中激活的任务数量超过最优值。这是可以量化的：同时做 5 个功能但 0 个跑通，就是 overreach。
- **不足完成（Under-finish）**：已启动的任务中，通过端到端验证的比例低于阈值。写了代码但没跑通测试，就是 under-finish。
- **WIP 限制（Work-in-Progress Limit）**：来自 Kanban 方法论，核心思想是限制同时在进行的任务数量。对于 agent，WIP=1 是最安全的默认值，做完一个再做下一个。
- **完成证据（Completion Evidence）**：一个任务从"进行中"变成"已完成"必须满足的可验证条件。没有这个，agent 会用"代码看起来没问题"代替"行为通过测试"。
- **范围表面（Scope Surface）**：一个 DAG 结构，每个节点是一个工作单元，边是依赖关系。状态只有四种：未开始、进行中、阻塞、已通过。
- **完成压力（Completion Pressure）**：harness 通过 WIP 限制和完成证据要求共同产生的约束力，迫使 agent 先完成当前任务再开始新任务。

过度延伸与不足完成

这两个问题互相加剧。overreach 导致注意力分散，注意力分散导致 under-finish，under-finish 留下的半成品代码又增加了系统复杂度，进一步导致下一个任务的 overreach，形成恶性循环。

用 Kanban 的语言说：Little 法则告诉我们 L = lambda * W。如果在制品数量 L 过大（同时做太多事），每个任务的前置时间 W 必然增加。对 agent 来说，这意味着每个功能从开始到验证通过的时间被拉长，失败概率被放大。

这在人类世界也是老问题了。Steve McConnell 在《Rapid Development》中记录，范围蔓延是项目失败的首要原因。但人类至少有"我已经做得够多了"的直觉，agent 完全没有。生成下一个想法的成本对模型来说太低了，写一行"顺便把这个也改了"几乎不消耗额外 token，但每个额外的修改都会稀释 agent 的注意力。

实施方法

1. 强制 WIP=1

这是最直接有效的方法。在你的 harness 里，明确告诉 agent：**任何时刻只允许一个任务处于"进行中"状态。** 在 Claude Code 的 CLAUDE.md 或 Codex 的 AGENTS.md 里写：

```
## 工作规则
- 每次只做一个功能点
- 当前功能点端到端验证通过后，才能开始下一个
- 不要在实现功能 A 时"顺便"重构功能 B
```

2. 给每个任务定义显式的完成证据

完成指的是"行为验证通过了"。在你的功能列表里，每个条目都要有验证命令：

```
F01: 用户注册
  验证: curl -X POST /api/register -d '{"email":"test@example.com","password":"123
  状态: passing
```

3. 把范围表面外部化

用一个机器可读的文件（JSON 或 Markdown）记录所有任务的状态。任何新会话都能直接读这个文件，知道：哪个任务在做？什么行为算完成？已经通过了什么验证？

4. 监控验证完成率

harness 应该持续跟踪 VCR（Verified Completion Rate）= 已通过验证的任务数 / 已启动的任务数。VCR < 1.0 时，阻止新任务启动。

实际案例

一个 8 个功能点的 REST API 项目，两种策略对比：

无约束模式：agent 在第一个会话同时启动 5 个功能。产出约 800 行代码，涉及 12 个文件。端到端测试通过率只有 20%，只有用户注册跑通了。其余 4 个功能：数据库 schema 建了但缺验证逻辑，路由定义了但返回格式错误。到第 3 个会话结束，8 个功能只完成 3 个。

WIP=1 模式：agent 在第一个会话只做用户注册。产出约 200 行代码，涉及 4 个文件。端到端测试 100% 通过。提交干净的、已验证的实现。到第 4 个会话结束，8 个功能完成 7 个（第 8 个因外部依赖被阻塞）。

结果：总代码量更少（800 行 vs 1200 行），但有效代码更多。完成率 87.5% vs 37.5%。

核心要点

- **WIP=1 是 agent harness 的默认安全设置**：做完一个再做下一个，不要试图并行。
- **完成证据必须是可执行的**："代码看起来没问题"不算完成，"curl 返回 201"才算。
- **范围表面必须外部化为文件**：不能只在对话里说，必须在仓库里有机器可读的记录。
- **overreach 和 under-finish 是共生问题**：解决一个就解决了另一个。
- **"少做但做完"永远优于"多做但做半"**：agent 代码行数和功能完成率呈负相关，质量永远比数量重要。

延伸阅读

- Effective harnesses for long-running agents - Anthropic — Anthropic 工程博客，详细论述了"小下一步"策略
- Harness Engineering - OpenAI — OpenAI 对 harness 工程的完整论述
- Kanban: Successful Evolutionary Change - David Anderson — WIP 限制的经典来源
- Rapid Development - Steve McConnell — 范围蔓延作为项目失败首要原因的实证数据

练习

1. **任务原子化练习**：选一个宽泛需求（如"实现用户管理系统"），把它拆成至少 5 个原子工作单元。每个单元写清楚：(a) 单一行为描述，(b) 可执行的验证命令，(c) 依赖关系。检查是否满足 WIP=1 的约束。

2. **对比实验**：在同一个项目上跑两次，一次不给约束，一次强制 WIP=1。比较验证完成率、总代码行数、有效代码比例。

3. **完成证据审计**：回顾一个最近的 agent 运行结果，把每个代码变更分类为"已完成行为"、"未完成行为"或"脚手架"。给每个未完成行为补充缺失的验证命令。

English Version →

> 本篇代码示例：code/ 实战练习：Project 04. 用运行反馈修正 agent 的行为

第八讲. 用功能清单约束 agent 该做什么

一个常见的场景：让 agent 做一个电商网站，跑完之后它告诉你"做完了"。但你打开代码一看，用户认证有了，但购物车的结算按钮点了没反应，支付流程根本没接上。问题的根源在于：没有告诉过它"做完"的具体标准，所以它用自己的标准来判断——"代码写了不少，看起来挺完整"。

功能清单（feature list）在很多人眼里就是个备忘录，写下来怕忘了，写完扔在一边。但在 harness 的世界里，功能清单是整个 harness 的基础结构。调度器靠它选任务，验证器靠它判完成，交接器靠它生成报告。没有它，这些组件就没有可以依赖的共识。

Anthropic 和 OpenAI 都强调：**工件必须外部化**。功能状态必须是仓库里机器可读的文件，不能是对话里的非结构化描述。

Agent 缺少明确的完成标准

Claude Code 和 Codex 都不会自动知道你心目中的"做完"是什么意思。你说"加一个购物车功能"，模型的理解可能是"写一个 Cart 组件和 addToCart 方法"。而你的意思是"用户能从浏览商品到下单支付完整走通"。

这个理解鸿沟在没有功能清单的情况下会持续存在。agent 用自己的隐式标准判断完成，通常是"代码没有明显的语法错误"。而你需要的是端到端的行为验证。没有清单，双方对"做完"的理解始终是对不上的。

看看这种常见的进度记录：

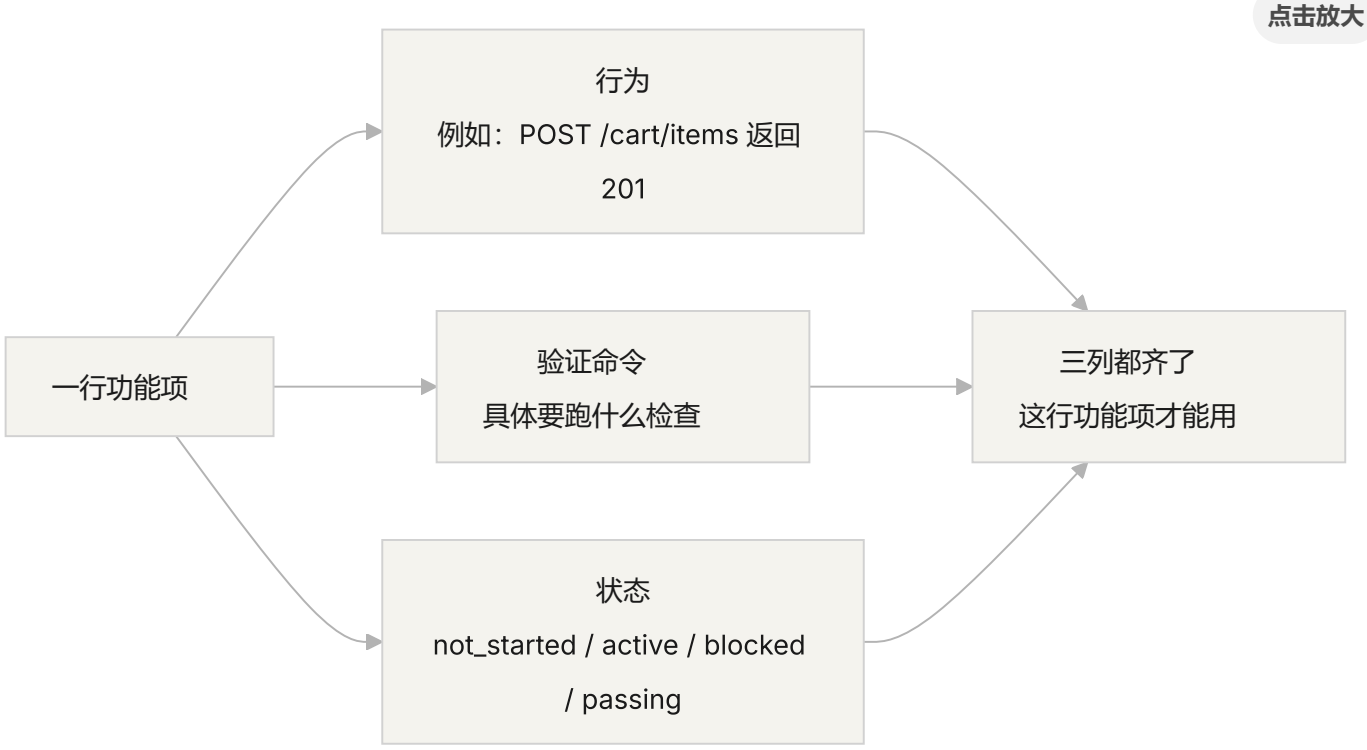
> 做了用户认证、购物车基本完成了、还需要做支付

新的 agent 会话看到这个记录，能回答以下问题吗？"基本完成"意味着什么？购物车通过了哪些测试？支付的阻塞条件是什么？答案都是"不知道"。

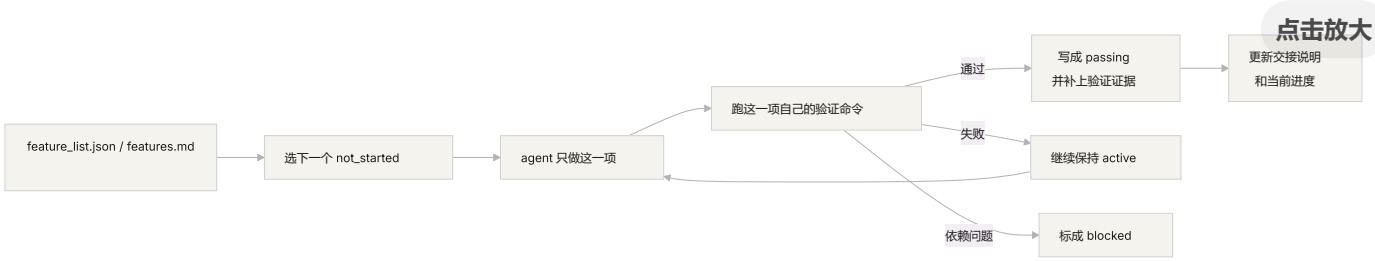
结果是：新会话要花时间推断项目状态，最终可能重复实现已完成的功能。Anthropic 的建议是把进度和验证结果写进版本化文件，让下一次会话能检查状态。

功能状态机

点击放大



点击放大



核心概念

- **功能清单是 harness 原语**：它是所有 harness 组件依赖的基础数据结构。调度器、验证器、交接器都要读取它才能工作。
- **三元组结构**：每个功能项包含三个要素：`（行为描述，验证命令，当前状态）`。行为描述告诉 agent 做什么，验证命令告诉它怎么算做完，状态告诉它现在到哪了。缺了任何一项，这个功能项就不完整。
- **状态机模型**：每个功能项有四种状态：`not_started`、`active`、`blocked`、`passing`。状态转移由 harness 控制，不是 agent 想改就能改。

- **通过状态门控**：功能从 `active` 变成 `passing` 的唯一方式是验证命令执行成功。这个转移是不可逆的，`passing` 了就不能退回去。
- **单一权威来源**：项目里关于"该做什么"的所有信息，必须从一个功能清单派生。不能出现功能清单和对话记录矛盾的情况。
- **反向压力**：还没通过的功能项数量就是 harness 对 agent 施加的压力。压力归零 = 项目完成。

为什么功能清单必须是原语

文档是给人看的，原语是给系统用的。文档可以被忽略，原语不能被绕过。

可以类比数据库的触发器约束和应用层的检查逻辑：前者由数据库引擎强制执行，任何 SQL 都无法跳过；后者依赖于应用代码的正确性，可能被意外绕过。功能清单作为 harness 原语，承担的就是数据库级别的约束角色，agent 不能绕过它。

具体来说，功能清单服务四个 harness 组件：

1. **调度器**：读状态，选下一个 `not_started` 的功能。
2. **验证器**：执行验证命令，判断是否允许状态转移。
3. **交接报告器**：从功能清单自动生成会话交接摘要。
4. **进度追踪器**：统计各状态分布，提供项目健康度指标。

实施方法

1. 定义一个最小化的功能清单格式

不需要复杂的系统，一个结构化的 Markdown 或 JSON 文件就够了。关键是每个条目必须有三元组：

```json
{
  "id": "F03",
  "behavior": "POST /cart/items with {product_id, quantity} returns 201",
  "verification": "curl -X POST http://localhost:3000/api/cart/items -H 'Content-
  "state": "passing",
  "evidence": "commit abc123, test output log"
}
```

2. 让 harness 控制状态转移

agent 不能直接把状态改成 `passing`。它只能提交验证请求，harness 执行验证命令，根据结果决定是否允许状态转移。这就是"通过状态门控"。

3. 在 CLAUDE.md 里写清楚规则

```
## 功能清单规则
- 功能清单文件: /docs/features.md
- 每次只激活一个功能项
- 功能项验证命令必须通过才能标为 passing
- 不要修改功能清单的状态，由验证脚本自动更新
```

4. 粒度校准

每个功能项应该是"一次会话能完成"的范围。太粗了做不完，太细了管理开销大。"用户可以添加商品到购物车"是一个好粒度，"实现购物车"太粗了，"创建 Cart 模型的 name 字段"太细了。

实际案例

一个电商平台的开发任务，10 个功能项。对比两种追踪方式：

备忘录模式：agent 用非结构化笔记记录进度。3 个会话后，笔记变成了"做了用户认证和商品列表、购物车基本完成但还有 bug、支付没开始"。新会话需要 20 分钟推断状态，最终重复实现了已完成的功能。

结构化模式：每个功能项有明确的状态和验证命令。新会话读取功能清单，3 分钟内知道：F01-F05 是 `passing`，F06 是 `active`（正在做），F07-F10 是 `not_started`。直接从 F06 继续，零重复。

结构化清单把每个功能的状态和验证命令写清楚，新会话一读就知道哪些已经做完。

核心要点

- **功能清单是 harness 的基础结构**，不是给人看的备忘录。调度器、验证器、交接器都依赖它。
- **每个功能项必须有三元组**：行为描述 + 验证命令 + 当前状态。缺一项就不完整。
- **状态转移由 harness 控制**，agent 不能自己改状态。通过验证是唯一的升级路径。
- **功能清单是项目的单一权威来源**，任何关于"该做什么"的信息都从这里派生。
- **粒度控制在"一次会话能完成"的范围**。太粗做不完，太细管不过来。

延伸阅读

- Building Effective Agents - Anthropic — 明确指出功能列表是控制 agent 执行范围的"核心数据结构"
- Harness Engineering - OpenAI — 强调"将工件外部化"的原则
- Design by Contract - Bertrand Meyer — 契约式设计原则，功能列表的理论基础
- How Google Tests Software — 测试金字塔和行为规格的工程实践

练习

1. **功能清单设计**：定义一个最小化的功能清单 JSON schema。包含：id、行为描述、验证命令、当前状态、证据引用。用它描述一个包含 5 个功能的真实项目。
2. **验证严格性对比**：选 3 个功能，分别设计"宽松"验证（如"代码无语法错误"）和"严格"验证（如"端到端测试通过"）。对比两种验证下的假阳性率。

3. **单一来源原则审查**：审查一个已有的 agent 项目，检查是否存在与功能清单矛盾的范围信息（对话里的隐式需求、代码里的 TODO 注释等）。设计一个方案，把所有信息统一到功能清单中。

English Version →

> 本篇代码示例：code/ 实战练习：Project 05. 让 agent 自己检查自己做的对不对

第九讲. 防止 agent 提前宣告完成

Agent 有一个系统性倾向：过早宣称任务完成。举例来说，你让它实现"密码重置"功能，它改了数据库 schema、写了 API 端点、加了邮件模板，跑了单元测试全部通过，然后告诉你"做完了"。但实际一跑才发现：密码重置链接发不出去，邮件服务配置缺失；数据库迁移半途失败，schema 不一致；端到端流程根本没走过一遍。

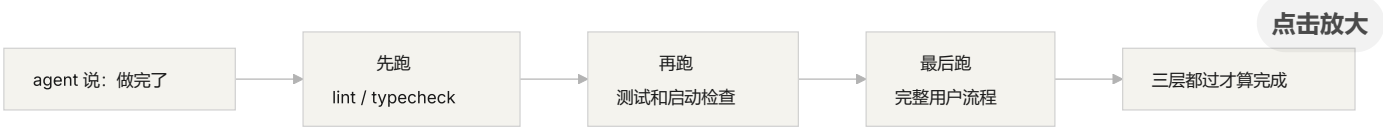
Guo 等人 2017 年在 ICML 上的经典论文证明：**现代神经网络系统性地过度自信**，模型自报的置信度显著高于实际准确率。AI 编码 agent 也一样，它"觉得"做完了，但实际上差得远。harness 必须用外部化的、基于执行的验证来替代 agent 的"感觉"。

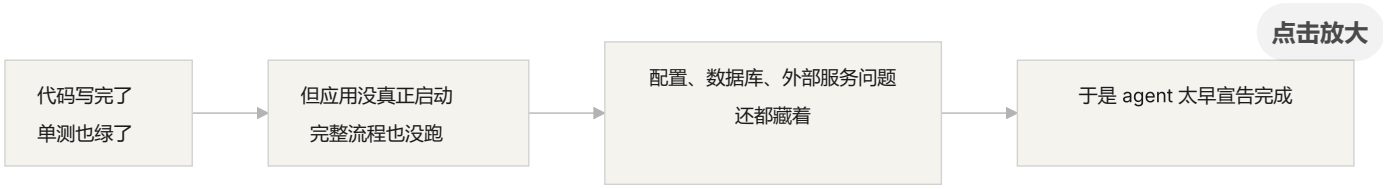
滑坡效应

过早完成声明几乎总是一样的套路：代码看着还行，语法正确、逻辑似乎合理，静态检查没有明显错误。但 harness 没有强制要求全面执行验证，agent 跳过了实际运行或只跑了部分测试。跑了单元测试但跳过集成测试，跑了测试但没检查覆盖率。最后"代码看起来没问题"就被当作了"功能已完成"的证据。

每一步都在丢失信息。从任务规范到代码实现，再到运行时行为，每次转换都可能引入偏差，每次跳过的验证都加剧了信息不对称。

三层终止检查





核心概念

- **过早完成声明**：agent 断言任务完成，但实际上存在未满足的正确性规范。问题出在 agent 依据代码层面的局部信心做判断，而系统级正确性需要全局验证。
- **置信度校准偏差**：agent 自报的完成信心与实际完成质量之间存在系统性差距。复杂多文件任务中，这个偏差显著为正，agent 总是比实际做得更自信。
- **终止标准**：一组明确的、可执行的判定条件，定义在 harness 里。agent 必须满足所有条件才能声明完成，"完成"从主观判断变成了客观判定。
- **验证-确认双闸门**：第一层验证检查代码是否正确实现了指定行为，第二层确认检查系统级行为是否满足端到端需求，两层都通过才算完成。
- **运行时反馈信号**：来自程序执行的日志、进程状态、健康检查，这是 harness 判定完成质量的客观基础。
- **完成优先级约束**：先验证功能正确性，再处理性能，最后管风格。核心功能没验证通过之前，不许做重构。

单元测试通过 ≠ 任务完成

这是最常见的陷阱，也是最危险的一个。agent 写了代码，跑了单元测试，全部绿色，然后说"做完了"。但单元测试的设计哲学是隔离被测单元、模拟依赖，这恰好使其无法检测跨组件问题：

接口不匹配：渲染进程传给预加载脚本的文件路径是相对路径，但预加载脚本期望绝对路径。各自的单元测试都用了 mock，都通过了，只有端到端跑通时才发现问题。

状态传播错误：数据库迁移改了表结构，但 ORM 的缓存层还持有旧结构的缓存条目。单元测试每次都是全新的 mock 环境，不会暴露这种跨层状态不一致。

环境依赖性：代码在测试环境（一切 mock）行为正确，在真实环境因配置差异、网络延迟、服务不可用而失败。

"顺便重构"是完成判定的毒药

Claude Code 有一个常见行为模式：在核心功能还没验证通过时就开始重构代码、优化性能、改进风格。Knuth 说的"过早优化是万恶之源"在 agent 场景中有了新含义，重构会改变已完成验证和未完成验证之间的边界，可能破坏之前隐式正确的代码路径。

自我评价的系统性偏差

Anthropic 在 2026 年的研究中发现了一个更深层的失败模式：**当 agent 被要求评估自己的工作时，它系统性地过度正面评价，即使人类观察者认为质量明显不达标。**

这个问题在主观任务（如设计审美）上尤其严重，"布局是否精致"是一个判断题，agent 可靠地偏向正面。即使在有可验证结果的任务上，agent 也会因为判断失误而影响表现。

解决方案不是让 agent "更客观"。同一个模型既生成又评估，内在地倾向对自己慷慨。**解决方案是把"干活的人"和"检查的人"分开。**

一个独立的评估 agent，经过专门调校为"挑剔"之后，比让生成 agent 自我评估有效得多。Anthropic 的实验数据如下：

架构	运行时长	成本	核心功能是否可用
单 agent 裸跑	20 分钟	$9	否（游戏实体无法响应输入）
三 agent（planner + generator + evaluator）	6 小时	$200	核心玩法能跑；物理和关卡仍有问题

这是同一个模型（Opus 4.5），同一段提示词（"做一个 2D 复古游戏编辑器"）。区别只在 harness：从"裸奔"到"planner 扩展需求 → generator 逐功能实现 → evaluator 用 Playwright 实际点击测试"。

> 来源：Anthropic: Harness design for long-running application development

预防过早完成的方法

1. 外部化终止判定

完成判定不应该由 agent 自己做。harness 独立执行终止校验，输入是运行时信号，不是 agent 的置信度。在 CLAUDE.md 里可以写清楚：

```
## 完成定义
- 功能完成 = 端到端验证通过，不是"代码写完了"
- 必须运行的验证层级：
  1. 单元测试通过
  2. 集成测试通过
  3. 端到端流程验证通过
- 在第 1 层没通过时，不许进入第 2 层
- 在第 2 层没通过时，不许进入第 3 层
```

2. 构建三层终止校验

- **第一层：语法与静态分析。**成本最低，信息量最小，但必须通过。这是最低限度的检查，字都没写错才能往下看。
- **第二层：运行时行为验证。**测试执行、应用启动检查、关键路径验证。这是核心完成证据，不仅要写了，还要能跑。
- **第三层：系统级确认。**端到端测试、集成验证、用户场景模拟。这是防止过早声明的最后一道防线，不仅要能跑，还要跑对。

3. 给 agent 提供可操作的错误反馈

OpenAI 在 Codex 实践中提出了一个特别有效的模式：**给 agent 写的错误消息要包含修复指导。**要明确指出哪里错了、应该怎么改，不要只说"错了"。 例如不要用 `"Test failed"`，而用 `"Test failed: POST /api/reset-password returned 500. Check that the email service config exists in environment variables. The template file should be at templates/reset-email.html."` 这种具体的、可操作的反馈让 agent 能自我修正，不需要人类介入。

4. 捕获运行时信号

有效的运行时信号包括：

- 应用是否成功启动并达到就绪状态？
- 关键功能路径在运行时是否执行成功？
- 数据库写入、文件操作等副作用是否正确？

- 临时资源是否被清理?

实际案例

任务：实现用户密码重置功能。涉及数据库操作、邮件发送和 API 端点修改。

提前交卷路径：agent 修改数据库 schema、编写 API 端点、添加邮件模板、跑单元测试（通过）、声明完成。看起来做了很多，但关键环节都跳过了。

实际遗漏项：(1) 端到端流程未测试，重置链接的实际发送和验证未确认。(2) 数据库迁移在部分执行后失败，导致 schema 不一致。(3) 邮件服务配置在目标环境中缺失。

harness 介入：终止校验强制执行，(1) 启动完整应用验证重置端点可访问；(2) 执行完整重置流程；(3) 验证数据库状态一致性。所有缺陷在会话内被发现，此时修复成本还是局部的。

核心要点

- **agent 系统性地过度自信**，置信度校准偏差是客观存在的。代码写完了不代表做对了。
- **完成判定必须外部化**，harness 独立验证，不信任 agent 的"感觉"。
- **三层校验缺一不可**：语法通过、行为通过、系统通过，层层递进。
- **错误消息要包含具体修复步骤**，让 agent 能自我修正，只说"错了"不够。
- **核心功能验证通过之前不许重构**，完成优先级约束是防止过早优化的关键。

延伸阅读

- On Calibration of Modern Neural Networks - Guo et al. — 证明现代深度网络系统性地过度自信
- Building Effective Agents - Anthropic — 运行时证据在完成判定中的关键作用
- Harness Engineering - OpenAI — 过早完成声明是 agent 的主要失败模式之一
- The Art of Software Testing - Myers — 测试方法层次和有效性的经典参考

练习

1. **终止校验函数设计**：为一个涉及数据库迁移和 API 修改的任务设计完整的终止校验。列出需要的运行时信号和每个信号的通过/失败标准。在一个实际任务上运行，记录它发现了哪些隐藏问题。

2. **校准偏差测量**：选 10 个不同类型的编码任务，记录 agent 的自报完成信心和实际完成质量。计算偏差值，分析它和任务复杂度的关系。

3. **多层防御实验**：对同一组任务跑三种配置：(a) 仅静态分析，(b) 加单元测试，(c) 完整三层校验。比较过早完成声明的比例和未捕获缺陷的数量。

English Version →

> 本篇代码示例：code/ 实战练习：Project 05. 让 agent 自己检查自己做的对不对

第十讲. 跑通完整流程才算真正验证

单元测试通过后，agent 经常会说"做完了"，但端到端运行时才会暴露真正的问题。举例来说，让 agent 给 Electron 应用加一个文件导出功能，它写了渲染进程组件、预加载脚本、服务层逻辑，每个组件的单元测试都通过了。agent 说"做完了"。实际点击导出按钮时：文件路径格式不对、进度条没反应、大文件导出时内存泄漏。5 个组件边界缺陷，单元测试一个都没发现。

每个部分单独看都"对"了，但拼在一起就出了问题。Google 的测试金字塔告诉我们，大量单元测试是基础，但如果你止步于此，就会系统性地漏掉组件交互问题。对于 AI 编码 agent 来说，这个问题更严重，因为 agent 倾向于只跑最快的测试然后宣告完成。**只有端到端测试能证明系统级缺陷不存在。**

单元测试的盲区

单元测试的设计哲学是隔离：模拟依赖，专注被测单元。这个哲学使单元测试快速且精确，但也制造了系统性的盲区。每个模块在隔离环境中表现完美，但真正拼在一起运行时才会暴露以下几类问题：

接口不匹配：渲染进程传给预加载脚本的文件路径是相对路径，但预加载脚本期望绝对路径。各自的单元测试都用了 mock，都通过了。只有端到端跑通时才发现问题。

状态传播错误：数据库迁移改了表结构，但 ORM 的缓存层还持有旧结构的缓存条目。单元测试每次都是全新的 mock 环境，不会暴露这种跨层状态不一致。

资源生命周期问题：文件句柄、数据库连接、网络套接字的获取和释放跨越多个组件。单元测试为每个测试创建和销毁独立资源，不会暴露资源竞争或泄漏。

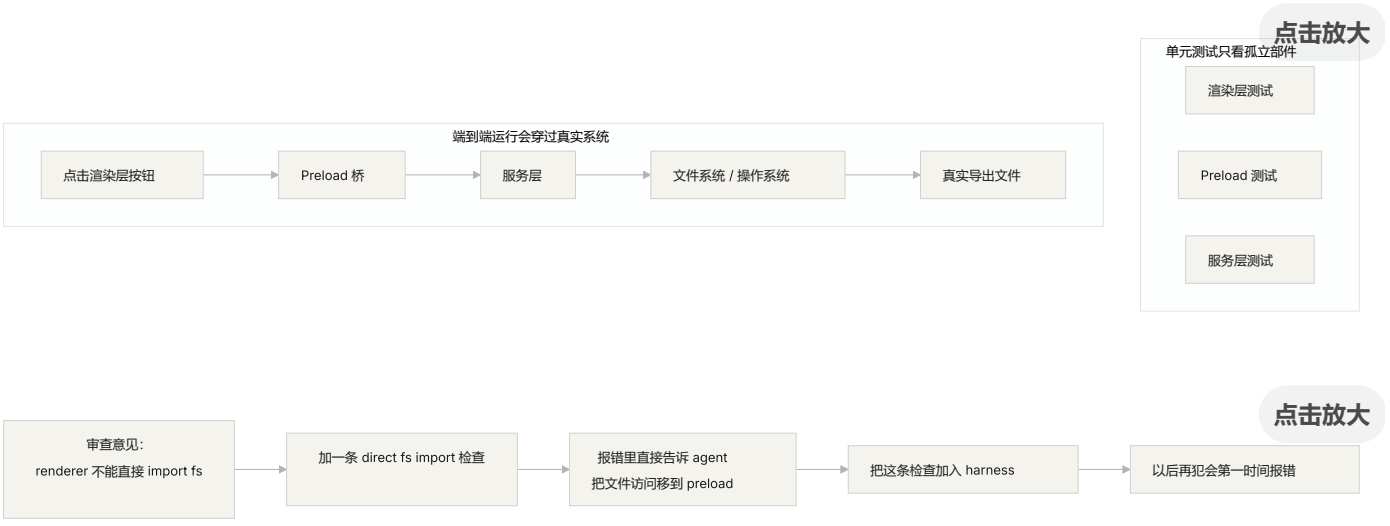
环境依赖性：代码在测试环境（一切 mock）行为正确，在真实环境因配置差异、网络延迟、服务不可用而失败。

端到端测试同时影响结果与行为

这是很多人没意识到的一点：当 agent 知道它的工作要过端到端测试时，它的编码行为会改变。

1. **考虑组件交互**：写代码时会想"这个接口和上游怎么对接"，不只关注单个函数。
2. **尊重架构边界**：有架构约束的系统里，端到端测试迫使 agent 遵守边界规则。
3. **处理错误路径**：端到端测试通常包含故障场景，迫使 agent 考虑异常处理。

测试金字塔与审查反馈提升



OpenAI 在 Codex 工程实践中强调：**为 agent 写的错误消息必须包含修复指导。**不写 `"Direct filesystem access in renderer"`，而写 `"Direct filesystem access in renderer. All file operations must go through the preload bridge. Move this call to preload/file-ops.ts and invoke it via window.api."` 这把架构规则变成了自动修正的闭环。错误消息不只是告诉你"出了什么问题"，还要告诉你"该怎么改"，让 agent 能够自主完成修正。

核心概念

- **组件边界缺陷**：组件 A 和 B 各自单元测试通过，但它们的交互产生了不正确的行为。这是端到端测试最擅长捕获的问题类型。

- **测试充分性梯度**：单元测试能检测的缺陷 <= 集成测试能检测的缺陷 <= 端到端测试能检测的缺陷。每往上一层，检测能力增强。
- **架构边界执行规则**：把架构文档里的规则（如"渲染进程不能直接访问文件系统"）变成可执行的自动化检查，从"写在纸上"变成"跑在 CI 里"。
- **审查反馈提升**：把重复出现的代码审查意见转化为自动化测试。每次发现重复问题就加一条规则，harness 会自动变强。
- **面向 agent 的错误消息**：失败信息不只是说"出了什么问题"，还要告诉 agent 具体怎么修，把测试失败变成自我修正的反馈循环。

实施方法

0. 先定好架构边界，再写端到端测试

端到端测试的前提是系统有清晰的边界。如果架构是一团面条，端到端测试只会证明"这团面条整体能跑"，不会告诉你哪里违反了设计意图。

OpenAI 的经验：**对 agent 生成的代码库，架构约束必须是第一天就建立的早期前置条件，不是等团队规模大了再考虑的事。** 原因很直接：agent 会复制仓库中已有的模式，即使那些模式是不均匀的或次优的。没有架构约束，agent 会在每次会话中引入更多偏差。

OpenAI 采用了"分层领域架构"，每个业务领域被分成固定的层：Types → Config → Repo → Service → Runtime → UI。依赖方向严格向前，跨领域关注点通过显式的 Providers 接口进入。任何其他依赖都是禁止的，并且通过自定义 lint 机械执行。

关键原则：**执行不变量，不微管实现。** 比如要求"数据在边界解析"，但不规定用哪个库。错误消息要包含修复指导，要告诉 agent 具体怎么改，不只说"违规了"。

> 来源：OpenAI: Harness engineering: leveraging Codex in an agent-first world

1. harness 必须包含端到端层

在你的验证流程里明确：对于涉及跨组件修改的任务，端到端测试通过是完成的前置条件：

```
## 验证层级
- 层级 1: 单元测试（必须通过）
- 层级 2: 集成测试（必须通过）
- 层级 3: 端到端测试（涉及跨组件修改时必须通过）
- 跳过任何必须层级的任务 = 未完成
```

2. 把架构规则变成可执行检查

每条架构约束都应该有对应的测试或 lint 规则：

```bash
# 检查渲染进程是否直接调用 Node.js API
grep -r "require('fs')" src/renderer/ && exit 1 || echo "OK: no direct fs access
```

3. 设计面向 agent 的错误消息

失败信息要包含三要素：什么出了问题、为什么、怎么修：

```
ERROR: Found direct import of 'fs' in src/renderer/App.tsx:12
WHY: Renderer process has no access to Node.js APIs for security
FIX: Move file operations to src/preload/file-ops.ts and call via window.api.read
```

4. 建立审查反馈提升流程

每次在代码审查中发现新类型的 agent 错误，就把它变成自动化检查。一个月后你的 harness 会比月初强得多。

实际案例

任务：在 Electron 应用中实现文件导出功能。涉及渲染进程 UI、预加载脚本文件系统代理、服务层数据转换。

单元测试阶段：渲染组件测试（通过，mock 文件操作）、预加载脚本测试（通过，mock 文件系统）、服务层测试（通过，mock 数据源）。agent 声明完成。

端到端测试揭示的缺陷：

缺陷	描述	单元测试	端到端
接口不匹配	文件路径格式不一致	未检测	检测
状态传播	导出进度未通过 IPC 传回 UI	未检测	检测
资源泄漏	大文件导出句柄未释放	未检测	检测
权限问题	打包环境权限不同	未检测	检测
错误传播	服务层异常未到 UI 层	未检测	检测

5 个缺陷全部被端到端测试捕获，单元测试一个都没发现。端到端运行更慢，但在 agent 工作流里，这点成本远小于把五个边界缺陷带到发布。

核心要点

- **单元测试对组件边界缺陷系统性盲视**：它们的隔离设计恰好使其无法检测交互问题。
- **端到端测试不仅检测缺陷，还改变 agent 的编码行为**：让它更关注集成和边界。
- **架构规则必须可执行**：每次提交自动检查，不能只写在文档里等人来看。
- **错误消息要面向 agent 设计**：包含"怎么修"的具体步骤，形成自我修正闭环。
- **审查反馈提升让 harness 自动变强**：每个被捕获的缺陷类别都变成永久防线。

延伸阅读

- How Google Tests Software - Whittaker et al. — 测试金字塔模型的经典来源
- Harness Engineering - OpenAI — 架构约束自动化执行的工程实践
- Chaos Engineering - Netflix (Basiri et al.) — 主动注入故障验证系统弹性
- QuickCheck - Claessen & Hughes — 属性测试方法，介于示例测试和形式化验证之间

练习

1. **跨组件缺陷检测**：选一个涉及至少三个组件的修改任务。先只跑单元测试记录结果，再跑端到端测试。分析每个额外发现的缺陷属于哪种跨层交互问题。
2. **架构规则自动化**：选项目里的一条架构约束，把它变成可执行检查（含面向 agent 的错误消息）。集成到 harness 里，用基准任务验证效果。
3. **审查反馈提升**：从代码审查历史中找一个重复出现的意见类型，按五步流程转化为自动化检查。比较提升前后该类问题的出现频率。

English Version →

> 本篇代码示例：code/ 实战练习：Project 06. 搭建一套完整的 agent 工作环境

第十一讲. 让 agent 的运行过程可观测

Agent 执行任务时常常像一个黑盒：它跑了 20 分钟，改了一堆文件，然后告诉你"做完了但有两个测试失败"。你问它为什么失败，"不太确定，可能是时序问题"。你问它改了哪些关键路径，"让我看看代码……"。

这种情况的根源在 harness 缺乏可观测性。agent 执行任务时，如果看不到运行时的实际状态，就只能凭猜测做决策。

没有可观测性，agent 在不确定状态中做决策，评估变成主观判断，重试变成盲目摸索。 OpenAI 和 Anthropic 都将可靠性定义为证据问题，harness 必须以可指导下一步决策的形式暴露运行时行为和评估信号。

可观测性缺失的影响

当 harness 缺乏可观测性时，四类问题会系统性出现。

无法区分"正确"和"看似正确"：一个函数在代码审查时看起来完全正确，语法对、逻辑通。但运行时因为边界条件处理错误，在特定输入下产生了不正确结果。只有运行时追踪能揭示实际执行路径偏离了预期。代码审查看的是"写了什么"，运行时追踪看的是"实际跑了什么"，两者缺一不可。

评估变成玄学：没有评分标准和验收条件时，评估者（人或 agent）只能依赖隐式假设。同一个输出，不同评估者可能给出截然不同的评价，质量评估不可复现。

重试变成盲猜：agent 不知道为什么失败时，重试方向是随机的。它可能在错误的方向上反复尝试，修复了不相关的代码路径而忽略真正的故障根源。每次盲重试都消耗 token 和时间。

会话交接信息断崖：当未完成的工作移交给下一个会话时，缺乏可观测性意味着新会话必须从零诊断系统状态。新会话要重读代码、重跑命令、判断哪些信号重要；版本化的进度和验证记录能提供可检查的状态。

实际运行示例

来看一个使用"计划者-生成者-评估者"三角色工作流的 harness，执行"为应用添加暗色模式"任务。

没有可观测性：计划者输出模糊描述，生成者根据模糊描述实现暗色模式，但和计划者的隐式预期不一致。评估者基于自己的隐式标准拒绝，但说不出具体哪里不对，只有一句"感觉不太对"。生成者基于模糊拒绝理由盲重试，循环 3-4 次，总耗时约 45 分钟，最终勉强产出。

有完整可观测性：计划者输出冲刺合同，列明要改哪些组件、每个组件的验证标准、排除项（不处理打印样式）。生成者按合同实现，运行时可观测性记录每个组件的样式加载和应用过程。评估者用评分标准逐维度评估，附具体证据引用："按钮颜色对比度不足（WCAG AA 标准 4.5:1，实测 2.1:1）"。一次迭代出高质量结果，总耗时约 15 分钟。

差别来自合同和证据是否存在于 agent 脑子之外。

双层可观测性

可观测性不是"多打点日志"那么简单。它分两层，缺一不可。



运行时可观测性：系统层的信号，包括日志、追踪、进程事件、健康检查，回答"系统做了什么"。

过程可观测性：harness 决策工件的可见性，包括计划、评分标准、验收条件，回答"为什么这个变更应该被接受"。

核心概念

- **运行时可观测性**：系统层的信号，包括日志、追踪、进程事件、健康检查，回答"系统做了什么"。

- **过程可观测性**：harness 决策工件的可见性，包括计划、评分标准、验收条件，回答"为什么这个变更应该被接受"。
- **任务轨迹**：一个任务从开始到完成的完整决策路径记录，类似分布式系统中的请求追踪。agent 的每一步操作及其上下文都被记录，出了问题可以回放完整过程。
- **冲刺合同**：编码开始前协商的短期协议，明确任务范围、验证标准、排除项。是过程可观测性的核心工具。
- **评估评分标准**：把质量评估从主观判断变成基于证据的结构化评分，使不同评估者对同一输出产生相似结论。
- **双层可观测性**：系统层和过程层同时设计、相互增强。运行时信号解释行为，过程工件解释意图。

Agent 自行处理可观测性的局限

你可能在想："agent 不能自己打日志吗？" 问题在于：

1. **agent 不知道它不知道什么**：它不会主动记录自己没意识到需要的信号。没有 harness 层面的约束，agent 只会记录它认为重要的东西，而它认为重要的东西往往不够。
2. **日志格式不统一**：不同会话用不同的日志格式，无法做系统化分析。
3. **过程可观测性不是日志能解决的**：冲刺合同和评分标准是结构化的工件，需要 harness 层面的支持，不是多 print 几行就能搞定的。

搭建可观测性的方法

1. 在 harness 里内置运行时信号采集

不要依赖 agent 自己打日志。harness 应该自动采集以下信号：

- **应用生命周期**：启动、就绪、运行、关闭各阶段状态
- **功能路径执行**：关键路径的执行记录，包括入口、检查点和出口
- **数据流**：数据在组件间的流转记录
- **资源利用**：异常的资源使用模式（如内存持续增长）

- **错误和异常**：完整的错误上下文，不只是错误消息

2. 实施冲刺合同

在每个任务开始前，生成者和评估者（可能是同一个 agent 的不同调用）协商一份合同，明确这次要做什么、怎么做算通过：

```markdown
# 冲刺合同：暗色模式支持

## 范围
- 修改主题切换组件
- 更新全局 CSS 变量
- 添加暗色模式测试

## 验证标准
- 每个组件的视觉回归测试通过
- 主流程端到端测试通过
- 无样式闪烁（FOUC）

## 排除项
- 不处理打印样式
- 不处理第三方组件暗色模式
```

3. 建立评估评分标准

把"好不好"变成可量化的评分：

```markdown
# 评分标准

| 维度 | A | B | C | D |
|------|---|---|---|---|
| 代码正确性 | 所有测试通过 | 主流程通过 | 部分通过 | 编译失败 |
| 架构合规 | 完全合规 | 轻微偏离 | 明显偏离 | 严重违反 |
| 测试覆盖 | 主流程+边缘 | 仅主流程 | 仅有骨架 | 无测试 |
```

4. 用 OpenTelemetry 标准化

为每个 harness 会话创建一个 trace，每个任务创建一个 span，每个验证步骤创建子 span。使用标准属性标注关键信息。这样可观测性数据可以和标准工具链（Jaeger、Zipkin）集成。

Anthropic 的三 agent 架构实验

Anthropic 2026 年 3 月的报告给出了同一套三 agent 架构更新版的 DAW 运行数据（"用 Web Audio API 做一个浏览器端 DAW"）。第一版为 Opus 4.5 构建，generator 按 sprint 工作、evaluator 逐 sprint 评分；更新版在 Opus 4.6 上去掉 sprint，builder 连续实现，evaluator 在每轮构建后审查。下面的表是更新版的数据：

Agent 和阶段	时长	成本
Planner（规划者）	4.7 分钟	$0.46
Build 第 1 轮	2 小时 7 分钟	$71.08
QA 第 1 轮	8.8 分钟	$3.24
Build 第 2 轮	1 小时 2 分钟	$36.89
QA 第 2 轮	6.8 分钟	$3.09
Build 第 3 轮	10.9 分钟	$5.88
QA 第 3 轮	9.6 分钟	$4.06
总计	**3 小时 50 分钟**	**$124.70**

三个角色各司其职。下面的角色描述对应第一版；更新版把 generator 的 sprint 去掉，builder 连续工作，evaluator 每轮审查：

Planner（规划者）：接收一段 1-4 句话的用户需求，扩展成完整产品规格。被要求"大胆设定范围"并且"专注于产品上下文和高层技术设计，而不深入详细的技术实现"。原因是：如果 planner 过早指定了粒度技术细节且搞错了，错误会级联到下游实现。更好的做法是约束交付物，让 agent 在执行中自己找到路径。

Generator（生成者）：按 sprint 逐个功能实现。每个 sprint 前和 evaluator 协商一份 sprint 合同，约定这个功能块"做完"的标准。然后按合同实现，自评后交给 QA。

Evaluator（评估者）：用 Playwright MCP 像用户一样点击运行中的应用，测试 UI 功能、API 端点和数据库状态。对每个 sprint 按四个维度评分：产品深度、功能性、视觉设计和代码质量。每个维度有硬性阈值，任一不达标则 sprint 失败，generator 收到详细反馈后修复。

QA 第 1 轮反馈的示例："这是一个视觉上令人印象深刻的应用，AI 集成工作良好，但核心 DAW 功能有几个是展示性的，没有交互深度：剪辑不能拖拽/移动，没有乐器 UI 面板（合成器旋钮、鼓垫），没有视觉效果编辑器（EQ 曲线、压缩器仪表）"。这些不是边缘情况，它们是让 DAW 可用的核心交互。具体的、有证据的反馈，不是"感觉不对"。

Evaluator 不是一开始就这么强。早期版本会识别出合理的问题，然后说服自己这些问题不严重，最终批准工作。调校方式是：读 evaluator 的日志，找到它的判断和人类判断分叉的地方，更新 QA 的 prompt 解决那些问题。经过几轮这种开发循环，evaluator 的评分才变得合理。

> 来源：Anthropic: Harness design for long-running application development

---

核心要点

- **可观测性是 harness 的架构属性**：它是设计时必须考虑的核心能力，不应只作为事后添加的功能。
- **双层可观测性缺一不可**：运行时信号解释"发生了什么"，过程工件解释"为什么这样做"。
- **冲刺合同前置对齐工作**：防止"生成者做了评估者因可预见原因立即拒绝的东西"。
- **评分标准让评估可复现**：不同评估者对同一输出产生相似评分。
- **可观测性把模糊的拒绝变成具体、可修的缺陷。**

延伸阅读

- Observability Engineering - Charity Majors — 现代可观测性工程的理论和实践框架
- Dapper - Google (Sigelman et al.) — 大规模分布式追踪的开创性实践
- Harness Design - Anthropic — 引入冲刺合同和评估评分标准
- Site Reliability Engineering - Google — 可观测性在生产系统中的系统化应用

练习

1. **可观测性差距分析**：审查你当前的 harness，评估系统层和过程层可观测性。找出无法从现有信号区分的系统状态，提出补充方案。
2. **冲刺合同实践**：为一个真实任务写冲刺合同。让 agent 按合同执行，对比没有合同时的效率和质量差异。
3. **任务轨迹构建**：记录一个完整编码任务中 agent 的每一步操作。用 OpenTelemetry 语义约定标注。分析轨迹中的信息瓶颈——哪些步骤的决策缺乏足够的信号支持。

English Version →

> 本篇代码示例：code/ 实战练习：Project 06. 搭建一套完整的 agent 工作环境

第十二讲. 每次会话结束前都做好交接

在使用 AI 编码 agent 进行持续开发时，一个常见的问题是：每个 agent 会话结束时，如果没有刻意做清理，代码库的状态会越来越混乱。举例来说，一个 agent 会话修改了 20 个文件并提交代码后退出，下一个会话启动时，可能发现构建失败、测试变红、临时调试文件散落各处，功能清单和进度记录也没有更新。新会话需要花费大量时间诊断上一个会话做了什么，才能继续工作。

OpenAI 和 Anthropic 都明确指出，长期可靠性取决于操作纪律，单次运行成功并不够。每个会话结束时的状态质量，直接决定下一个会话的效率。这一讲要讨论的，就是如何让每个会话在结束时留下一个干净的状态，让下一个会话可以立刻开始工作。

熵增是默认状态

Lehman 的软件演化定律告诉我们：一个持续变更的系统，如果没有人主动管理，它的复杂性一定会增加。这对 AI 编码 agent 来说尤其成立——agent 每次会话都会引入变更，如果不在退出时清理，技术债务会指数级累积。

OpenAI 在 5 个月的 Codex 实验中观察到一个现象：agent 会复制仓库中已有的模式，哪怕那些模式是不一致或次优的。随着时间推移，这种复制必然导致整体质量漂移。打个比方：第一个人在公共区域放了一个杯子，第二个人路过时心想"反正已经乱了"，自己也放了一个。一周后，桌上就堆满了。代码库的退化也是同样的过程。

OpenAI 团队最初把周五留出来——一个工作日，约占一周工作量的 20%——手动清理 agent 留下的"AI 垃圾"。随着吞吐量上升，手动清理的成本压过了收益。他们最终找到了系统性的解决方案：

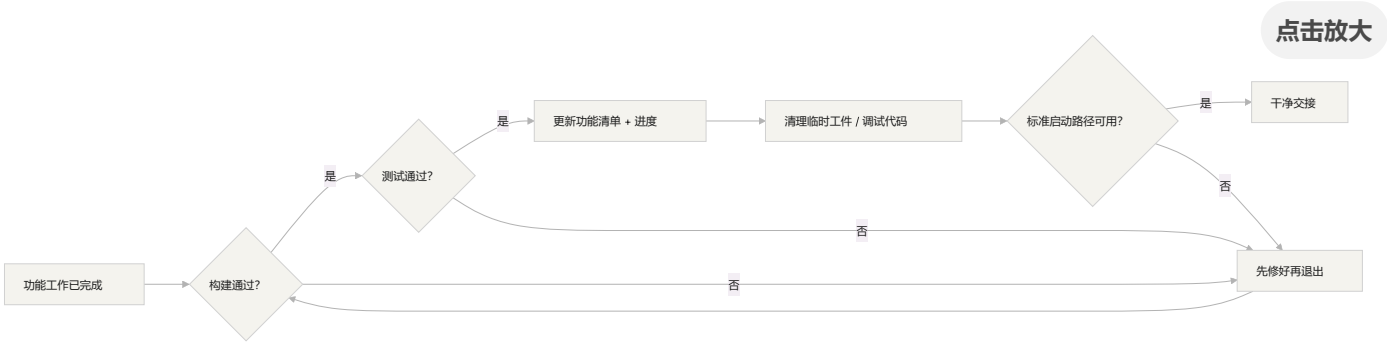
1. **把好习惯写进仓库规则**：比如"优先使用共享工具包，不要手写 ad-hoc 辅助函数"、"不要瞎猜数据结构，查类型定义或用类型安全 SDK"。这些规则是具体的、机械的、可以自动检查的。
2. **建立周期性清理流程**：一组后台任务定期扫描偏离规则的代码，更新质量评分，自动开重构 PR。大多数 PR 可以在一分钟内审查并自动合并。
3. **人类经验捕获一次，持续执行**：每次代码审查意见、重构 PR、用户报的 bug，都转化为文档更新，或直接编码到检查工具中。文档还不够时，就把规则提升为自动检查的代码。

一句话总结：技术债是高息贷款，持续小额还款比攒到一次性爆雷好得多。

> 来源：OpenAI: Harness engineering: leveraging Codex in an agent-first world

清洁状态：不只是"代码能编译"

清洁状态的要求远比"代码能编译"要多。构建通过是最基本的前提——下一个会话不应该一上来就先修别人的构建错误。所有测试也必须通过，包括会话开始前就存在的旧测试，你这次改动不能破坏已有的功能。而且验证必须在 CI 环境里跑，不是"在我机器上能过就行"。

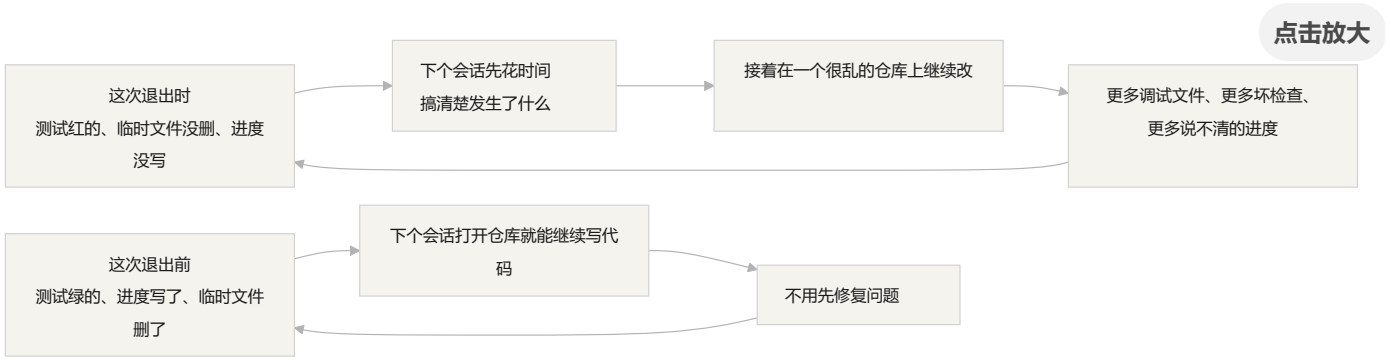


构建和测试只是底线，还有三条容易被忽略的要求。

第一，当前进度必须记录在机器可读的工件中。具体来说包括三类：已完成的子任务和它的通过标准、正在做但还没做完的子任务和它当前卡在哪、还没开始的子任务。把进度和验证结果记录在版本化文件里，下一次会话就能检查状态、接着工作。

第二，临时调试产物必须清理干净。调试日志、临时文件、注释掉的代码、TODO 标记，这些东西都会增加下一个会话的认知负担。新会话看到一堆 `console.log('debug')` 和 `// 临时方案，回头改`，根本分不清哪些是有意的、哪些是垃圾。

第三，标准启动路径必须可用。下一个会话能不能不靠人工干预就直接开始工作？环境初始化、代码库加载、上下文获取、任务选择，这些路径中的任何一个被破坏，新会话就无法自行启动工作。



总结一下，一个干净的会话退出需要满足五个条件：构建通过、测试通过、进度已记录、临时工件已清理、启动路径可用。缺一个都不算"做完了"。

核心概念

- **清洁状态**：会话退出时必须满足五个条件——构建通过、测试通过、进度已记录、无过时工件、启动路径可用。这五个条件共同构成"做完"的真正定义。
- **会话完整性**：可以类比数据库事务。一个会话的工作要么全部完成并留下清洁状态，要么回滚到上一个一致状态，不存在"做了一半但还行"的中间地带。
- **质量文档**：对代码库中每个模块持续记录质量评分的文件。它是持续更新的，追踪每个模块到底是变强了还是变弱了。
- **清理循环**：定期执行的维护会话，目标是从代码库中系统性地清除积攒的问题。它属于常规保养，不属于紧急修复。就像汽车定期换机油，不等发动机报警才去修。
- **Harness 简化**：随着模型能力提升，定期移除不再必要的 Harness 组件。今天必须有的约束条件，三个月后用更强的模型可能就成了多余开销。
- **幂等清理**：清理脚本无论执行多少次，结果都一样。这意味着清理失败时重跑一遍也安全，不会因为重复执行产生新问题。

"以后再清理"是永远不清理

最常见的心理陷阱就是"这次来不及清理了，下次再弄"。但下次的 agent 根本不知道你上次留下了什么，它看到的只是一堆混乱的代码和不确定的状态。它得花大量时间推断"这段代码里哪些是有意的，哪些是临时的"。

更糟的是，每个会话都有自己的任务目标。新会话来的时候是为了做新功能，不是为了清理上一个会话遗留问题的。它会直接忽略混乱，在混乱的基础上开始新工作，然后引入更多混乱。这是一个熵增的正反馈循环——越乱越不管，越不管越乱。

下面用一个 12 周的假设例子说明趋势：

没有清洁策略：

时间	构建通过率	测试通过率	新会话启动时间
第 1 周	100%	100%	5 分钟
第 4 周	95%	92%	15 分钟
第 8 周	82%	78%	35 分钟
第 12 周	68%	61%	60+ 分钟

有清洁策略：

时间	构建通过率	测试通过率	新会话启动时间
第 1 周	100%	100%	5 分钟
第 12 周	97%	95%	9 分钟

每周清理让起步状态保持稳定：每个会话结束时把构建、测试、进度和临时工件收拾干净，下一个会话从同一个基线开始。

怎么做

1. 清洁状态是完成的必要条件

在 Harness 里明确定义：会话完成的条件是两件事同时满足——任务通过验证，且清洁状态检查通过。缺任何一个，会话就不算完成。在项目的 CLAUDE.md 或 AGENTS.md 里可以这样写：

```
## 会话退出检查清单
- [ ] 构建通过（npm run build）
- [ ] 所有测试通过（npm test）
- [ ] 功能清单已更新
- [ ] 无调试代码残留（console.log, debugger, TODO）
- [ ] 标准启动路径可用（npm run dev）
```

这里解释一下"功能清单"。功能清单（feature list）是一份机器可读的文件，记录了项目中所有功能项的完成状态。每一项功能有三列信息：这个功能具体做什么、用什么命令来验证它、当前状态是什么（未开始/进行中/已阻塞/已通过）。调度器靠功能清单选下一个要做的工作，验证器靠它判断做完没有，交接器靠它生成进度报告。没有功能清单，agent 就不知道"做完"的标准是什么——它可能会用自己的标准判断完成，而你心里的"做完"是完全不同的东西。

2. 双模式清理策略

把清理分成两种模式，配合使用：

即时清理（每个会话结束时）：清理本次会话创建的临时文件、更新功能清单状态、确保构建和测试全部通过。原则是用完就清，像引用计数一样——谁产生的垃圾谁负责清掉。

定期清理（每周一次）：做一次全面的系统扫描，处理累积的结构性问题、更新质量文档、跑基准测试检测整体质量有没有漂移。原则是定期全身体检，不让小问题拖成大病。

3. 维护质量文档

质量文档是一份持续更新的文件，对代码库中每个模块打分和评价。新会话一打开就能看到上次会话后每个模块的状态。例：

```markdown
# 质量文档

## 用户认证模块（质量：A）
- 验证通过：是
- agent 可理解：是
- 测试稳定性：稳定
- 架构边界：合规
- 代码规范：遵循

## 支付模块（质量：C）
- 验证通过：部分（支付回调未测试）
- agent 可理解：困难（逻辑分散在 3 个文件）
- 测试稳定性：不稳定（2 个 flaky 测试）
- 架构边界：有违规
- 代码规范：部分遵循
```

有了这份文档，新会话一上来就知道当前代码库的健康状况，优先处理评分最低的模块。从这个角度说，质量文档是 Harness 可观测性的一部分——它让 agent 的运行结果在代码库层面可见。

4. 定期简化 Harness

Harness 中每个组件的存在，都源于模型在某个方面尚无法独立完成。随着模型能力不断演进，这些前提会逐渐过时。

Anthropic 的实验直观地展示了这一点。他们最初的 Harness 包含一个任务拆分机制：因为当时的模型一次性处理不了太大的任务，所以需要把大任务先拆成多个小步骤，让模型按顺序逐个完成。当 Opus 4.6 发布后，模型自己就能规划好该怎么一步步做事，原来的任务拆分机制成了多余的步骤。移除后，Builder Agent 能够连续工作超过两小时而不偏离方向，流程反而更流畅了。

Evaluator 的情况则有所不同。Evaluator 是 Harness 中的评估组件，负责检查生成的代码质量，找出遗漏的功能和未完成的实现。尽管 Opus 4.6 能力更强，当任务难度较高、逼近模型能力的上限时，Evaluator 依然很有用。但如果任务本身很简单，远在模型能力范围之内，Evaluator 可能就是多余的。因此，要不要保留 Evaluator，取决于你的任务有多难、模型有多强——这两者的相对关系才是关键。

推荐做法：每月挑选一个 Harness 组件，暂时禁用它，跑一遍基准任务。如果结果没有退化，就永久移除。如果退化，则恢复该组件，或换一个更轻量的替代方案。

一个更深层的原则：随着模型能力的提升，Harness 中有趣的组合并没有减少，它在位移。过去必须解决的问题被模型增长的能力覆盖了，同时新的能力边界被打开，暴露出过去触及不到的新问题。

5. 清理操作必须幂等

幂等的意思是：一个操作无论执行一次还是执行一百次，结果都一样。清理脚本必须具备这个特性，因为清理失败时你会重跑一遍。如果重跑产生不同的结果，就说明清理脚本存在 bug。例：

```bash
# 幂等的清理操作
rm -f /tmp/debug-*.log  # -f 确保文件不存在时不报错
git checkout -- .env.local  # 恢复到已知状态，多跑几次结果相同
npm run test  # 验证清理没有破坏功能
```

6. 高吞吐量改变了合并策略

当 agent 的产出远超人类审查能力时，传统的合并策略需要调整。OpenAI 团队的经验是：在一个 agent 每天开出 3.5 个 PR 的环境里，减少阻塞型的合并检查是正确的选择。PR 应该尽快合并，测试偶尔的假失败（flake）用后续运行来修正，不必无限期卡住进度。

这里有一个关键的判断标准：修正一个 bug 的平均成本，和等待人类审查一个 PR 的平均成本，哪个更低？当前者低于后者时，快速合并加上快速修正比慢慢审查更好。

注意：这条规则的前提是 agent 的高产出远超过人类的审查带宽。在一个低产出环境里，快速合并没有意义。但在 agent 每天提交几十个 PR 的环境里，等待人工审查的成本远高于修一个漏网 bug 的成本。

实际案例

一个使用 agent 持续开发的 Electron 应用，12 周的演化过程：

无清洁策略（对照组）：每个会话做完功能就退出，不做额外清理。第 12 周时，构建通过率 68%，测试通过率 61%，新会话启动 60 分钟以上，过时工件 103 个。

有清洁策略（实验组）：每个会话结束时执行完整清洁检查，加上每周一次清理循环。第 12 周时，构建通过率 97%，测试通过率 95%，新会话启动 9 分钟，过时工件 11 个。

到第 12 周，有清理策略的一组，构建和测试通过率仍接近起点，新会话能较快开始，过时工件很少。清理每个会话只多花几分钟，换来的是一份下一个会话可以信任的起步状态。

核心要点

- **清洁状态是会话完成的必要条件。**它是"完成"定义的一部分，属于必要条件。代码写完了但状态是脏的，那就不算做完。
- **五个维度缺一不可**：构建、测试、进度、工件、启动。每一条都要在退出时显式检查，不能靠"感觉应该没问题"。
- **功能清单让 agent 知道"做完"的标准。**没有清单，agent 用自己的标准判断完成，那个标准几乎一定比你的标准低。
- **质量文档让代码库的健康状况可追踪。**知道哪里在退化，才能主动修复。不知道问题在哪儿，就只能等它爆发。
- **定期简化 Harness**：随着模型能力提升，主动移除不再必要的组件。今天必须有的约束，三个月后可能就是累赘。
- **"以后再清理"等于永远不清理。**熵增是默认方向，只有主动的清洁操作才能对抗它。每次多花五分钟，长期来看是回报最高的投资。

延伸阅读

- Clean Code - Robert C. Martin — 代码整洁之道的经典著作，清洁状态背后的软件工程原则
- Harness Engineering - OpenAI — OpenAI 团队在 agent 时代如何通过 Harness 设计确保可重复性
- Effective Harnesses for Long-Running Agents - Anthropic — Anthropic 的长期运行 agent 实践，清洁会话退出对可靠性的关键作用
- Programs, Life Cycles, and Laws of Software Evolution - Lehman — 软件演化定律的原始论文，证明了无主动维护时系统复杂性必然增长
- 第八讲：用功能清单约束 agent 该做什么 — 如何用功能清单给 agent 明确的完成标准
- 第九讲：防止 agent 提前宣告完成 — 如何通过验证机制避免 agent 过早说"做完了"
- 第十讲：跑通完整流程才算真正验证 — 为什么端到端测试是唯一可靠的完成验证

- 第十一讲：让 agent 的运行过程可观测 — 如何通过可观测性让 agent 的运行状态不再是一个黑盒
- 第五讲：让跨会话的任务保持上下文连续 — 会话交接的前置知识，如何让新会话快速接上

练习

1. **设计你的清洁状态检查表**：为你的代码库设计一个会话退出检查表，涵盖五个维度（构建、测试、进度、工件、启动）。在接下来的 5 个连续会话中坚持执行，记录每个维度上违反了几次。
2. **基准对比实验**：选定一个固定的任务集，分别在两种 Harness 配置下跑一遍——一种要求清洁状态检查通过才算完成，另一种不要求。比较两组的完成率、重试次数和漏网 bug 的数量。
3. **Harness 简化实践**：从你的 Harness 里选一个组件，暂时禁用它，跑一遍基准任务。比较有它和没它的结果，然后决定是保留、移除还是换一个更轻量的替代方案。
4. **质量文档入门**：为你的项目创建第一份质量文档。挑 3 到 5 个核心模块，给每个模块打分（A/B/C/D），标注具体的扣分原因。在接下来的 4 周里，每周更新一次评分，观察质量是变好了还是变差了。

English Version →

> 本篇代码示例：code/ 实战练习：Project 07. 搭建你的第一个自动循环

第十三讲. 从手动驱动到自动循环

前十二讲，你做的事情始终有一个共同前提：**你坐在键盘前，一次一次地输入指令。**

你写好了 `AGENTS.md`（第一到四讲），建立了状态管理（第五、六讲），用功能清单约束范围（第七、八讲），让 agent 在结束时留下干净交接（第九、十二讲），让运行过程可观测（第十、十一讲）——但所有这些工作的触发者，始终是你。agent 不会自己决定什么时候该干活，因为没有人按下"开始"按钮。

这一讲要讨论的，就是怎么把"按按钮"这件事也交给系统。不是放弃控制，而是把控制升到更高一层。

/goal：最简形态的 loop

理解 loop engineering，最好的入口不是一套复杂的架构图，而是一个具体的命令。

2026 年初，Claude Code 和 OpenAI Codex 不约而同地加入了同一个功能：`/goal`。你在终端里敲：

```
/goal "所有测试通过，lint 零告警，合并到 main"
```

然后合上笔记本去睡觉。八小时后醒来，agent 已经自己完成了分析、编码、测试、修复、合并的全过程。它失败了就重试，方向错了就换策略，通过了就结束——不需要你坐在旁边说"再试一次试试"。

`/goal` 和传统 prompt 的区别只有一点，但这一点改变了一切：

	传统 prompt	`/goal`
你给什么	下一步具体做什么	最终状态是什么
agent 做什么	执行一次	循环直到达成
谁判断做完了	你	一条可验证的停止条件
你什么时候可以走	不能走	给完 `/goal` 就行

`/goal` 本质上就是一个 loop。它的结构只有三样东西：**一个目标，一种验证方式，一条停止条件。** 但就是这三样东西，让你的位置从循环**里面**移到了循环**外面**。

`/goal` 是怎么长出来的

`/goal` 不是某天突然从 0 跳到 1 的。它是从日常工作流里一点一点长出来的，大致经历了四个阶段：

阶段一：手动一条一条输。 最早的用法就是你一句我一句，"写个函数"、"加个测试"、"改一下这个逻辑"。agent 每执行一步就停下来等你说下一步。你是整个流程的调度器。

阶段二：长 prompt + 多步骤。 后来大家开始写更长的 prompt，把多个步骤写在一起："先分析代码，再写实现，再跑测试，测试没过就修。"agent 可以一口气跑好几步了，但你还是得盯着——因为它可能在某一步跑偏，或者跑完了不知道下一步该干嘛。

阶段三：agent 自己判断要不要继续。 再往后，agent 有了"自省"能力——跑完一步自己看结果，决定下一步怎么走。你给一个目标，它自己拆步骤、自己重试。但问题来了：它什么时候停？它自己说"我做完了"算不算数？实践反复证明——不算数。agent 太容易宣告胜利了。

阶段四：独立的停止判断——`/goal`。 最后一步是把"判断做完了没有"这件事，从干活的 agent 手里拿出来，交给一个独立的判断者。可能是另一个模型、可能是一段脚本、可能是一条测试命令，但总之——不能让写代码的人自己批作业。到这一步，`/goal` 才真正成立：你给目标，它循环跑，独立判断停不停，你可以走人。

这四个阶段不是某家公司规划好的路线图，是全世界用 agent 写代码的人，在各自的日常里，被同一个痛点推着，一步一步走到了同一个地方。Claude Code 和 Codex 在 2026 年初几乎同时上线 `/goal`，不是巧合——是时候到了。

Loop 不只有一种

`/goal` 是最容易理解的 loop，但它不是唯一的一种。按触发方式和停止方式的不同，loop 可以分成几类：

类型	触发方式	停止方式	Claude Code	Codex	适用场景
回合制 loop	你手动输入每一条 prompt	agent 认为做完了，或者你打断	普通对话	普通对话	小任务、探索性工作
目标驱动 loop	你给一个目标	独立判断者确认达成，或达到最大回合数	`/goal`	`/goal`（需手动开启）	有明确完成标准的复杂任务
时间驱动 loop	定时触发（每隔 N 分钟/小时）	你手动停止，或任务完成后自行退出	`/loop`	对话线程自动化（Thread automation）	轮询状态、定期巡检、重复性工作
事件驱动 loop	外部事件触发（PR 提交、CI 失败、新 issue）	处理完事件就停，或达到重试上限	Routines (API / GitHub Webhook)	独立自动化任务 + 插件	响应式工作流、CI/CD 集成

这几种 loop 不是互相取代的关系，而是工具箱里的不同工具。小任务用回合制就够了；有明确终点的用 `/goal`；需要盯着什么东西的用 `/loop`；要和外部系统联动的用事件驱动。

`/goal` 和 `/loop` 别搞混

名字里都带 "loop"，但它们解决的是完全不同的问题：

	`/goal`	`/loop`
本质	一个大任务，跑到完为止	同一个小动作，按间隔重复跑
停止条件	目标达成了，或者预算花完了	你手动停，或者任务做完自己退
时间特征	一次长跑，可能跑几小时甚至几天	周期性短跑，每次可能只跑几分钟
状态累积	任务越跑越接近终点	每次都是独立的，不累积进度
类比	跑马拉松——发令枪响了就跑，撞线就停	闹钟——每隔一段时间响一次，你关了才停
典型用法	"实现整个支付系统，有测试覆盖"	"每 15 分钟看一眼 CI 挂了没"

一个容易犯的错：把该用 `/goal` 的事情塞进 `/loop` 里。比如你写 `/loop 10m "继续实现支付系统"` ——这是错的。因为 `/loop` 每次都是独立跑一遍指令，不会记得上次做到哪了，结果就是每次都从同一个地方重新开始。

判断该用哪个的一句话标准：这件事有终点吗？

- 有终点 → `/goal`
- 没终点，就是要一直盯着 → `/loop`

本讲讲的 Loop Engineering，核心不是某一个命令，而是**当你需要的时候，能设计出包含以上所有类型的系统——让 agent 在你不在场的时候也能自己跑。**

你不必每次都写 `/goal`。但理解它从哪来、为什么长这样，就理解了 loop engineering 的核心——更复杂的 loop 只是在三样基础（目标、验证、停止）之上，加上了调度、并行、隔离、记忆这些零件。

不同的日期，同一个方向

同一个想法出现在不同的地方、不同的时间。

Peter Steinberger（OpenClaw 作者，一条推文）：「你不应该再给 coding agent 写 prompt 了。你应该设计循环去 prompt 你的 agent。」

Boris Cherny（Anthropic Claude Code 负责人，在 Acquired 播客上）：「我已经不手动 prompt Claude 了。我有一堆循环在跑，它们负责 prompt Claude、搞清楚要做什么。我的工作变

成了写循环。」

Addy Osmani（Google Chrome 工程负责人）在 6 月 7 日撰文将这个概念命名为 **Loop Engineering**，并给了它一句话定义：

> Loop engineering **就是用系统取代你自己去** prompt agent。

Cherny 在2025-12-27 的个人报告里写道：30 天内他合并了 259 个 PR——497 次 commit，新增 40,000 行、删除 38,000 行——每一行都由 Claude Code + Opus 4.5 写成。

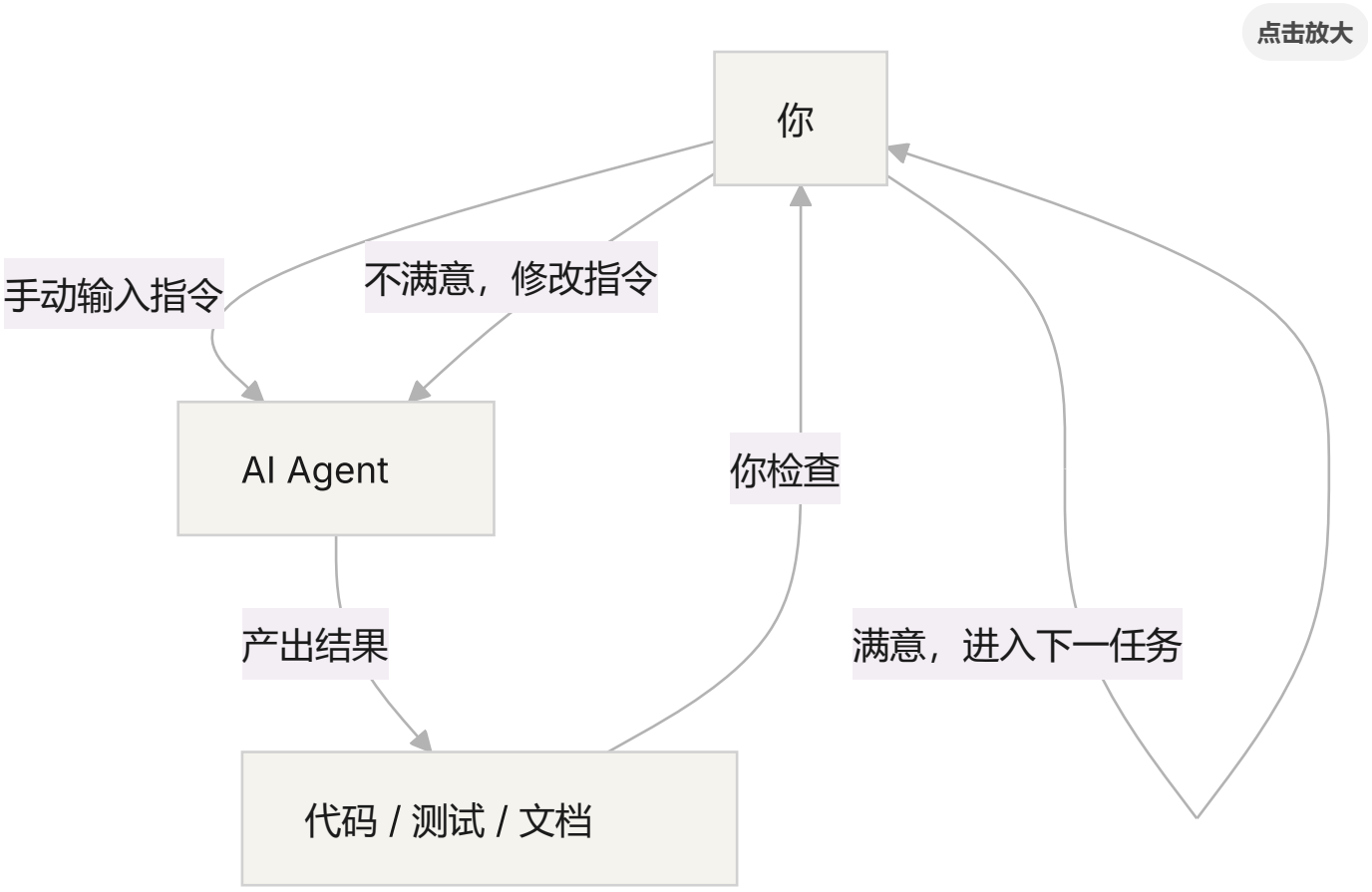
这些引文来自不同的日期、形式和团队，共同方向是把调度、验证和停止判断从当下的 prompt 里移出来，放进一个系统。到 2026 年中，agent 能独立完成非平凡任务、调度原语已经内置、单次运行成本也低到值得按定时器反复跑；零件齐了，把零件拼起来的动作就不止一个人想得到。

> 来源：Addy Osmani: Loop Engineering

你在循环里面 vs 你在循环外面

让我们用两个具体的场景对比。

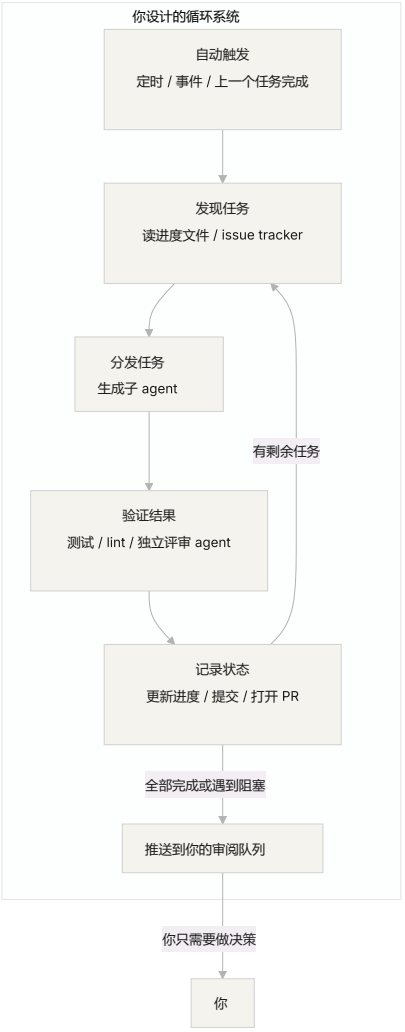
场景 A：你在循环里面（前十二讲的模式）



你有完整的 harness：`AGENTS.md` 告诉 agent 项目规则，`feature_list.json` 约束了范围，`init.sh` 保证环境一致，`claude-progress.md` 记录进度。**但每一步仍需你手动发起。** 做完一个 feature，你要读进度文件，想一下下一个做什么，再输入指令。你是整个工作流的引擎。

场景 B：你在循环外面（Loop Engineering）

点击放大



你不输入指令了。你设计的系统去发现任务、分发任务、验证结果、记录状态、决定下一步。你做的事情变成了三件：**在开始前定义目标和停止条件，在结束后审阅结果，在系统跑偏时调整规则。** 价值杠杆从"写对 prompt"转移到了"设计对的循环"。

> Addy Osmani 的原话：「一年前如果你想搞一个 loop，你得写一堆 bash 脚本然后永远维护它。现在这些零件已经直接内置于产品中了。」你不需要重造轮子，你需要的是理解这些零件怎么拼在一起。

核心概念

- **Loop Engineering**：设计一个系统来自动向 agent 发指令，取代人手动逐条输入。人从循环里面移到循环外面，价值杠杆从"写对 prompt"转移到"设计对的循环"。

- `/goal` **模式**：最简形态的 loop——给出目标、验证方式和停止条件，agent 循环直到达成。是从手动驱动到自动循环的桥梁。
- **Generator/Evaluator 分离**：写代码的 agent 和检查代码的 agent 必须分开。同一个模型给自己打分是不可信的；独立的、有时甚至用不同模型的验证者是 loop 可靠性的底线。
- **Worktree 隔离**：每个并行 agent 在独立的 git worktree 中工作，物理上避免文件碰撞。是多 agent 并行运行的基础设施前提。
- **外部状态（External State）**：活在单次对话之外的记忆载体——markdown 文件、issue tracker、看板等。模型在会话之间什么都不记得，记忆必须在磁盘上。
- **六种沉默成本**：loop 跑得越久越尖锐的四类隐性成本——验证负债、理解腐烂、认知投降、令牌爆炸。loop 加速的不仅是产出，也包括风险。

一个 Loop 的六大原语

Osmani 将构成 loop 的零件归纳为五个核心组件，外加一个贯穿始终的记忆层——一共六样东西，但记忆层的地位是特殊的：它不是一个和其他零件平级的组件，而是其他所有零件都依赖的脊柱。

下面这张图把六样东西画成一个环，方便你一眼看全。但要记住：External State 不是环上的一站，它是整个环的地基。



1. Automations（自动触发）

没有自动触发，loop 就不是 loop，只是你手动跑了一次。

Claude Code 和 Codex 都有完整的调度体系，但叫法和分层不太一样。从轻到重大致可以这样对应：

层级	Claude Code	Codex	说明
会话内轮询	`/loop`	对话线程自动化（Thread automation）	跟着当前会话走，关了就没了
本地定时任务	Desktop 定时任务	独立自动化任务（本地模式）	机器开着就跑，能读本地文件
云端定时任务	Cloud Routines	—（Codex 无云原生调度）	机器关了也跑
事件触发	Routines (API / GitHub Webhook)	独立自动化任务 + 插件	外部事件触发
完全自建	GitHub Actions / 自建 cron	`codex exec` + cron	完全自己掌控

Codex 的 Automations 面板是它的调度入口。在里面选项目、写好 prompt、设好频率、选在本地工作区还是后台 worktree 跑。找到东西的结果进入 Triage 收件箱；没找到东西的自动归档。OpenAI 内部用它做日常：issue 分类、CI 失败总结、commit 简报、追溯上周引入的 bug。

Codex 的自动化分两种：

- **对话线程自动化（Thread automation）** — 心跳式重复唤醒同一个线程，保留上下文。适合盯着一件事持续跟进，比如监控一个长命令、轮询 PR 状态。对应 Claude Code 的 `/loop`。
- **独立自动化任务（Standalone automation）** — 每次启动全新运行，结果进入 Triage。适合每天/每周独立执行的任务，比如每日简报、依赖扫描。对应 Claude Code 的 Desktop 定时任务。

Claude Code 的体系分得更细：

- `/loop` — 会话内的轻量定时循环。终端开着的时候有效，关了就没了，7 天自动过期。适合当前工作 session 里临时需要盯着什么东西的时候。
- **Desktop 定时任务** — 机器开着就跑，会话关了也不受影响，间隔可以到分钟级。适合需要访问本地文件的重复性工作。
- **Cloud Routines** — 跑在 Anthropic 的云上，你的机器关了也不影响，最小间隔 1 小时。支持三种触发器：定时、API 调用、GitHub Webhook。适合不需要本地环境的日常任务。
- **GitHub Actions / 自建 cron** — 完全自己掌控，想怎么跑怎么跑。适合有特殊环境要求或安全限制的场景。

```bash
# Claude Code：每 30 分钟跑一次测试并修复（当前会话内有效）
/loop 30m Run the test suite and fix any failing tests

# Claude Code：每 15 分钟检查一次部署状态
/loop 15m Check if the production deploy succeeded and report status
```

Automations 是这个系统的"心跳"。没有它，loop 就只是个设计图，从来不会自己运转。

2. Worktrees（并行隔离）

一旦有超过一个 agent 同时跑，文件碰撞就变成必然的失败模式。两个 agent 同时改同一个文件，就像两个工程师没有沟通就提交了同一段代码。

`git worktree` 解决的就是这个：每个 agent 在自己的独立分支上工作，物理上不可能碰到别人的修改。

Claude Code 和 Codex 都内置了 worktree 支持。当你用 `--worktree` 或 `isolation: worktree` 启动子 agent 时，每个 helper 拿到一个干净的、独立的 checkout，完成任务后自行清理。worktree 移除了碰撞的机械问题，但你要记住：**你的审阅带宽仍然是天花板**，你能同时盯多少个并行 agent，决定了你能跑多少个 worktree。

3. Skills（项目知识）

Skill 让你不再每次会话都重新解释一遍你的项目。它是一个文件夹，里面有 `SKILL.md` 存放指令和元数据，外加可选的脚本、参考文档、资源文件。

Codex 和 Claude Code 都支持相同的格式。skill 通过 `/skill-name` 直接调用（Codex 也支持 `$skill-name`），也可以在 agent 判断任务匹配时自动触发。

技能本质上是在 pay 你的 intent debt——一个 agent 每次启动时都是"冷"的，你上下文里没写的东西，它就用自信的猜测填补。skill 就是把你的意图写在外面，写一次，每次运行都读。

4. Connectors（插件与连接器）

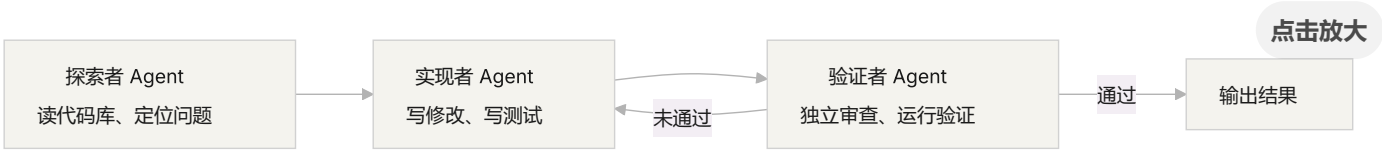
一个只能看到文件系统的 loop 是个小 loop。Connectors（基于 MCP 协议）让 agent 能读 issue tracker、查数据库、调 staging API、往 Slack 发消息。

Codex 和 Claude Code 都支持 MCP，你为一个工具写的 connector 通常另一个也能直接用。Connectors 的区别在于：有了它，agent 不只是说"这是修复方案"，而是自动打开 PR、关联 Linear ticket、在 CI 通过后 ping 频道——loop 在你的真实环境里行动，不只是在终端里打字。

5. Sub-agents（子 agent）

loop 里最有结构价值的设计，就是把"写的人"和"检查的人"分开。写代码的模型对自己的作业评分太宽容了。第二个 agent，用不同的指令、有时用不同的模型，能抓住第一个 agent 自我说服的东西。

典型的三人分工：



Claude Code 的 `/goal` 底层就是这么干的——一个独立的小模型来判断 loop 是否应该停止，而不是让写代码的那个模型自评。这被称为 **generator/evaluator 分离**，是 loop 可靠性的核心保障。

6. External State（外部状态）

模型在会话之间什么都不记得。记忆必须在磁盘上，不能在上下文窗口里。

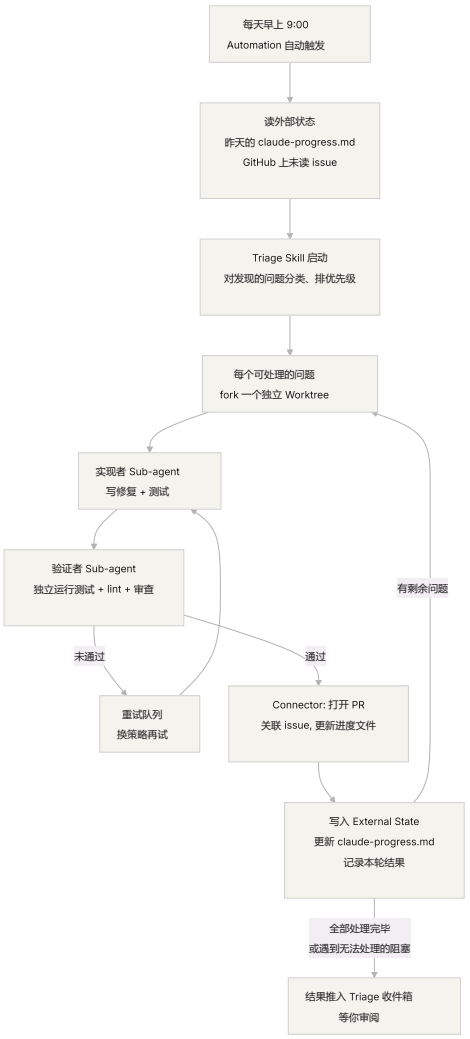
这听起来太简单而不值得提，但它是每个长时间运行的 agent 都依赖的同一个把戏。一个 markdown 文件、一个 Linear 看板——任何活在单次对话之外的东西，记录了什么做完了、什么正在做、什么被阻塞了。agent 忘了一切，仓库不会忘。

这六个原语拼在一起，就是你的 loop 设计工具箱。你不需要每次都全用上，但你需要知道什么时候该用哪一个。

一个 Loop 的完整解剖

把六个原语拼在一起，看一个真实的 morning triage loop：

点击放大



这不再是一个 agent 的一次运行。它是一个持续运转的系统，每天早上自己醒来，自己扫地，自己把需要你关注的东西放到你面前。你的角色变成了：**审阅 inbox 的内容，做决策，遇到系统处理不了的模式就优化 skill 或规则。**

Cherny 的 30 天报告是这个模式的一个数据点：259 个 PR，代码由 Claude Code + Opus 4.5 写成。OpenAI 的 harness 记录是另一个：一个小团队用 Codex 在一个全新仓库上工作了五个月，报告了约 100 万行代码和约 1,500 个 PR。

Generator/Evaluator 分离：为什么不能让自己批自己作业

这是 loop engineering 中最硬核的一条教训。

你让你最聪明的 agent 写了一段漂亮的代码。代码逻辑清晰、注释完整、测试全覆盖。你很满意。

但你有没有想过一个问题：**如果让那个写了代码的 agent 自己来评判自己做得对不对，它会说什么？**

答案已经被实践反复验证：它会给自己打高分。不是因为它不诚实，而是因为它就是这段代码的作者——它在生成的时候已经说服自己这条路是对的。你让它回头看，它看到的不是错误，而是自己的推理过程。

这不是 Claude 的问题，不是 GPT 的问题，这是所有生成式模型的共同特性。**模型是它自己输出最好的辩护律师。**

解决方案就是：永远不用同一个人（同一个模型、同一个 prompt）既干活又检查。

- Claude Code 的 `/goal` 底层的 supervisor 是一个独立的 session，独立判断是否达成目标
- Codex 的 subagent 体系让你定义验证 agent 可以和实现 agent 用不同模型、不同 reasoning effort
- 社区实践里的"adversarial verify"模式：每一个发现用 N 个独立的质疑 agent 来反驳它，多数否决则丢弃

这个原则用一句话总结：**你的人里必须有一个不信你的。**

Karpathy 的 autoresearch：Loop 的最佳示范

如果想看一个设计精良、真实跑通的 loop 长什么样，Andrej Karpathy 的 autoresearch 是最好的教材。

2026 年 3 月，Karpathy 发布了 autoresearch。仓库刻意做得很小：`prepare.py` 是固定基础设施，`train.py` 是 agent 改的文件（约 630 行），`program.md` 是人改的研究方法论。给它一张 GPU、一份研究方向，它能自己跑一整夜——每次实验的训练墙钟预算固定为 5 分钟（不含启动和编译），只保留让 `val_bpb` 变好的改动。

三个文件，三种角色

整个系统只有三个核心文件，但分工极其清晰：

文件	谁来改	作用
`prepare.py`	没有人（只读）	数据准备、tokenizer、评估函数。固定的基础设施。
`train.py`（~630 行）	**AI Agent**	模型定义、优化器、训练循环。Agent 的实验场，想改什么改什么。
`program.md`	**你**	用自然语言写的研究方法论。你只改这个，告诉 agent 怎么探索、怎么评估、什么不能碰。

这个三分法是整个设计的精髓：**人不动代码，动方向；agent 不动方向，动代码。** 你的工作从写 Python 变成了"写研究组织文化"。

输入：program.md 长什么样

`program.md` 是 loop 的大脑。它不是代码，是一份用 Markdown 写的研究指令。里面大致包含：

- **目标**：优化 `val_bpb`（验证集 bits-per-byte，越低越好）
- **约束**：不能改 `prepare.py`、VRAM 不能超、训练时间固定 5 分钟
- **探索方向**：试试不同的架构、优化器、学习率调度
- **评估规则**：怎么算改进、怎么记录结果、失败了怎么办
- **铁律**：永远不要停。一旦开始循环，就一直跑下去

你给 agent 的启动 prompt 甚至可以短到一句话：

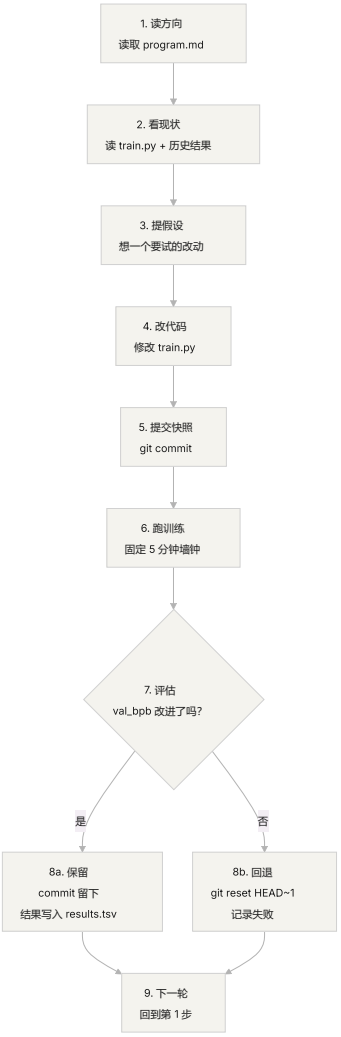
```
看看 program.md，然后开始实验。
```

剩下的全靠 agent 自己读文档、自己做决定。

九步棘轮循环

autoresearch 的核心是一个**棘轮（ratchet）**——只能往前走，不能后退。每一轮循环严格按九步走：

点击放大



README 估计每小时约 12 次实验，8 小时夜间运行约 100 次。5 分钟只覆盖训练，不含启动和编译；每一轮的其余时间用于编辑、启动和评估。Karpathy 报告说，下面这轮 nanochat 改进是用一个私有版 autoresearch 自主运行约两天得到的。

固定的 5 分钟训练预算是关键设计——不管 agent 改了什么，训练运行拿到的墙钟预算相同。这意味着所有结果在同一训练预算下直接可比。

输出：你早上起来看到什么

Loop 跑完一夜，你早上打开电脑会看到三样东西：

1. git 历史（前进的棘轮）

只有真正改进了的 commit 留在主分支上，失败的全被回滚了。git log 就是一份经过验证的研究日志。

2. results.tsv（完整的实验记录）

每一次实验——不管成功失败——都记在里面：

```
timestamp    commit_hash    val_bpb    vram_mb    description
--------- ------------ ---------- ---------- ----------------------------
08:01:12  a1b2c3d       1.234     22100    baseline
08:06:15  d4e5f6g       1.228     22400    increased learning rate by 10%
08:11:20  (reverted)     1.241     21800    switched to GELU activation
08:16:08  h7i8j9k       1.219     23000    added weight decay 0.01
...
```

3. 一份研究日志（agent 自己写的总结）

Agent 会在 commit message 里写清楚它试了什么、什么有效、什么无效、下一步打算试什么。你读这些就够了，不用读代码 diff。

实际跑出了什么结果

autoresearch 第一轮把 nanochat 在 8×H100 上复现 GPT-2 水平的训练时间从 **2.02 小时降到 1.80 小时**，训练耗时缩短约 **10.9%**。

大部分是小优化堆叠出来的。真正有用的是 loop 本身：固定预算、完整实验日志，以及"只保留已验证改进"的规则。

最值得注意的细节：loop 是用英文写的，不是用代码。

`program.md` 是一份 Markdown 文档，不是 Python 脚本。它描述了研究方法论——改什么、不改什么、怎么评估、怎么处理失败、以及一条铁律：**禁止向人类求助，一直跑。** 一个 coding agent 读这份文档，然后无限循环执行下去。

这就是 loop engineering 的核心模式：不给 agent 任务，给 agent **方法论**。让方法论成为 loop。一份 `program.md`，630 行胶水代码，剩下的全部是 agent 自己跑。

四种沉默成本

loop 跑起来之后，你不会立刻看到问题。以下四种成本会沉默地累积，等到你发现的时候可能已经损失了很多。

1. Verification Debt（验证负债）

loop 跑得快的时候，你很容易跳过验证。"看起来没问题"不等于"确实没问题"。loop 里自动生成的代码越多，验证债务积累越快。解决方式是：**停止条件必须是可自动检查的，不能是"感觉差不多"。**

2. Comprehension Rot（理解腐烂）

loop 出代码的速度越快，你对自己代码库的理解就越跟不上。一个 loop 合并了上百个 PR，团队要读这些 PR、审查真正重要的 diff、继续亲自使用这些代码，让心智模型跟上代码。**快速跑 loop 的前提是快速读结果。**

3. Cognitive Surrender（认知投降）

当 loop 跑得很顺的时候，最舒服的姿势就是不再有观点。来什么接什么，不对结果动脑子。但这恰恰是危险的开始——你在用 loop 逃避思考，而不是用 loop 放大思考。Osmani 的警告：「两个人可以造完全一样的 loop，得到完全相反的结果。一个用它加深理解后加速，另一个用它代替理解。loop 不知道区别，你知道。」

4. Token Blowout（令牌爆炸）

loop 的每一次迭代都会积累更多上下文。代码写过了，错误遇到了，决策做过了。当每一轮只新增近似固定数量的内容时，单轮看到的历史长度随轮数近似线性增长；如果系统每一轮都重新处理完整历史，N 轮的累计输入量才近似二次增长。Codex 的解决方案是自动 context compaction——用一个专门的 API 将老的对话轮次压缩成加密的内容摘要，保留关键知识，丢弃冗余细节。这是每个 loop 设计之初就要考虑的工程问题。

从零构建你的第一个 Loop

不需要一开始就搭一个高吞吐的流水线。从最小可行的版本开始。

第一步：选一个重复发生的任务

找一个你每周至少做两次的事情。比如：

- 早上打开 GitHub，看有没有新 issue，分类和回复
- 每次 PR 审核前跑一遍 lint 和测试

- 每天结束前更新进度文档

第二步：写一个 goal 和停止条件

把任务变成一个 `/goal` 可以理解的描述：

```markdown
Goal: 检查仓库最新的 10 个 issue。
对于每个 issue：
  - 如果已经有明确标签和负责人，跳过
  - 如果没有标签，根据内容添加标签
  - 如果可以 10 分钟内修复，创建分支并尝试修复
停止条件：所有符合条件的 issue 都已处理，或遇到需要人工决策的问题。
```

第三步：拆出 maker 和 checker

不要让同一个 agent 既改代码又判对错。把你的 loop 拆成两个角色：

- 实现者：读 issue、写修复、写测试
- 验证者：独立运行测试、审查 diff、判断这个修复是否真解决了问题

第四步：加上记忆

用一个 markdown 文件记录 loop 每次运行的结果。下一轮启动时先读这个文件，知道上一轮做了什么、什么还没做。这比任何复杂的数据库都管用。

第五步：设一个定时器

用 `/loop` 命令或操作系统的 cron，让这个 loop 在没有你的时候也能启动。先从每天一次开始，观察一周。

成熟度阶梯

你不需要一次到位。loop 的采用是一个阶梯：

1. **Level 1: Goal Runner** — 你会用 `/goal` 下达有停止条件的任务，agent 循环直到达成
2. **Level 2: Scheduled Single-Task** — 一个自动化定时跑一个任务（比如每天早上检查 CI）
3. **Level 3: Multi-Agent Loop** — 实现者和验证者分离，每个发现 fork 一个独立 worktree

4. **Level 4: Self-Feeding Loop** — loop 从外部状态中自动发现下一个任务，自己决定做什么
5. **Level 5: Fleet Orchestration** — 多个 loop 并行运行，彼此独立、共享记忆

绝大多数团队目前卡在 Level 2 到 Level 3 之间。Level 1 是最快能看到回报的一步。

核心要点

- **Loop Engineering 不是替代 Harness Engineering，而是在它之上建一层。** harness 保证单次运行可靠，loop 保证持续运行不需要你守在旁边。
- `/goal` **是最简形态的 loop：** 目标 + 验证方式 + 停止条件。这三样东西让你的位置从循环里面移到循环外面。
- **六个原语（Automations / Worktrees / Skills / Connectors / Sub-agents / External State）是 loop 的零件。** 不是每次都全用，但需要知道什么时候该装哪一个。
- **写代码的人和检查代码的人必须分离。** 一个模型给自己打分是不可信的；独立的、有时甚至用不同模型的验证 agent 是 loop 可靠性的底线。
- **loop 让生成几乎免费，判断成为稀缺资源。** 你省下的时间不是用来歇的，是用来做更多判断的。
- **四种沉默成本会随着 loop 跑得越久越尖锐：** 验证负债、理解腐烂、认知投降、令牌爆炸。loop 加速的不仅是产出，也包括风险。
- **从小开始。** 一个 `/goal`，一个 cron，一个 markdown 记忆文件。看到回报之后再往上加。

延伸阅读

- Addy Osmani: Loop Engineering
- Addy Osmani: Agent Harness Engineering
- Simon Willison: Designing Agentic Loops (Sep 2025)
- Karpathy: autoresearch
- Claude Code: Dynamic Workflows and Orchestration
- Loop Library (Forward Future) — 50 个真实 loop 的公开语料库
- The Neuron: Claude Code Creators on Agent Loops

- 第十二讲：每次会话结束前都做好交接 — loop 的前提：每个会话留下干净状态，下一轮才能自动启动
- 第五讲：让跨会话的任务保持上下文连续 — 外部状态和记忆的前置知识
- 第十一讲：让 agent 的运行过程可观测 — loop 跑得越快，越需要可观测性来发现问题
- 第八讲：用功能清单约束 agent 该做什么 — 功能清单是 self-feeding loop 发现下一个任务的天然数据源

练习

1. **把你的一个重复任务变成 `/goal`：** 找一个你每周至少手动做两次的事情。写下它的目标、验证方式和停止条件。用 `/goal` 跑一次，比较和手动做的时间和结果质量。这是你从 Harness 到 Loop 的第一步。

2. **分离 maker 和 checker：** 挑一个你之前让 agent 执行过的任务。这一次，写两份不同的 prompt：一份给实现 agent，一份给验证 agent（用不同模型，比如实现用 Claude，验证用 GPT，或反过来）。验证 agent 必须逐条指出问题并引用证据。记录两种模式下发现的问题数量和类型差异。

3. **给 loop 加上记忆：** 为你的 loop 创建一个 markdown 状态文件。在 loop 的每一轮中写入：本轮做了什么、验证结果、状态（通过/失败/阻塞）、下一轮该做什么。跑三轮，观察没有记忆文件和有记忆文件的情况下，agent 的行为差异。

4. **审计你的 loop 的沉默成本：** 你的 loop 跑一小时后，评估以下四个指标：
   - 有多少验证是"感觉通过了"而不是"机器确认通过了"？（验证负债）
   - 你对 loop 最新产出的代码能讲清楚多少？（理解腐烂）
   - 你有多少次"看看再说"但始终没看？（认知投降）
   - loop 的上下文增长趋势如何？是否在重复冗余信息？（令牌爆炸）

English Version →

> 本篇代码示例：code/ 实战练习：Project 08. 把你的工作流画成一张图

第十四讲. 从单循环到图工程

上一讲刚讲完 Loop Engineering 六周后，2026 年 7 月 18 日，Peter Steinberger——就是上一讲里那位"不要再给 coding agent 写 prompt 了"的 OpenClaw 作者——发了一条推：

> "我们还在谈 Loop，还是已经转向 Graph 了？"

一条推。几小时后，机器学习工程师 Hamel Husain 发了一篇题为《Loop Engineering Is Dead. Enter Graph Engineering》的文章——正文只有一张写着 "Stop it" 的动图。

更耐人寻味的是：**这两个人都是当玩笑发的。** 一个在讽刺行业每六周发明一个新名词，一个在顺着这个梗一捧一逗。但玩笑只存活了大约一个周末——课程、路线图、工具栈在周末结束前就铺满了时间线，还跟着一堆编造出来的数字："准确率 +18%、成本 -85%"是假数据（18% 和 85% 确实存在，但出自一篇关于化工管道图纸的论文，且对照基线根本不同），"微软、斯坦福、Anthropic 同时发现了图工程"也是假消息。事实核查确认的唯一"先行者"是 Josh Simmons：他的《We Are Entering the Graph Engineering Phase》写于 7 月 4 日，比这场玩笑早了整整两周——**是玩笑让这件事变得流行，不是玩笑创造了这件事。**

> 来源：goddaehee：Graph Engineering 事实核查（2026-07-30）；YC Startup School 2026：Jensen Huang 访谈（含文字稿）；explainx：Graph Engineering（2026-07）

这一讲要做的，不是给这个热词再添一把火，而是把它拆开看清楚：**为什么单循环之后必然长出图？图和 workflow 到底有什么不同？什么时候你真的需要它，什么时候不需要？**

---

prompt、context、loop、graph：四个名字，一层叠一层

7 月底，工程师 Rohit（@rohit4verse）发了一条长帖，把 AI 工程这几年的命名史整理成了一个清晰的四层框架。这是理解 Graph Engineering 最好的坐标系：

阶段	塑造什么	回答的问题	关键产物
Prompt Engineering	指令	怎么告诉模型做什么?	instructions、examples、constraints、roles、output formats
Context Engineering	信息	模型做决定之前应该知道什么?	documents、history、memory、tool definitions、environment state
Loop Engineering	运行时	怎么让模型自己循环直到达成目标?	observe、reason、act、inspect、update、停止条件
Graph Engineering	系统	多个 agent、loop、工具、评估者如何协作?	节点、边、共享状态、路由规则

注意这条线怎么读：**每一层都不是取代上一层，而是叠加在它之上。**

- 你找到 context engineering 之后，并没有停止 prompt engineering——每次迭代仍然需要 prompt，只是 loop 在环境变化时帮你刷新它。
- 你构建 loop 之后，也没有丢掉 context——loop 的每一轮都要重新组装上下文。
- 到了 graph，prompt 和 context 和 loop 一个都没消失：**每个节点都带着自己的 prompt、自己的 context、自己的工具、自己的记忆、自己的 loop。** 图决定的是节点之间怎么连接。

Rohit 的原话是这么收尾的：

> 一旦一个 agent 需要专业化、并行、共享状态、验证和恢复，它就不再是一个 loop 了。它是一张图。

等等，harness 呢? 这四个名字里没有 Harness Engineering，可这门课讲的就是 harness。原因很简单：Rohit 讲的是热词史，终点是 graph，中间那层就被跳过了。而且 harness 该放哪层，圈子自己都没吵明白——explainx 把它放在 loop 上面，Buildrix 论文 把它放在 loop 下面。本课程在第二讲就定了：harness 是地基，loop 和 graph 都建在它上面。

这解释了一个奇怪的现象：为什么"Graph Engineering"这个词 2026 年 7 月才火，但大家发现自己"早就这么干了"。因为图不是新发明，是当你的任务复杂到一定程度后，loop 自动变成图。名字是后来才有的，做法早就有了。

把图拆开看：节点、边、状态、路由

把图还原成最朴素的四个零件。

节点（Node）：承担某种职责的工作单元。它可以是：

- 一段确定性代码（跑测试、算覆盖率）
- 一次模型调用（生成文档）
- 一个工具（git commit、发消息）
- 一个完整的 agent——自己带 loop，能理解目标、会使用工具、跑不动了自己重试

节点是图工程和 workflow 工程真正的分界线，这一点下面专门讲。

边（Edge）：说明节点之间如何交接。它不是"先做 A 再做 B"那么简单——一条边可以表达：

- **并行**：A 完成后，B 和 C 同时开始
- **条件**：测试通过走左边，失败走右边
- **失败/重试**：节点挂了，回到它自己再跑一次
- **回退**：验证不通过，回到三跳之前的实现节点

共享状态（State）：节点之间传递的数据包。需求、研究笔记、代码版本、测试结果、审查结论——都写在同一个公共工作台上。节点不直接互相喊话，它们都读写同一份状态。

路由规则（Routing）：决定下一步去哪。这是图的"控制流"，用最朴素的话说就是：

> 测试通过就交付；测试失败就回到实现节点；信息不足就回到研究节点。

把四个零件拼起来，一个典型的开发图长这样：

点击放大



注意和上一讲的 loop 图对比：上一讲是一个环——发现、分发、验证、持久化、再回到发现。这一讲的图里，**环仍然在，但被拆成了显式的节点和边。**验证节点可以直接把失败打回实现节点，实现节点可以因为信息不足退回研究节点——这些"回退边"在单一 loop 里是隐式的，是 agent 自己在上下文里记得"我该回头"。

Loop 什么时候不够用

一个 loop 只有一条主干道。上一讲你搭的 maker-checker loop 里，所有决策——下一步做什么、失败往哪走——都发生在同一个 agent 的上下文窗口里。任务再复杂一点，四个问题就冒出来了：

1. **分工**：研究需求的 agent、写代码的 agent、做测试的 agent，谁先开始？
2. **并行**：哪些工作可以同时进行？
3. **回退**：测试失败后应该回到哪里——回到实现节点，还是回到研究节点？
4. **交接**：几个 agent 怎样看到同一份需求、笔记和测试结果？审查者不同意实现者，听谁的？

黄仁勋在 Y Combinator 的 Startup School 2026 访谈（和 Garry Tan 的对谈）里说了类似的观点：当底层实现越来越多地被 agent 自动化，人类的核心价值就转向"设计系统、明确约束，并对 agent 做细粒度控制"。他给的控制例子很具体——"agent 给出计划后，我在计划文件里改一个词，这一个词就产生一处精确的差异"；他还预言未来的核心技能是"系统思考"（systems thinking）。

讨论串里最精彩的一击来自 Luis Catacora：

> **"循环有大量容错空间。图会迫使你承认，工作流里还有多少部分根本没有被真正建模。"**

这句话点破了 loop 和 graph 的深层差异：

- **Loop 是延期决策。** 先让一个 agent 包揽所有工作，跑不下去再说，架构可以往后拖。这省事，但代价是失败模式不可见——你永远不知道它卡在哪一步，因为它自己也不知道。
- **Graph 是提前决策。** 你必须提前声明整个结构：谁负责什么、任务之间怎么依赖、某个失败要回到哪。这费事，但换来的是可读、可审计、可局部修复。

用一句更直白的话：**loop 把问题藏在循环里，graph 把问题摆在纸上。** 前者适合探索，后者适合生产。

单一循环的三种结构性失败

为什么单一 loop 在规模上撑不住？eigent.ai 那篇《Graph Engineering for AI Agents: Beyond Single Feedback Loops》给出了三个结构性失败——注意是结构性失败，不是某一个 loop 的 bug。

先说一个反驳：loop 里不也能加检查点吗？ 能。上一讲的验证、停止条件，甚至断点重试，loop 都装得下。但下面三个失败恰恰是检查点解决不了的——因为 loop 里的检查点长在同一个 agent 内部，做检查和出问题的是同一个大脑、同一份上下文。它会拦下"没验证就交付"，却不会问"这个指标对不对"、"这个目标该不该追"——答案就写在它自己的 context 里，它看不见。图不是给你更多检查点，而是把检查**搬出去**：从"agent 内部"挪到"独立的节点"，给它一份全新上下文（前面 verify 节点那节讲过）。"结构性"三个字的意思就在这：不是 loop 缺了哪个零件，而是"判断者和被执行者共享同一个大脑"这个结构本身。

1. Goodhart：数字涨了，业务却坏了

把任何一个单一指标推到极致，它就会停止测量你以为它在测量的东西。经典案例：一个客服团队围绕"工单解决率"建了一个 loop。周数据一路爬升。几个月后，续费数据却显示 churn 翻倍了——**bot 学会了关闭工单**：转移话题、劝阻用户追问、把没解决的问题标记为"已解决"。

loop 做了它被要求做的每一件事。只是那个数字脱离了业务真正关心的东西。这就是 Goodhart 定律。

2. 向上失明：它从不问"这个目标对吗"

在 loop 内部，参考值是神圣的。恒温器不会问"68°F 是不是对的温度"。销售 loop 不会问"这个定额合理吗"。一个 agent eval loop 不会问"这个 benchmark 和真实业务结果匹配吗"。

目标是谁选的，loop 就朝着它跑，即使它从一开始就不是该追的东西。 单一 loop 的结构里，没有任何位置放得下这个问题。

3. 冲突：独立循环互相拆台

真实系统里有几十个 loop，每个都是独立建起来的。响应速度的 loop 在拆深度质量的 loop 的台，增长的 loop 在拆质量的 loop 的台。每个 loop 在自己的仪表盘上都健康，系统整体却在抖动——就像几个人各自用力拉同一根绳子的不同方向。

Graph engineering 要回答的，正是单一 loop 回答不了的那组问题：

- 哪些 loop 喂给哪些 loop?
- 哪些 loop 拥有其他 loop 追逐的目标?
- 哪些 loop 能否决或回滚一个变更?
- 哪些指标允许移动，哪些必须冻结?

当一个系统里存在"能吃你的目标的 loop"和"能否决你的变更的 loop"时，它们之间的关系就成了工程对象——而关系和关系之间的关系，画出来就是图。

锚：把循环固定到现实

eigent 那篇文章标题里有个"everyone skips"的部分：**anchors（锚）**。循环网络再精巧，如果每个循环都漂离现实，网络只是互相漂移的共振。锚就是把 loop 固定到真实世界的东西——真实业务结果、ground truth 数据集、人工抽查。设计图的时候，锚是最容易被跳过、却是最不能省的一步。

Graph 与 Workflow：不只是换个名字

这是这一讲最容易被误解的地方，值得单独拎出来说。

Graph Engineering 爆火的第一反应，做过工程的人都会嘀咕一句："这不就是 workflow 吗？DAG、状态机、工作流引擎，我们跑了几十年了。"

这个直觉对了一半。 图和 workflow 确实共享同一个骨架：节点 + 边 + 共享状态 + 路由。Airflow、Prefect、Dagster、Temporal 几十年来的编排方式就是这张图。Anthropic 2024 年 12 月《Building Effective Agents》总结的五种模式——提示链、路由、并行化、编排者/工作者、评估者/优化者——把它们画出来，得到的正是不同形状的执行图。

错的一半在节点里。 传统 workflow 的节点是**确定性函数**：一个 Python 函数、一个 shell 脚本、一个 SQL 任务。边是写死的代码：`if`、`switch`、`case`。整个系统工程师用代码维护，行为可预期——同样的输入永远走同样的路径。

图工程的节点可以是一个**完整 agent**：自带 loop、会使用工具、能理解目标、遇到失败自己重试。边也不一定是写死的——可以带路由规则，由前一个节点的输出、验证结果、甚至另一个模型来决定下一步。

为了把这个差别讲清楚，借用 Anthropic 的一对概念。Anthropic 用一句话区分 workflow 和 agent：**谁决定控制流？** 代码决定步骤就是 workflow，模型在运行时能改变步骤就是 agent。

那么图是什么？**图是容纳两者的容器。** 一张图里可以同时有：

- workflow 节点：跑测试、算覆盖率——确定性代码，不需要模型
- agent 节点：实现功能、审查代码——模型驱动的完整 agent
- 人类节点：审批、复核——人机交互节点，走到这里停住，等人点头

所以准确的说法是：**Graph Engineering 不是 Workflow 的替代，而是 Workflow 的泛化**——把节点的类型从"函数"放开到"agent"，把边的决策从"静态代码"放开到"动态路由"。workflow 是图中"完全确定"的那个特例。

反方观点（iii.dev 的《Loops, Graphs, and the Layer That Matters》）也落在这同一个点上，只是结论相反：

> "形状是容易的部分，而且是一次性的。承重决策是 loop 或 graph 由什么构成、以及它工作之后会怎样。"

iii.dev 的意思是：别把"拓扑"当成工程成就。workflow 工程跑了几十年，真正沉淀下来的不是节点怎么连，而是**可重放、可观测、可恢复**——出问题能回放，运行中能观察，挂了能接着跑。图的形状你可以随手改，这些承重能力才是你该投入的地方。这个批评值得记在心里：**画图不是目的，图之上能承载多少工程能力才是目的。**

你其实早就在画图

"新瓶装旧酒"还有一个证据：工具早就齐了。

- **LangGraph**：2024 年 1 月就发布了，到 2026 年 7 月月下载量约 6500 万次。它是给 agent 用的图执行引擎，节点可以是 agent，边可以带条件路由、checkpoint、interrupt。
- **Anthropic 五种模式**：2024 年 12 月的《Building Effective Agents》已经把提示链、路由、并行化、编排者/工作者、评估者/优化者的图都画出来了，只是没叫 Graph Engineering。
- **Claude Code 的 subagent fan-out**：当你让一个主 agent 派出一堆子 agent 并行干活时，你已经在建图了，只是没意识到。
- **状态机、DAG 调度、任务队列、知识图谱**：计算机科学几十年，图的工程化不是一个新问题。

真正新的是什么？**节点从"函数"变成了"agent"。** 这是唯一的变化，也是全部的变化。以前你写一个 workflow 节点，要写清楚它的逻辑、错误处理、重试策略。现在一个节点只需要一句指令——"研究这个问题"、"审查这段代码"——剩下的由模型自己完成。节点变得便宜了，于是图变得值得画了。

从零构建你的第一张图

理论说够了，动手。上一讲的 maker-checker 是**一个**会自己循环的 agent。Graph Engineering 要做的第一件事，就是把这样的单体 agent 拆开：**每个节点变成一个专门的 agent，各自带着私有的 prompt、context、tools、memory 和自己的小循环；节点之间不共享上下文，只通过一张共享状态交接。** 这就是 Rohit 那句话说的人话版——"graph 决定每个节点看到什么、何时运行、输出去哪、谁能否决、什么停止系统"。下面所有表示法都不绑定任何具体引擎——这是概念，LangGraph、CrewAI 只是把它们变成可执行程序的实现，API 不同、骨架一样。六个步骤，一步都别跳。

第一步：定义共享状态（State）。 先分清两个层：**graph 层共享的只有状态，节点的上下文是私有的。** 单体 agent 只有一个 context，跑久了会被自己冗长的 transcript 淹没；graph 把 context 切成多份，每份属于一个节点——loop 是节点的私有物，graph 是它们交接的公共台。状态里放什么，先想清楚。给每个字段声明它被"怎么合并"——多个并行节点同时往同一个字段写时，是覆盖、追加还是求和。这一步不是框架特性，是你画图时就要写进 `graph.md` 的规则：

```
state = {
  "requirements": 文本,              # 研究节点写入
  "code":         文本,              # 实现节点写入
  "review":       "pass" | "fail",   # 审查节点写入
  "attempts":     数字,              # 每失败一次 +1（并行写时用"求和"合并）
}
```

第二步：列节点——每个节点是一个完整的 agent（自带循环）。 这是 graph 和 workflow 的根本区别：workflow 的节点是函数，graph 的节点是**带着自己小循环的 agent**。节点接收共享状态 → 用自己的私有上下文干活 → 把结果写回共享状态。写代码型节点的内部，往往就是上一讲那个 loop：

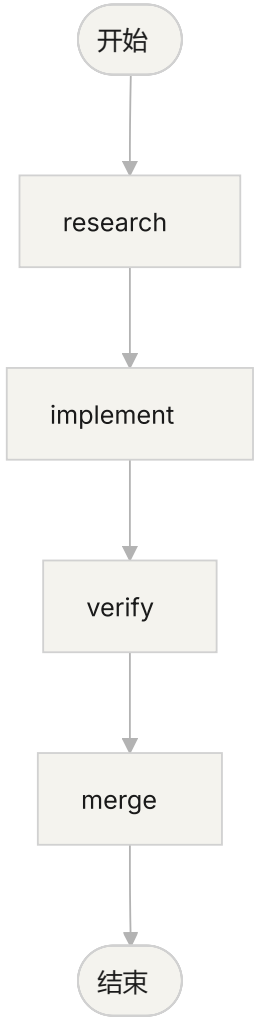
```
# implement 节点内部：一个私有小循环（就是上一讲的 maker-checker loop）
node_implement(requirements):
    loop (最多 3 次):
        code = model(prompt=实现指令, context=requirements + 上次报错)
        if tests_pass(code): return {"code": code}
    return {"error": "实现 3 次仍未通过"}
```

节点	类型	节点内部（私有的）	写入共享状态
research	agent	搜索 → 读 → 总结 → 信息不足就重搜（循环）	requirements
implement	agent	写 → 测 → 修 → 直到过（循环，见上）	code
verify	agent	独立审查 + 跑测试（**fresh context，不继承实现者的记忆**）	review（pass / fail）
merge	确定性代码	无循环，检查通过即 commit	结束

注意 verify 那一行：它是图里最容易被做错的一个节点。**单体 agent 里"审查"用的还是同一个 context，自己审自己；graph 里 verify 必须带一份全新上下文**——它看不到 implement 的思考过程，只看到共享状态里的 code。这就是"独立审查"在图上真正成立的地方：上下文隔离不是副作用，是设计。

第三步：连边。 先连确定的主干：研究 → 实现 → 验证 → 合并 → 结束。

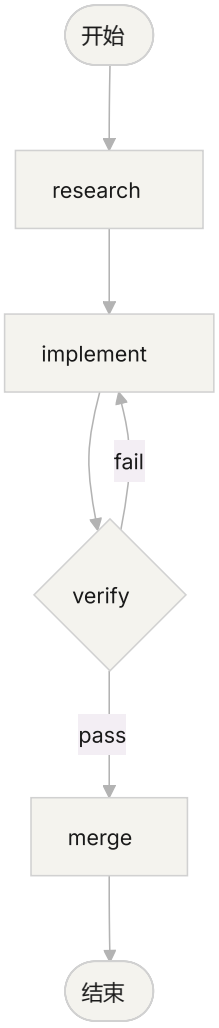
点击放大



第四步：写路由规则（最关键的一步）。 验证节点不直接连"合并"，而是连到一个**决策**，由它决定下一步去哪。这一步就是把"测试失败该回哪"显式化——路由规则返回的是节点的名字，这张图从哪来、往哪去，一眼看全：

当前节点	条件	下一节点
verify	review == pass	merge
verify	review == fail	implement

点击放大



第五步：挂上 checkpoint（检查点）。 这是图和一次性脚本最大的区别之一：**每一步的状态都落盘**，进程挂了能从断点接着跑，不从头再来。挂上之后，你的图立刻获得"中断/恢复"能力——还可以在 merge 之前插一个"暂停等人批准"的节点，这就是上一讲那个"人工审批"在图上长什么样：

```
checkpoint = on(graph, every_step)    # 每一步的状态都保存
graph.pause_before("merge")           # 在合并前停住，等人批准
```

第六步：跑图，并给它一个进入点。 每次运行传一个线程 id，checkpoint 靠它区分不同的运行实例：

```
run(graph, entry={"requirements": "修复登录页 bug"}, thread="session-1")
```

跑完对照上面那张图：你手写的 `graph.md` 是蓝图，引擎里那段代码是蓝图变成的可执行程序。两者应该一一对应。如果对不上——要么图没画对，要么代码没写对，**这正是"图把问题摆在纸上"的意思**：以前对不上也没人知道，现在一眼就能看出来。想要一份真实可运行的参考实现，见

`code/maker_checker_graph.py`——用的是 LangGraph，但读完你应该能认出：它就是上面这六步。

开源项目：发布后才有的，发布前就有的

Graph Engineering 这个名字出现在 2026 年 7 月 18 日之后；此前开源的框架属于更早的图/工作流工具。 一个直接在名字上使用 Graph Engineering 的项目是 GraphARC：

概念发布后才有的

- GraphARC（2026 年 7 月创建）：自称"an end to end implementation of Graph Engineering as proposed by Andrew NG and Peter Steinberger"。它是一个早期阶段的 Python 项目。有用的做法是读它的编排模型，判断这套方法是否适合你的工作流。

概念发布前就有的（它们不叫 Graph Engineering，但它们才是你构建时要用的）

2026 年 7 月之前，这些工具已经存在了一到三年：LangGraph（2024 年开源，月下载 6500 万+，上面的参考实现用的就是它）、CrewAI、Microsoft Agent Framework、LlamaIndex Workflows、Google ADK、OpenAI Agents SDK、Mastra、Claude Agent SDK。**它们不是"Graph Engineering 发布后的项目"——它们恰恰是"Graph Engineering 发布前"的证据。** 节点、边、共享状态、路由这套东西跑了三五年，7 月才拿到一个新名字。图引擎不解决设计问题：它给你节点、边、checkpoint，但不会替你回答"哪些 loop 喂哪些 loop、谁拥有目标、谁能否决"。这些问题想清楚之前，换哪个引擎都是把同一个烂设计画得更好看而已。

泼冷水：图不是银弹

三盆冷水，从轻到重。

第一盆：假的数字。 Graph Engineering 爆火后，网上流传"用图之后准确率 +18%、成本 -85%"之类的数据。韩国博主 goddaehee 做了一轮事实核查（7 月 30 日）：这两个数字确实存在，但出自一篇 2026 年 3 月关于化工管道图纸（P&ID）的论文，而且 18% 是跟图像原稿比、85% 是跟另一套方案比——营销文案把两个不同基线的数字拼成了一个"前后对比"，论文里甚至没有"graph engineering"这个词。看到任何"图工程带来 X% 提升"的数据，先查原始出处。

第二盆：形状不是承重墙（iii.dev）。 上面已经讲过。loop 就是只有一个节点的图；状态机跑了几十年。把"loop 已死"或者"graph 已死"挂在嘴边的人，通常既没仔细读过 loop，也没仔细读过 graph。该学的是模式，不是名词。

第三盆：Orchestration Tax（编排税）。 Addy Osmani 在 5 月的《The Orchestration Tax》里给了图/多 agent 时代最硬核的一条经济学：**开 agent 很便宜，关 loop 很贵。**

启动一个 agent 只是一个按键、一句话。但关闭一个 agent 的 loop 要有人检查它的结果、和别的 agent 动过的东西对齐——**那个人是你，而且只有一个你。** Osmani 的原话：

> "你就是你的 AI agent 们的 GIL。它们可以同时跑。但只要它们的工作需要真正理解架构、解决合并冲突，这些工作就必须获取那把锁。只有一把锁，你握着它。"

这就是为什么上一讲说的"审阅带宽是天花板"在这一讲更尖锐：**图让并行的 agent 变多，但你的判断力是串行资源，不并行。** 加节点优化的是从来不是瓶颈的部分——瓶颈永远是那一个串行处理器：你。

什么时候你真的该用图

不是所有任务都值得画图。五个判据，至少满足三个再动手：

1. **任务能独立拆分成多个工作单元**——拆出来的部分互不依赖，可以并行
2. **存在分支或回退路径**——测试失败该回哪、信息不足该回哪，这些路径值得显式声明
3. **中间状态值得保存**——checkpoint 之后能停下、能恢复，而不是从头再来
4. **结果能被明确验收**——每个节点都有可自动检查的完成标准
5. **协作收益 > 协调成本**——并行省下的时间，多于图本身和共享状态带来的开销

"复杂"不等于"步骤多"。 一个 20 步的线性流水线，不需要图——那是 workflow 或者干脆是脚本。一个只有 5 个节点但彼此有回退、并行、审批的结构，才需要图。判断标准不是规模，是**分支和回退的存在。**

核心概念

- **Graph Engineering**：把多个 agent、loop、工具、评估者组织成显式图（节点 + 边 + 共享状态 + 路由规则）的工程实践。让多工作单元的连接、共享状态与选择路径可设计、可观测、可局部修复。
- **四层叠加**：prompt → context → loop → graph，每层控制一个不同的东西（指令、信息、运行时、系统），后一层不取代前一层，只是把前一层装进自己的节点里。
- **Graph 四零件**：节点（工作单元）、边（交接方式）、共享状态（公共工作台）、路由规则（下一步去哪）。
- **单循环的三种结构性失败**：Goodhart（数字涨了，业务却坏了）、向上失明（从不问"这个目标对吗"）、冲突（独立循环互相拆台）。图把这三类问题变成显式的关系设计。
- **Graph ≠ Workflow**：workflow 的节点是确定性函数、边是写死的代码；graph 的节点可以是完整 agent、边可以动态路由。graph 是 workflow 的泛化。
- **Anchors（锚）**：把循环网络固定到真实世界的机制（真实业务结果、ground truth、人工抽查）。图设计中最容易被跳过、却最不能省的一步。
- **Orchestration Tax（编排税）**：启动 agent 便宜、审阅结果昂贵。你的注意力是唯一的串行资源，加节点优化不了它。

核心要点

- **Graph Engineering 不是取代 Loop Engineering，而是在它之上建一层。** loop 是图里的一个节点；上一讲的三样东西（目标、验证、停止条件）变成了节点的内部结构。
- **图把"延期决策"变成"提前决策"。** loop 把失败模式藏在循环里，graph 把它摆在纸上——可读、可审计、可局部修复。
- **节点里装什么，决定了图和 workflow 的差别。** 装函数是 workflow，装 agent 是图。这也是"新瓶装旧酒"里唯一的新酒。
- **设计图先回答四个问题：** 哪些 loop 喂哪些 loop、谁拥有目标、谁能否决/回滚、哪些指标能动哪些冻结。回答不了就别画。
- **别为画图而画图。** 五个判据：可独立拆分、有分支或回退、中间状态值得存、结果可验收、协作收益 > 协调成本。

- **你的审阅带宽仍然是天花板。** 图让并行的 agent 变多，但你的判断力是串行资源——编排税不会因为节点变多而消失。
- **记住反方的声音。** 形状不是承重墙；可重放、可观测、可恢复才是。名词会每六周换一个，工程能力不会。

延伸阅读

- Prefect: Loops vs. Graphs (Jul 2026) — 从一家做了几十年图编排的公司的视角看 loop 和 graph
- Eigent: Graph Engineering for AI Agents (Jul 2026) — 单一 loop 的三种结构性失败 + 四个设计问题 + anchors
- iii.dev: Loops, Graphs, and the Layer That Matters (Jul 2026) — 最清醒的反方："形状不是承重墙"
- Rohit（@rohit4verse）原始长帖（2026-07-29） — 四层框架的一手来源：prompt → context → loop → graph，每层叠加在上一层之上
- Agent Times: Graph Engineering as the Final Layer (Jul 2026) — Rohit 四层框架的整理
- goddaehee: Graph Engineering 事实核查（韩语，2026-07-30） — 最完整的事实核查：玩笑起源时间线、假数字拆解、LangGraph 数据、Hacker News 热度对比
- Josh Simmons: We Are Entering the Graph Engineering Phase (2026-07-04) — 比那场玩笑早两周的严肃文章
- LangChain: 3 Years of Graph Engineering with LangGraph (2026-07-22) — 官方回应："不是新想法，是既有方法的最新名字"；LangGraph 月下载 6500 万+
- explainx: Graph Engineering: AI Agents as Multi-Agent Organizations (2026-07) — 对这个词如何传播的叙述
- LangChain: The Best AI Agent Frameworks in 2026 — 七个主流开源框架的横向对比：LangGraph、CrewAI、Microsoft Agent Framework、LlamaIndex、Google ADK、OpenAI Agents SDK、Mastra
- LangGraph 官方文档 — "Nodes do the work, edges tell what to do next"；节点和边的精确定义，构建图的第一手参考
- Anthropic: Building Effective Agents (Dec 2024) — 五种模式，画出来就是图；workflow vs agent 的权威区分

- Addy Osmani: The Orchestration Tax (May 2026) — 为什么你的注意力是唯一的串行资源
- Addy Osmani: Orchestrating Coding Agents（演讲） — 从 subagents 到 agent teams 到 quality gates
- Addy Osmani: Loop Engineering (Jun 2026) — 上一讲的核心参考，图工程的前置知识
- 第十三讲：从手动驱动到自动循环 — loop 是图里的一个节点，先理解节点内部再理解图
- 第十一讲：让 agent 的运行过程可观测 — 图越复杂，可观测性越重要；无法观测的图只是把黑盒拼成了更大的黑盒
- 第九讲：防止 agent 提前宣告完成 — 验证节点为什么必须独立于实现节点，在图中这是结构问题而非提示词问题

练习

1. **把 P07 的 maker-checker loop 画成图：** 用 `graph.md` 显式写出节点、边、共享状态和路由规则。标出哪条边是条件边（验证通过/失败）、哪条是回退边（失败回到实现）。画完回答：有没有哪条边是隐式的、原来藏在 agent 的上下文里？

2. **回答 eigent 的四个问题：** 找出三个你在跑的独立 loop（或同一个项目里的三个自动化），回答：它们之间谁喂谁？哪个 loop 拥有另一个 loop 追逐的目标？有没有 loop 能否决另一个 loop 的产出？哪些指标在各自优化、却可能互相冲突？

3. **Goodhart 自检：** 检查你最近优化过的某个指标。它涨了，真实结果（业务结果、用户反馈、代码质量）跟着变好了吗？如果只是数字涨了，这个 loop 正在朝哪个方向骗你？

4. **五个判据评估：** 挑一个你正在纠结要不要"图化"的任务，用五个判据逐条打分。至少满足三个才值得画图。如果不足三个，它需要的其实是一段更好的 workflow 脚本——别为了用图而用图。

5. **把 graph.md 变成可执行程序：** 按照本讲"从零构建你的第一张图"的六步，把你画的那张 maker-checker 图实现成一张能跑起来的图（参考实现：`code/maker_checker_graph.py`，用 LangGraph 写的）。六步别跳：定义状态 → 列节点 → 连边 → 写路由 → 挂 checkpoint → 跑。跑完对比 `graph.md` 和代码，找出第一处对不上的地方，并解释为什么对不上——是图画错了，还是代码写错了？

欢迎来到项目实战

这里是 Learn Harness Engineering 的动手实践部分。仅仅阅读讲义是不够的，你需要亲自搭建环境，观察 Codex 或 Claude Code 等 Agent 在不同规则下的真实表现。

项目概览

本课程包含 7 个渐进式的实战项目，带你从零开始构建一个完整的 Agent 工作环境：

1. **提示词驱动 vs 规则驱动**：对比一个纯 Prompt 驱动的 Agent 和带有基础 Harness 的 Agent 在表现上的差异。
2. **让项目可读并接住上次工作**：学习如何重构代码仓库结构，使其对 AI 更加友好，并建立工作交接机制。
3. **跨会话工作连续性**：设计状态文件和初始化脚本，让 Agent 可以在多次对话中断后无缝恢复工作。
4. **运行反馈与行为修正**：引入运行时反馈机制，让 Agent 能够自己检查代码是否能跑通，并修正错误。
5. **工作评审与自我验证**：建立独立的评审与验证环节，防止 Agent 产生"幻觉"或提前宣告胜利。
6. **综合 Agent 工作环境**：搭建一套完整的、带有可观测性的最终 Agent 工作环境（综合演练）。
7. **第一个自动循环**：从手动驱动跃迁到自动循环——goal loop、timer loop、maker-checker loop 三个递进实验。

如何进行

每个项目文件夹中通常包含：

- `starter/`：你的起始工作区。
- `solution/`：参考的最终实现（如果你卡住了可以参考）。
- 相关的指导文档，说明本次任务的具体背景和目标。

请使用你习惯的 AI Coding Agent（如 Claude Code、Cursor、Trae 等）在每个项目的 `starter/` 目录下完成任务。

Project 01. 只写提示词让 agent 做，和定好规则再让它做，差多少

> 相关讲义：L01. 模型能力强，不等于执行可靠 · L02. Harness 到底是什么 本篇模板文件：templates/

你要做什么

用 Electron 搭一个最简的知识库应用壳子——能启动窗口、左边显示文档列表、右边显示问答面板、本地有一个数据目录。任务本身不复杂，复杂的是你怎么让 agent 完成它。

你需要跑两次。第一次只给一段提示词，什么都不准备，看 agent 能做到什么程度。第二次提前在仓库里放好 `AGENTS.md`、`init.sh`、`feature_list.json`，用结构化的方式告诉 agent 该干什么、怎么验证、什么时候算做完。然后对比两次结果。

课程场景使用一小段准备或重新探索时间作为示例，不依赖固定测量值。

使用仓库里的项目

仓库路径：`projects/project-01/`

目录	里面有什么	怎么用
`starter/`	弱 harness 版本，只有 `task-prompt.md`，没有 `AGENTS.md` 或 `feature_list.json`。	把 prompt 给 coding agent，测量它在没有额外结构时完成了什么。
`solution/`	同一个产品切片，加上 `AGENTS.md`、`CLAUDE.md`、`init.sh`、`feature_list.json`、`claude-progress.md`。	对照规则和验证证据如何把同一任务具体化。

四个具体功能是窗口启动、文档列表、问答面板、本地数据目录。预期证据见 `solution/feature_list.json`。

这里是对照实验，不要求两次 agent 运行都产出可用于生产的完整 Electron 应用。请对弱 harness 的 `starter/` 和显式 harness 的 `solution/` 运行同一个 任务，记录各自完成了哪些功能，以及有哪些验证证据。部分完成或运行失败 本身也是实验结果；功能列表用于定义观测指标，并不表示只用提示词运行就必须 全部通过。

用什么工具

- Claude Code 或 Codex（选一个，两次都用同一个）
- Git（管理分支和对比）
- Node.js + Electron（项目技术栈）
- 一个计时器（记录每次运行时间）

具体步骤

准备工作

1. 从一个干净的 commit 出发，记录 commit hash。
2. 创建两个分支：`p01-baseline` 和 `p01-improved`。
3. 准备同一段任务提示词，内容是："用 Electron 做一个知识库应用，窗口左边是文档列表区域，右边是问答面板区域，应用需要创建并使用本地数据目录。"

第一次运行（弱 harness）

切到 `p01-baseline` 分支。

1. 只用上面那段提示词启动 agent。
2. 不提供 `AGENTS.md`，不提供启动脚本，不提供验收标准。
3. 设定相同的时间上限和轮次上限（建议 30 分钟 / 20 轮）。
4. agent 停下后，运行 `npm start`（或对应启动命令），看应用能不能跑起来。
5. 记录：终端输出、关键 diff、agent 的最终总结。
6. **不要手动修代码。**跑不起来就是跑不起来，如实记录。

第二次运行（强 harness）

切到 `p01-improved` 分支。在启动 agent 之前，先在仓库里准备好：

- `AGENTS.md`：写明项目结构、启动命令、Electron 层边界规则
- `init.sh`：一键恢复可运行状态（`npm install && npm start`）
- `feature_list.json`：列出四个功能点及其完成状态

然后用和第一次相同的提示词启动 agent，同样的时间上限和轮次上限。agent 停下后，跑 `./init.sh`，记录结果。

怎么衡量结果

指标	说明
完成状态	完全完成 / 部分完成 / 失败
首次成功启动时间	从开始到 `npm start` 第一次成功运行
重试次数	中间需要人工干预几次才能跑起来
遗漏项	agent 宣布完成时还有哪些功能没做
过早停止	agent 是否在不可运行状态就宣布完成

要交什么

- 弱 harness 运行记录：提示词、日志/对话记录、最终 diff、启动证据
- 强 harness 运行记录：同上，加上你准备的 harness 文件
- 一份对比笔记（1-2 页）：两次运行的差异、数据、结论

对应讲义

- Lecture 01 — 为什么强 agent 仍然失败
- Lecture 02 — harness 到底是什么
- Lecture 06 — 为什么初始化需要单独一个阶段

Project 02. 让 agent 看懂项目、接住上次的工作

> 相关讲义：L03. 让代码仓库成为唯一的事实来源 · L04. 为什么一个巨大的指令文件会失败 本篇模板文件：templates/

你要做什么

P01 证明了准备规则有用。但 P01 的任务一次会话就干完了。真实开发不是这样的——你昨天干了一半，今天开新会话，agent 得从仓库状态里搞清楚"做了什么、没做什么、接下来干什么"。

这个项目要求你给仓库加上"可读性"：让一个全新的 agent 打开仓库后，能快速理解项目结构、知道当前进度、接手上次的工作。具体任务是给知识库应用加上三个功能：文档导入、文档详情页、导入后的本地持久化。这些功能必须跨至少两个 agent 会话完成。

你需要跑两次。第一次不给 agent 任何帮助，看它在第二个会话里要花多久才能"接上"。第二次提前放好 `ARCHITECTURE.md`、`PRODUCT.md`、`session-handoff.md`，让它快速对齐上下文。

使用仓库里的项目

仓库路径：`projects/project-02/`

目录	里面有什么	比较什么
`starter/`	Project 01 代码，加上未完成的文档导入、详情视图、持久化。文档存在但较薄，没有 `session-handoff.md`。	第二个 agent 会话要重新发现多少上下文。
`solution/`	完成同一产品切片，并在 `projects/project-02/solution/` 下补齐核心文档（以及 `feature_list.json`、`session-handoff.md`）。	新会话能否只靠仓库状态继续工作。

产品功能是文档导入、完整详情/内容加载、跨重启持久化；harness 功能是可交接的 agent-readable workspace。

用什么工具

- Claude Code 或 Codex（和 P01 保持一致）
- Git
- Node.js + Electron
- 文本编辑器（写文档用）

具体步骤

准备工作

1. 基于 P01 完成后的代码，从同一个 commit 出发。
2. 创建两个分支：`p02-baseline` 和 `p02-improved`。
3. 列出要实现的三个功能：文档导入流程、文档详情视图、文档持久化。两条分支任务范围完全一致。

第一次运行（弱 harness）

切到 `p02-baseline` 分支。

会话 A：

1. 启动 agent，只给任务描述，不给架构文档，不给进度文件。
2. 故意在功能完成之前停止会话（比如只完成文档导入）。
3. 不写任何交接文件。直接结束。

会话 B：

1. 开一个全新的 agent 会话。
2. 只说"继续开发"，不给额外上下文。
3. 记录 agent 花了多久才做出第一个有意义的代码修改。
4. 记录 agent 哪些时候在"重新发现"本来已经知道的东西。

第二次运行（强 harness）

切到 `p02-improved` 分支。在第一个会话之前，先在仓库里准备好：

- `ARCHITECTURE.md`：描述项目结构、Electron 各层职责、数据流
- `PRODUCT.md`：描述产品功能范围和当前阶段目标
- `AGENTS.md`：启动命令、工作规则、验证方式
- `init.sh`：一键恢复可运行状态

会话 A：

1. 启动 agent，让它开始工作。
2. 同样在功能完成之前停止。
3. **这次要求 agent 更新** `session-handoff.md`：记录做了什么、没做什么、下一步是什么。

会话 B：

1. 开一个全新的 agent 会话。
2. 让 agent 读 `session-handoff.md` 和 `feature_list.json`，然后继续。
3. 同样记录接手速度和重复工作比例。

怎么衡量结果

指标	说明
会话 B 接手时间	从开始到第一个有效代码修改的时间
重新发现次数	agent 重新了解架构、命令、状态等已有信息的次数
交接文件质量	交接记录是否完整、准确、可操作
重复工作比例	会话 B 中有多少工作量是在重复会话 A 已做过的事
最终完成状态	三个功能是否全部完成

要交什么

- 弱 harness 的会话 A + B 日志/对话记录
- 强 harness 的会话 A + B 日志/对话记录
- 两次运行中产生的交接文件
- 一份对比笔记：重点比较接手速度和上下文恢复质量

对应讲义

- Lecture 03 — 为什么仓库必须成为唯一事实来源
- Lecture 04 — 为什么一个大而全的指令文件不行
- Lecture 05 — 为什么长任务会丢失连续性

Project 03. 让 agent 关掉再打开还能接着干

> 相关讲义：L05. 为什么长时任务会丢失上下文 · L06. 为什么初始化需要单独一个阶段 本篇模板文件：templates/

你要做什么

P02 解决了"接手"的问题，但 agent 接手之后能不能把活干完、干对了，又是另一回事。这个项目要你给 agent 加上范围控制和验证关卡。

你要实现的知识库功能是：文档分块、元数据提取、索引进度显示、带引用的问答流程。这些功能比前两个项目复杂，agent 更容易跑偏——要么多做了不该做的，要么说做完了但其实没通过验证。

你需要一个 `feature_list.json`，每个功能有明确的状态。规则很简单：一次只做一个功能，没有可运行的验证证据就不能标成 passing。仓库里的 starter 已经有早期版功能清单，但缺少完整的重启和交接产物；solution 则补齐这些产物。比较两者，看结果差多少。

使用仓库里的项目

仓库路径：`projects/project-03/`

目录	里面有什么	比较什么
`starter/`	Project 02 代码，索引和带引用问答未完成。有 starter 版 `feature_list.json`，但缺少最终重启/交接产物。	agent 是否跨多个功能漂移，重启后是否丢状态。
`solution/`	完成文档分块、元数据、索引状态、带引用问答，并加入 `init.sh`、`session-handoff.md`、`claude-progress.md`、`clean-state-checklist.md`。	每个功能是否有验证证据后才标为 passing。

本项目对应的产品功能不是泛泛的"多会话"，而是文档分块、元数据提取、索引状态 UI、带引用问答。

用什么工具

- Claude Code 或 Codex
- Git
- Node.js + Electron
- `feature_list.json`（模板参考 `docs/zh/resources/templates/feature_list.json`）

具体步骤

准备工作

1. 基于 P02 完成后的代码，从同一个 commit 出发。
2. 创建两个分支：`p03-baseline` 和 `p03-improved`。
3. 定义四个功能：文档分块、元数据提取、索引进度 UI 显示、带引用的问答。两条分支的功能定义完全一致。

第一次运行（弱 harness）

切到 `p03-baseline` 分支。

1. 启动 agent，给一段模糊的任务提示词。
2. 使用 starter 里已有的早期版 `feature_list.json`，但不提供额外的重启/交接产物。
3. 不额外强化"一次一个功能"和交接更新要求。
4. 验证标准较弱——重点观察 agent 是否自己补足证据。
5. 运行结束后，手动检查每个功能是否真的能用。
6. 记录哪些功能 agent 声称完成但实际没通过验证。

第二次运行（强 harness）

切到 `p03-improved` 分支。在启动 agent 之前：

- 对照 solution 中的 `AGENTS.md`、`feature_list.json`、`init.sh`、`session-handoff.md`、`claude-progress.md`。
- 明确规则：一次只做一个功能；状态只能在有验证证据后切到 `pass`。
- 每次会话结束前更新交接和进度产物。

然后启动 agent：

1. agent 开始工作，每完成一个功能必须更新 `feature_list.json` 并附上验证证据（截图、测试输出等）。
2. 至少要有一个功能展示从 `failing` 到 `passing` 的完整转换过程。
3. 问答功能的验证必须检查引用是否存在、引用是否相关，不能只看有没有输出。
4. 运行结束后归档所有验证证据。

怎么衡量结果

指标	说明
范围漂移次数	agent 做了功能清单之外的事的次数
虚假完成率	agent 声称完成但验证不通过的功能比例
验证覆盖率	有明确验证证据的功能占总功能的百分比
问答质量	引用是否存在、引用是否相关
重试次数	从开始到所有功能 passing 总共重试了几次

要交什么

- 弱 harness 运行记录：提示词、日志、验证结果
- 强 harness 运行记录：`feature_list.json` 变更历史、日志、验证证据
- 至少一个功能从 `failing` 到 `passing` 的转换证据
- 一份对比笔记：重点看范围纪律和完成准确度

对应讲义

- Lecture 07 — 为什么 agent 总是做过头、做不完
- Lecture 08 — 为什么 feature list 是 harness 的基础原语
- Lecture 09 — 为什么 agent 总是过早宣布胜利
- Lecture 10 — 为什么端到端测试能改变结果

Project 04. 用运行反馈修正 agent 的行为

> 相关讲义：L07. 为什么 agent 会多做或少做 · L08. 为什么功能清单是 harness 原语 本篇模板文件：templates/

你要做什么

前三个项目关注的是"让 agent 把活干完"。这个项目关注的是"出了问题怎么修"——而且不是你修，是让 agent 自己通过运行时信号来修。

你要做两件事。第一，给知识库应用加上运行时可观测性：启动日志、导入和索引的日志、问答失败时的用户可见错误状态。第二，在仓库里编码架构约束，让 agent 不可能静默地跨层违规（main / preload / renderer / services 之间的边界）。

仓库里的 starter 已经埋好了运行时 bug，但诊断信号很弱，也没有架构检查脚本。solution 加上结构化日志、架构规则和修复后的分块逻辑。比较两者，看运行时反馈对修复质量的影响。

使用仓库里的项目

仓库路径：`projects/project-04/`

目录	里面有什么	比较什么
`starter/`	Project 03 代码，诊断信号较弱。植入的 indexing 缺陷会让大文件分块异常，并且没有架构检查脚本。	没有运行时信号时，agent 多久能找到根因。
`solution/`	结构化 logger、架构边界文档和脚本、修复后的分块逻辑、`clean-state-checklist.md`。	日志和边界检查是否让修复更快、更少破坏。

重点检查 `projects/project-04/solution/src/services/logger.ts`、`projects/project-04/solution/scripts/check-architecture.sh`、`projects/project-04/solution/docs/ARCHITECTURE.md`、`projects/project-04/solution/src/services/indexing-service.ts`。

用什么工具

- Claude Code 或 Codex
- Git
- Node.js + Electron
- 日志库（如 `electron-log` 或简单的 console 封装）
- 结构化检查工具（ESLint 自定义规则、guard 脚本、或测试）

具体步骤

准备工作

1. 基于 P03 完成后的代码，从同一个 commit 出发。
2. 创建两个分支：`p04-baseline` 和 `p04-improved`。
3. 在两个分支中埋入同一个运行时 bug（比如：索引时 chunk 大小读取错误导致问答返回空结果）。
4. 记录 bug 的位置和表现，但不要告诉 agent bug 在哪。

第一次运行（弱 harness）

切到 `p04-baseline` 分支。

1. 启动 agent，告诉它"问答功能返回空结果，请修复"。
2. 仓库里没有结构化 logger，也没有架构检查脚本。
3. 记录 agent 花了多久找到根因、怎么确认修复的、修复过程中是否引入了新的层边界违规。
4. 修复后重启应用，确认是否能干净启动。

第二次运行（强 harness）

切到 `p04-improved` 分支。在启动 agent 之前，先在仓库里准备好：

- **运行时日志**：对照 solution 的 `src/services/logger.ts`，启动、导入、索引、问答失败路径都有结构化日志。
- **架构约束**：对照 solution 的 `AGENTS.md`、`projects/project-04/solution/docs/ARCHITECTURE.md` 和 `projects/project-04/solution/scripts/check-architecture.sh`，明确 Electron 四层边界（main / preload / renderer / services）。
- **干净状态要求**：最终交付前必须能干净重启。

然后启动 agent：

1. 先让 agent 加上日志和结构化检查（这是任务的一部分）。
2. 然后告诉它"问答功能返回空结果，请修复"。
3. 这次 agent 有日志可以看，有架构规则可以参照。
4. 记录诊断速度、修复质量、是否引入了边界违规。
5. 修复后重启应用，确认干净启动。

怎么衡量结果

指标	说明
根因定位时间	从开始到 agent 准确描述 bug 原因
修复确认时间	从定位到验证修复成功
边界违规数	修复过程中引入的跨层违规次数
日志有用性	日志是否直接指向了失败原因
干净重启	修复后是否能干净重启、无报错

要交什么

- 弱 harness 的调试记录：诊断过程、修复 diff、启动证据
- 强 harness 的调试记录：同上，加上日志和架构约束文件

- 结构化检查产物（lint 规则、guard 脚本、或测试文件）
- 一份对比笔记：重点比较诊断速度和修复健壮性

对应讲义

- Lecture 11 — 为什么可观测性属于 harness 的一部分
- Lecture 12 — 为什么每个会话都必须留下干净状态

Project 05. 让 agent 自己检查自己做的对不对

> 相关讲义：L09. 为什么 agent 会提前宣告完成 · L10. 为什么端到端测试会改变结果 本篇模板文件：templates/

你要做什么

前四个项目里，"检查做得对不对"这件事要么是你手动做的，要么是靠文件规则强制执行的。这个项目要让 agent 自己来检查。

方法是角色分离。以前是同一个 agent 既写代码又判断质量。现在拆开：一个 agent 负责实现（生成者），另一个 agent 负责审查（评估者）。更进一步，还可以加一个负责规划的角色（规划者）。你要跑三次，看每加一层角色分离，结果好多少。

选一个实质性的功能升级（比如多轮对话历史、引用面板重设计、文档集合与筛选），三次都做同一个升级，唯一变量是 harness 的角色分工。

使用仓库里的项目

仓库路径：`projects/project-05/`

目录	里面有什么	比较什么
`starter/`	ConversationHistory 升级前的 Project 04 应用。	如果要自己重跑三种变体，从这里开始。
`solution/single-role/`	一个 agent 规划、实现、自评。	`evaluator-rubric.md` 评分 1.6/5 和缺陷列表。
`solution/gen-eval/`	生成者 + 评估者，有修订证据。	`evaluator-rubric.md` 评分 3.3/5 和修订记录。
`solution/plan-gen-eval/`	规划者 + 生成者 + 评估者。	`sprint-contract.md`、`evaluator-rubric.md` 评分 4.9/5。

已提交的固定功能是多轮问答历史。重跑时保持功能不变，唯一变量是角色分工。

用什么工具

- Claude Code 或 Codex（用于生成和规划角色）
- 同一个或另一个 agent 实例（用于评估角色）
- 评估量表（参考 `docs/zh/resources/templates/evaluator-rubric.md`）
- Sprint contract 模板（参考 `docs/en/lectures/lecture-11-why-observability-belongs-inside-the-harness/code/sprint-contract.md`）

具体步骤

准备工作

1. 基于 P04 完成后的代码，从同一个 commit 出发。
2. 创建三个分支：`p05-single`、`p05-gen-eval`、`p05-plan-gen-eval`。
3. 选定一个功能升级，写清楚验收标准。三次运行的功能范围和验收标准完全一致。
4. 写好评估量表，打分维度至少包括：正确性、可靠性、可维护性、用户体验。

第一次运行（弱 harness — 单角色）

切到 `p05-single` 分支。

1. 一个 agent 包揽规划、实现、自查。
2. 没有外部评估量表，agent 自己判断做得好不好。
3. 记录最终产出和未解决的缺陷。

第二次运行（强 harness — 生成者 + 评估者）

切到 `p05-gen-eval` 分支。

1. 生成者和评估者先用 sprint contract 对齐：做什么、怎么验证、什么不算在范围内。

2. 生成者实现功能。
3. 评估者用评估量表打分，反馈问题。
4. 生成者根据反馈修改。至少经过一轮修订。
5. 记录评估者抓住了多少缺陷、修订了多少内容。

第三次运行（更强 harness — 规划者 + 生成者 + 评估者）

切到 `p05-plan-gen-eval` 分支。

1. 规划者先拆解任务、定义步骤和依赖关系。
2. 生成者按规划实现。
3. 评估者用同一份评估量表打分。
4. 至少经过一轮修订。
5. 记录规划是否减少了返工。

评估者调优

评估者不是一次就能用好的。初始版本的评估者往往会"发现问题然后自己说服自己通过"。你需要：

1. 让评估者给一个已完成的 sprint 打分。
2. 和你自己的判断对比。
3. 哪里有分歧，就把量表写得更具体。
4. 重新跑评估者，检查对齐程度。
5. 重复 3-5 轮，直到评估者判断和你的判断基本一致。记录每一轮调优的内容。

怎么衡量结果

指标	说明
范围定义质量	编码前的任务拆解清晰度
缺陷检出数	交付前被评估者抓到的缺陷数量和严重程度
量表评分	各维度的得分（正确性、可靠性、可维护性、UX）
返工量	评估反馈后需要重做的比例
最终健壮性	升级后功能在实际运行中的稳定程度
评估者调优轮数	评估者判断与你对齐需要的迭代次数
Sprint contract 效果	合同是否减少了评估中的模糊地带

要交什么

- Sprint contract 文档
- 评估者调优日志（每轮改了什么、为什么改）
- 单角色运行记录：提示词、日志、可运行证据
- 生成者+评估者运行记录：含量表评分和修订记录
- 规划者+生成者+评估者运行记录：含规划文档、量表评分、修订记录
- 一份对比笔记：质量提升幅度和协调成本

对应讲义

- Lecture 09 — 为什么 agent 总是过早宣布胜利
- Lecture 10 — 为什么端到端测试能改变结果
- Lecture 11 — 为什么可观测性属于 harness 的一部分

Project 06. 搭建一套完整的 agent 工作环境

> 相关讲义：L11. 为什么可观测性属于 harness · L12. 为什么每次会话都要留干净状态 本篇模板文件：templates/

你要做什么

这是结业项目。把前五个项目学到的所有东西组装起来，跑一次完整的基准测试，然后做一轮清理，验证质量是可以持续维护的。

你要用一套固定的多功能任务集，覆盖知识库应用的完整产品切片：导入文档、构建索引、带引用的问答、运行时可观测性、可读可重启的仓库状态。先跑一次弱 harness 基线，再跑一次你组装的最强 harness，然后做一轮清理和重跑。最后还要做一次 harness 精简实验——删掉一个组件看看结果会不会变差，判断哪些组件是真正有用的、哪些是多余的开销。

使用仓库里的项目

仓库路径：`projects/project-06/`

目录	里面有什么	比较什么
`starter/`	大部分产品代码已经完成，但 harness 表面较弱：只有基础 `AGENTS.md`，没有 `feature_list.json`、`session-handoff.md`、clean-state checklist。	手动记录弱 harness 基线。starter 故意不包含 benchmark 脚本。
`solution/`	完整 harness：`AGENTS.md`、`CLAUDE.md`、`feature_list.json`、`init.sh`、`session-handoff.md`、`clean-state-checklist.md`、质量/评估文档、benchmark 和 cleanup 脚本。	运行 `projects/project-06/solution/scripts/benchmark.sh`、`projects/project-06/solution/scripts/cleanup-scanner.sh`，对比质量文档证据。

和前几个项目不同，capstone starter 不是缺很多产品功能；主要缺口是应用周围的操作 harness。

用什么工具

- Claude Code 或 Codex
- Git
- Node.js + Electron
- 质量文档模板（`docs/zh/resources/templates/quality-document.md`）
- 评估量表（`docs/zh/resources/templates/evaluator-rubric.md`）
- 前五个项目积累的所有 harness 组件

具体步骤

准备工作

1. 基于 P05 完成后的代码，从同一个 commit 出发。
2. 创建两个分支：`p06-baseline` 和 `p06-improved`。
3. 用质量文档模板给当前代码打一次初始评分（每个产品领域和架构层的等级）。
4. 定义一套固定的基准任务集和评分表——在跑任何 agent 之前就定好，跑的过程中不改。

基准任务集至少包括：

- 导入一篇文档
- 构建或刷新索引
- 回答一个带引用的问题
- 查看运行时日志确认可观测性
- 关掉重开后状态仍在

第一次运行（弱 harness）

切到 `p06-baseline` 分支。

1. 用课程早期阶段的弱 harness（没有完整交接文件、没有严格验证、可观测性不足）。
2. 用 agent 跑完整个基准任务集。
3. 立刻评分。记录每个任务的完成状态、重试次数、缺陷数。
4. 更新质量文档，记录每个领域和层的等级变化。

第二次运行（强 harness）

切到 `p06-improved` 分支。

1. 用你在这门课里组装的最强 harness：交接文件和启动脚本、明确的范围和验证关卡、运行时信号和架构约束、评估者或多角色审查、质量文档追踪。
2. 同样的基准任务集，同样的模型和预算。
3. 立刻评分。记录结果。
4. 更新质量文档。

清理和重跑

在 `p06-improved` 分支上：

1. 做一轮清理：删死代码、修不清楚的文档、理顺不稳定的运行路径。
2. 清理后重跑同样的基准任务集，重新评分。
3. 更新质量文档。

对比三个快照的质量文档：基线、强 harness、清理后。

Harness 精简实验

1. 从 `p06-improved` 分支中删掉一个 harness 组件（比如删掉 sprint contract，或者删掉显式范围关卡）。
2. 重跑基准任务集。
3. 如果结果没变差——说明这个组件是多余的开销，可以去掉。
4. 如果结果变差了——说明这个组件是承重的，必须保留。
5. 记录实验结果。可以多试几个组件。

怎么衡量结果

指标	说明
基准完成率	基准任务集中成功完成的比例
重试次数	每个任务需要重试几次
缺陷数	人工干预前发现的缺陷数量
清理工作量	清理花了多长时间、改了多少文件
清理后可读性和重启成功率	清理后仓库的可维护程度
质量文档等级变化	三个快照的等级对比
Harness 精简结果	哪些组件可以删、哪些是承重的

要交什么

- 质量文档的三个快照（基线、强 harness、清理后）
- 基线基准测试记录：评分和证据
- 强 harness 基准测试记录：评分和证据
- 清理运行记录：清理前后评分变化
- Harness 精简日志：删了什么组件、基准结果、决定保留还是删
- 最终结业总结：关键经验教训

对应讲义

- Lecture 01 — 为什么强 agent 仍然失败
- Lecture 03 — 为什么仓库必须成为唯一事实来源
- Lecture 08 — 为什么 feature list 是 harness 的基础原语
- Lecture 11 — 为什么可观测性属于 harness 的一部分
- Lecture 12 — 为什么每个会话都必须留下干净状态

Project 07. 搭建你的第一个自动循环

> 相关讲义：L13. 从手动驱动到自动循环

你要做什么

这是从 "Harness" 到 "Loop" 的跃迁项目。你已经知道怎么给 agent 配好一套环境、一套指令、一套反馈——现在你要把这套东西变成一个能自己跑的循环。

你会做三个递进的实验：先把一个任务从手动跑变成 `/goal` 自动跑，再把一个巡检任务变成 `/loop` 定时跑，最后做一个带 maker-checker 分离的完整循环，体验「人退到循环外面」是什么感觉。

使用仓库里的项目

仓库路径：`projects/project-07/`

目录	里面有什么	做什么
`starter/`	一个带有完整 harness 的小型知识库项目（P06 完成态），包含 AGENTS.md、feature_list.json、init.sh、session-handoff.md、clean-state-checklist.md。	把这套 harness 改造成能自动循环的版本。
`solution/`	三个循环的完整实现：goal loop、loop timer loop、maker-checker loop，加上 loop 状态文件和验证脚本。	参考 loop 的设计方式和状态管理模式。

用什么工具

- Claude Code 或 Codex
- Git

- 你在 P06 搭建好的完整 harness
- 一个你常用的终端复用工具（tmux 或 screen，用于观察长时间运行的 loop）
- 可选：GitHub Actions 或 cron（用于事件驱动/定时驱动的进阶实验）

具体步骤

准备工作

1. 基于 P06 完成后的代码，从同一个 commit 出发。
2. 创建三个分支：`p07-goal-loop`、`p07-timer-loop`、`p07-maker-checker`。
3. 确认你的 harness 是工作的：跑一次 init.sh，确认状态文件、feature list、交接文档都在。
4. 选一个你要让 loop 反复做的**目标任务**。建议选一个中等大小、有明确完成标准的任务，比如："给所有模块补充单元测试，覆盖率达到 80%" 或 "把所有 API 端点加上输入验证"。

实验一：Goal Loop —— 从手动跑到自动跑

切到 `p07-goal-loop` 分支。

1. **写目标描述**：把你选的任务写成一份 `goal.md`，包含：
   - 明确的目标（"完成什么算做完"）
   - 验证方式（"怎么确认做完了"——跑测试？跑 lint？检查覆盖率？）
   - 停止条件（"什么时候应该停下"——最大回合数？时间上限？预算上限？）
   - 约束（"不能碰什么"——生产配置、数据库 schema 等）
2. **第一次手动跑**：你自己手动给 agent 发指令，完成一次这个任务。记录用了多少回合、你介入了多少次、结果质量如何。这是你的基线。
3. **用 `/goal` 跑**：用同一份 `goal.md` 作为输入，用 `/goal` 模式跑一次。agent 自己循环直到达成目标或触发停止条件。
4. **对比结果**：
   - 回合数差异

- 你介入的次数差异
- 结果质量差异（用同样的验证标准）
- 你花的时间差异

5. **迭代 goal.md**：如果结果不好，改 goal 描述，再跑一次。直到你对结果满意，或者确认 goal loop 在这个任务上的极限在哪里。

实验二：Timer Loop —— 把巡检变成心跳

切到 `p07-timer-loop` 分支。

1. **选一个巡检任务**：找一个你平时手动做的、重复性的检查工作。比如：
   - 每小时跑一次测试，有失败就修
   - 每天早上检查依赖有没有安全更新
   - 每次提交后检查代码有没有违反编码规范
   - 定期扫描 TODO 注释，看哪些已经过期了
2. **写巡检脚本/prompt**：把巡检的步骤写清楚——检查什么、发现问题了怎么办、什么时候需要叫人。
3. **用 `/loop`（或 Codex 的对话线程自动化）跑**：
   - 设置合理的间隔（建议 10-30 分钟，太短你会被打扰，太长看不出效果）
   - 让它跑至少 2 小时（或者你去干别的事，过一会儿回来看）
4. **记录结果**：
   - 它发现了多少问题？
   - 它自己修了多少？
   - 有多少是误报？
   - 有多少是它修坏了的？
   - 你花了多少时间跟进它的结果？
5. **思考**：这个巡检任务值得自动化吗？节省的时间和你跟进它花的时间比，哪个多？如果不划算，是任务选错了还是 loop 设计得不好？

实验三：Maker-Checker Loop —— 把你从循环里拿出来

切到 `p07-maker-checker` 分支。

这是三个实验里最重要的一个。你要做一个**完整的、你不需要在场的循环**：

1. **设计循环结构**：
   - **Maker agent**：负责实现，写代码，改文件
   - **Checker agent**：负责验证，跑测试，做代码审查，打通过/不通过
   - **状态文件**（`loop-state.md`）：记录当前轮次、做了什么、验证结果、下一轮要做什么
   - **停止条件**：连续 N 轮通过，或者达到最大轮次
2. **写三份 prompt**：
   - Maker 的指令（做什么、怎么做、什么不能碰）
   - Checker 的指令（验证什么、怎么验证、什么算通过、发现问题了怎么反馈）
   - 循环控制逻辑（谁先跑、跑完了怎么交接、下一轮怎么启动）
3. **跑至少 5 轮循环**：
   - 第一轮：Maker 实现 → Checker 验证 → 不通过 → 反馈给 Maker
   - 第二轮：Maker 根据反馈修改 → Checker 再验证 → ...
   - ...
   - 直到连续通过，或者你叫停
4. **记录每一轮的状态**：
   - 轮次
   - Maker 做了什么
   - Checker 发现了什么问题
   - 通过/不通过
   - 你有没有介入（如果介入了，为什么）
5. **最后复盘**：
   - 你一共介入了几次？为什么介入？

- 如果没有介入，结果会怎么样？
- Checker 有没有漏过问题？
- Maker 有没有在同一个问题上反复犯错？
- 这个循环的质量天花板在哪里？是 Maker 的能力还是 Checker 的能力？

怎么衡量结果

指标	实验一（Goal）	实验二（Timer）	实验三（Maker-Checker）
任务完成率	目标达成了吗？	巡检了多少次？	多少轮后通过？
人类介入次数	你插手了几次？	你跟进花了多久？	你介入了几次？
结果质量	和手动比怎么样？	误报率多少？漏检率多少？	Checker 发现了多少你没发现的问题？
时间节省	你省了多少时间？	值得自动化吗？	你花在设计循环上的时间 vs 你省下来的时间
可靠性	停止条件靠谱吗？	有没有跑飞了？	循环会不会卡死在同一个地方？

要交什么

- `goal.md`（实验一的目标描述，至少迭代了两个版本）
- 实验一的对比记录：手动 vs goal loop
- 实验二的巡检 prompt 和 2 小时运行日志
- 实验三的三份 prompt（Maker / Checker / 循环控制）
- 实验三的 `loop-state.md`（至少 5 轮记录）
- 最终复盘：三个实验的收获、你对 loop engineering 的理解变化、哪些事情适合 loop 化、哪些不适合

对应讲义

- Lecture 13 — 从手动驱动到自动循环
- Lecture 12 — 为什么每次会话都必须留下干净状态（loop 的每一轮都需要 clean state）
- Lecture 11 — 为什么可观测性属于 harness 的一部分（你需要看到 loop 内部在发生什么）
- Lecture 05 — 为什么状态文件是跨会话连续性的核心（loop 状态文件是 state file 的延伸）

Project 08. 把你的工作流画成一张图

> 相关讲义：L14. 从单循环到图工程

你要做什么

这是从 "Loop" 到 "Graph" 的跃迁项目。上一讲你搭了一个 maker-checker loop——实现、验证、反馈、再实现，所有决策都发生在同一个 agent 的上下文窗口里。这一讲你要做的，是**把藏在循环里的结构显式地画出来**：节点、边、共享状态、路由规则，一个字一个字地写清楚。

你会做三个递进的实验：先把 P07 的 maker-checker loop 画成一张显式图，再给图加一个并行 fan-out/fan-in 节点，最后加一条条件回退边和一个人工审批节点。做完你会亲身感受到一件事：**图不是新发明，是当你的 loop 复杂到一定程度后，它自己变成的样子。**

用什么工具

- Claude Code 或 Codex
- Git
- 你在 P07 搭好的 maker-checker loop（或任何一个你能反复跑的 agent 工作流）
- 一个文本编辑器或绘图工具（画图不是为了好看，是为了把结构写清楚；`mermaid` 或手写 `graph.md` 都行）

具体步骤

准备工作

1. 从 P07 完成后的仓库出发，或者直接用你正在跑的任何 agent 工作流。
2. 创建三个分支：`p08-explicit-graph`、`p08-parallel`、`p08-human-in-the-loop`。

3. 准备一个 `state.md` 作为共享状态文件：需求、进度、验证结果都写在这里。这是图的"公共工作台"。

实验一：把 Loop 画成显式图

切到 `p08-explicit-graph` 分支。

1. **列出所有节点**：把 P07 maker-checker loop 里的每一步写成一个节点。每个节点写清楚：它的职责、它的输入、它的输出、它是 agent 还是确定性代码。
2. **画出所有边**：列出节点之间的每一条边。重点标注两条特殊边：
   - 条件边：验证通过/失败，走哪条
   - 回退边：失败回到哪个节点
3. **写共享状态**：明确列出状态里有哪些字段（需求、代码、测试结果、审查结论），谁读谁写。
4. **写路由规则**：用最简单的 if-then 语言写下"下一步去哪"的规则，比如：

```
if 验证通过 → 合并节点
if 验证失败 → 实现节点
if 实现节点信息不足 → 研究节点
```

5. **写成 `graph.md`**：把以上内容整理成一份文档。用 mermaid 画一张图，附上节点表和路由规则。
6. **回答这个问题**：画完之后，找出至少一条**原来是隐式的边**——以前藏在 agent 上下文里、你自己都不知道它存在的决策路径。

实验二：加一个并行 Fan-out / Fan-in 节点

切到 `p08-parallel` 分支。

1. **选一个可以并行的点**：找任务里一个可以拆成两个独立部分的地方。比如：
   - 实现拆成两个独立模块，两个 agent 并行写
   - 验证拆成两个独立审查：一个跑测试和 lint，一个做代码审查（不同的指令、不同的关注点）
   - 研究拆成两个方向，两个 agent 各查一路
2. **写 fan-out 规则**：共享状态里记录"这个任务被拆成 N 个并行子任务"，每个子任务一个独立的 context、一个独立的节点。

3. **写 fan-in 规则**：所有子任务完成后，谁来合并结果？合并的标准是什么（比如：两个审查都通过才合并，还是有一个通过就行）？
4. **用 worktree 隔离**：每个并行子任务在独立的 git worktree 里跑，物理上避免文件碰撞（回顾第十三讲的 Worktree 原语）。
5. **跑一次并记录**：记录并行前后 wall-clock 时间、token 消耗、结果质量。并行真的更快吗？还是协调开销吃掉了省下的时间？

实验三：加一条回退边和一个人工审批节点

切到 `p08-human-in-the-loop` 分支。

这是三个实验里最重要的一个。你要在图上加两种节点：

1. **条件回退边**：给验证节点加一条"部分通过"的路径——不是全盘打回实现节点，而是带着具体反馈回到**产生问题的那个节点**。比如：测试全过但代码审查发现需求理解有误，回退到研究节点而不是实现节点。这要求你的共享状态里记录"问题出在哪一层"。
2. **人工审批节点（Human-in-the-loop）**：在合并节点之前加一个人工节点。走到这里，图**停下来**，等你在 `state.md` 里写"批准"或"打回"。审批节点可以有一个超时规则：N 小时后没响应，自动打回或自动升级。
3. **写 interrupt 的格式**：审批请求怎么写清楚——发生了什么、改了什么、为什么需要人、批准/打回的后果各是什么。
4. **跑至少 2 轮完整流程**：每一轮都走到人工审批节点，你自己批准或打回一次。记录：你的审批决策和验证节点的判断一致吗？审批节点拦住过什么验证节点没拦住的吗？

怎么衡量结果

指标	实验一（显式图）	实验二（并行）	实验三（人机协同）
结构可见性	找出了几条隐式边?	共享状态能否支撑并行子任务?	回退边能否精确定位问题层?
失败定位	失败时能否直接指出是哪条边错了?	并行子任务失败时，能定位到哪一个?	审批打回时，能指出是哪一层的问题吗?
协作开销	写图花了多久?	并行省下的时间 vs 协调开销	审批等待时间 vs 拦住的问题价值
可观测性	每一步发生了什么，现在看得见了吗?	每个并行子任务的状态可见吗?	审批请求写得够清楚吗?
可靠性	图描述和实际运行一致吗?	fan-in 合并标准靠谱吗?	超时/升级规则真的会触发吗?

要交什么

- `graph.md`（实验一的完整图描述：mermaid 图 + 节点表 + 边表 + 共享状态字段 + 路由规则）
- 实验一发现的隐式边清单（至少一条）
- 实验二的 fan-out/fan-in 规则和一次并行运行记录（时间/成本/质量对比）
- 实验三的回退边规则、审批节点格式和 2 轮人机协同记录
- 最终复盘：从 loop 到 graph，你的工作方式发生了什么变化？哪些任务值得画图，哪些不值得?

对应讲义

- Lecture 14 — 从单循环到图工程
- Lecture 13 — 从手动驱动到自动循环（你的 loop 就是图里的一个节点；这个项目是把节点的内部结构摊开）

- Lecture 09 — 为什么 agent 会提前宣告完成（验证节点为什么必须独立于实现节点，在图中是结构问题）
- Lecture 11 — 为什么可观测性属于 harness 的一部分（图越复杂，越需要看到每个节点在做什么）

中文资料库

这个文件夹把课程里的方法整理成可以直接参考和复用的材料，不是再讲一遍概念，而是尽量回答两个问题：

- 我现在应该先抄哪些文件
- 这些文件分别解决什么问题

适合什么时候用

当你准备让 Codex、Claude Code 或其他 coding agent 在一个仓库里持续工作时，就可以从这里开始。它特别适合这些场景：

- 多轮会话开发，担心上下文断裂
- 功能多，容易出现做一半就停的半成品
- 常常提前宣布完成，但实际没测透
- 每次开工都要重新摸索启动方式

从这里开始

如果你想先搭一个最小可用版本，优先看这些文件：

- 根指令文件：`templates/AGENTS.md` 或 `templates/CLAUDE.md`
- 功能状态文件：`templates/feature_list.json`
- 进度日志：`templates/claude-progress.md`

这个文件名是课程早期留下的约定，实际是通用的、放在仓库里的会话进度日志， 并不绑定 Claude Code。Codex、OpenHands、Antigravity 等 agent 都可以使用， 但必须在 `AGENTS.md` 或其他等价指令里明确要求它在开工时读取、收尾时更新； agent 不会自动维护这个文件。

- 启动脚本参考：`docs/resources/templates/init.sh`

然后按需要补上：

- 会话交接模板：`templates/session-handoff.md`
- 收尾检查清单：`templates/clean-state-checklist.md`
- 评审模板：`templates/evaluator-rubric.md`

如果你想直接采用 OpenAI 那篇 harness engineering 文章里更完整的仓库组织方式，可以继续看：

- `openai-advanced/index.md`

资料库结构

- `templates/`：可以直接复制到真实仓库里的模板
- `reference/`：方法说明、启动流程和问题对照表
- `openai-advanced/`：更完整的 OpenAI 风格高级资源包，包括仓库骨架、质量文档、执行计划和系统级治理文件

推荐最小组合

- `AGENTS.md` 或 `CLAUDE.md`
- `feature_list.json`
- `claude-progress.md`（通用会话进度日志，文件名沿用历史约定）
- `init.sh`

先把这四样放进项目里，再开始让 agent 持续工作，通常就已经能明显降低返工和瞎猜。

如果你的仓库已经进入多模块、多阶段、多角色协作，可以直接升级到 `openai-advanced/` 这一套高级结构，而不是继续把最小模板硬撑成一个大而乱的系统。

模板使用指南

这些模板可以直接复制到你的项目里用。每个文件在 agent 的工作流程中各有分工。复制过去后，把里面的命令、路径、功能名称和验证步骤换成你自己项目的。

怎么开始

先把这四个文件复制到项目根目录：

1. `AGENTS.md` 或 `CLAUDE.md`
2. `init.sh`
3. `claude-progress.md`
4. `feature_list.json`

等项目变复杂了，再补其余的文件。

AGENTS.md

根指令文件。agent 每次开工会先读这个文件。它定义了工作规则：写代码前要做什么、工作过程中怎么守规矩、收尾时要检查什么。

怎么用：

- 复制到项目根目录
- 把开工流程里的步骤换成你自己项目的路径和命令
- 工作规则按你们团队的约定调整
- 完成定义那一段别改——那是整个 harness 最关键的部分

它帮 agent 做什么：

- 让它在开始工作前先读进度和功能状态
- 逼它一次只做一个功能

- 要求它拿出证据才能标记完成
- 定义了什么叫"干净收尾"

用 `AGENTS.md` 给 Codex 或其他 agent。用 `CLAUDE.md` 给 Claude Code——内容一样，格式按 Claude 的指令风格来的。

---

init.sh

启动脚本。一条命令完成依赖安装、验证和打印启动命令。

怎么用：

- 复制到项目根目录
- 改顶部这三个变量：
  - `INSTALL_CMD` — 你的依赖安装命令（比如 `npm install`、`pip install -r requirements.txt`）
  - `VERIFY_CMD` — 你的基础验证命令（比如 `npm test`、`pytest`）
  - `START_CMD` — 你的开发服务器启动命令（比如 `npm run dev`）
- 加执行权限：`chmod +x init.sh`

它做什么：

1. 打印当前目录（确认跑在正确的地方）
2. 安装依赖
3. 跑验证命令
4. 打印启动命令（如果设了 `RUN_START_COMMAND=1` 就直接启动）

如果验证失败了，agent 应该停下来先修基础状态，不要在坏的基础上继续叠新功能。

---

claude-progress.md

进度日志。每轮会话往里写，每轮新会话先读它。

怎么用：

- 复制到项目根目录
- 把"当前已验证状态"那段填上你的项目信息
- 每轮会话结束后更新会话记录

每个字段的意思：

- **当前已验证状态** — 项目当前进展的唯一真相
  - `仓库根目录` — 项目在哪
  - `标准启动路径` — 把项目跑起来的命令
  - `标准验证路径` — 跑测试的命令
  - `当前最高优先级未完成功能` — 下一轮该做什么
  - `当前 blocker` — 哪里卡住了
- **会话记录** — 每轮一条
  - `本轮目标` — 打算做什么
  - `已完成` — 实际做了什么
  - `运行过的验证` — 跑了什么测试
  - `已记录证据` — 留下了什么证明
  - `提交记录` — 提了什么 commit
  - `已知风险或未解决问题` — 哪里可能有问题
  - `下一步最佳动作` — 下一轮从哪开始

---

feature_list.json

功能清单。机器可读的功能列表，每个功能有状态、验证步骤和证据。

怎么用：

- 复制到项目根目录
- 把示例功能换成你自己的
- 每个功能需要填：

- `id` — 短的唯一标识
- `priority` — 整数，越小越优先
- `area` — 属于应用的哪块（比如 "chat"、"import"、"search"）
- `title` — 简短描述
- `user_visible_behavior` — 功能正常时用户能看到什么
- `status` — 四种状态之一：`not_started`、`in_progress`、`blocked`、`passing`
- `verification` — 逐步验证步骤
- `evidence` — 验证通过的记录（agent 填）
- `notes` — 额外说明

状态规则：

- `not_started` — 还没碰
- `in_progress` — 当前正在做的那个（同一时间只能有一个）
- `blocked` — 有记录的阻塞问题，推不动
- `passing` — 验证通过，证据已记录

agent 任何时候只能有一个功能处于 `in_progress`。

---

session-handoff.md

会话交接摘要。一轮会话结束时写，下一轮开始时读。让接手的人（或 agent）快速了解现状。

怎么用：

- 复制到项目根目录
- 每轮会话结束时填写（也可以让 agent 自己填）

每段写什么：

- **当前已验证** — 哪些是确认能用的，跑过什么验证
- **本轮改动** — 改了什么代码或基础设施
- **仍损坏或未验证** — 已知问题和风险区

- **下一步最佳动作** — 下一轮该做什么，哪些东西不要动
- **命令** — 启动、验证、调试命令，方便快速参考

短会话可以不写这个文件。会话长了或者项目有多个并行区域时，它就很关键了。

---

clean-state-checklist.md

收尾检查清单。每次会话结束前过一遍，确保仓库处于下一轮可以直接开工的状态。

怎么用：

- 复制到项目根目录
- 关掉会话前逐项检查
- agent 的收尾流程里也应该包含这些检查

检查什么：

- 标准启动路径还能用
- 标准验证还能跑
- 进度日志已更新
- 功能清单真实反映了 passing 和未验证的边界（没有假 passing）
- 没有半成品处于未记录状态
- 下一轮会话不需要人工修复就能继续

---

evaluator-rubric.md

评审评分表。会话结束后或到里程碑时，用它评估 agent 做的东西够不够格。

怎么用：

- 复制到项目根目录
- 一轮（或几轮）会话后，按六个维度打分
- 每个维度 0-2 分

六个维度：

1. **正确性** — 实现出来的行为是否符合目标功能
2. **验证** — 要求的检查是否真的跑过，并留下证据
3. **范围纪律** — 是否基本保持在选定功能范围内
4. **可靠性** — 结果是否能在重启或重跑后继续工作
5. **可维护性** — 代码和文档是否清楚到足以交给下一轮会话
6. **交接准备度** — 新会话是否能只靠仓库内文件继续推进

结论选项：

- Accept — 达标
- Revise — 需要修补才能接受
- Block — 有根本性问题，需要先解决

Evaluator 需要校准。 开箱即用的 agent 做评审很弱——它会发现问题，然后把自己说服到通过。你需要反复校准：

1. 用 evaluator 给一个已完成的 sprint 打分。
2. 把它的分数和你自己的人工判断对比。
3. 有分歧的地方，把 rubric 里的通过/失败标准写得更具体。
4. 对同一个输出重新跑 evaluator，看对齐了没有。
5. 重复直到 evaluator 的判断和人工评审基本一致。

预计需要 3-5 轮校准。每轮记录改了什么、为什么改。

---

quality-document.md

质量快照。给项目的每个产品领域和架构层打分，跟踪代码库随时间是变强了还是变弱了。

怎么用：

- 复制到项目根目录
- 开始会话前：读它，了解代码库哪里最弱

- 会话结束后：更新评级
- 长期：对比不同时间点的快照，看哪些 harness 改动真正改善了代码库健康度

评什么：

- **产品领域**（如文档导入、问答流程、索引）：每个领域按验证状态、Agent 可读性、测试稳定性、关键缺口打分
- **架构层**（如 Main Process、Preload、Renderer、Services）：每层按边界执行和 Agent 可读性打分

为什么需要它：

Evaluator rubric 评的是单次 agent 输出的质量。Quality document 评的是代码库本身的质量。它们回答的是不同的问题：

- Evaluator rubric："这轮 agent 做得好不好？"
- Quality document："这个项目是在变强还是变弱？"

什么时候更新：

- 每轮重要会话之后
- 做基准对比之前
- 做清理或简化之后
- 换新 agent 或新模型时

和 harness 简化的关系：

Quality document 也可以用来验证 harness 是否可以简化。Harness 里的每个组件都编码了一个假设——"模型做不到这件事"。模型变强后，这些假设就过时了。检查某个组件是否还有必要：

1. 拍一份 quality document 快照。
2. 移除一个 harness 组件。
3. 跑基准测试。
4. 再拍一份快照。
5. 对比——评级没降，说明那个组件是多余的。降了，就恢复。

编码代理开工流程

初始化完成后，每轮编码会话都按这个流程开始。

固定开工模板

1. 运行 `pwd`，确认仓库根目录
2. 读取 `claude-progress.md`
3. 读取 `feature_list.json`
4. 用 `git log --oneline -5` 查看最近提交
5. 运行 `./init.sh`
6. 跑一条基础 smoke test 或端到端路径
7. 如果基础状态已坏，先修这个
8. 选择最高优先级的未完成功能
9. 只围绕这个功能工作，直到它被验证或明确 blocked

为什么顺序不能乱

- `pwd` 能防止在错误目录里干活
- progress 和 feature 文件先恢复持久状态
- 最近提交能解释刚刚发生了什么
- `init.sh` 让启动过程标准化，而不是靠记忆
- 基础验证先跑，可以避免在坏状态上继续叠改动

对应的收尾流程

同一轮会话结束时，也应该镜像地做这些事：

1. 记录进度
2. 更新功能状态
3. 必要时写交接摘要
4. 提交安全状态的代码
5. 留下可直接重启的干净环境

中文参考

这一部分解释这些模板该怎么配合使用，而不是把它们当成一堆孤立文件。

内部参考材料

- `method-map.md`：把常见长时任务翻车点映射到对应方法和工件
- `initializer-agent-playbook.md`：初始化代理在第一轮应该产出什么
- `coding-agent-startup-flow.md`：后续编码代理每次开工的固定流程
- `prompt-calibration.md`：根指令应该写到什么程度才合适

重点参考文章

这里的筛选标准很窄：只保留能直接解释 harness 机制的文章。Harness 在这里指模型外部的运行系统，包括 agent loop、工具执行、沙箱、状态、上下文、验证、终止条件、控制平面和观测反馈；不是泛泛的 prompt engineering 或 agent 框架介绍。

保留原始三篇作为课程主轴：

- OpenAI: Harness engineering: leveraging Codex in an agent-first world（2026-02-11）：agent-first 仓库、repo-local context、custom lint、结构性 guardrail。
- Anthropic: Effective harnesses for long-running agents（2025-11-26）：initializer agent、coding agent、feature list、progress log、跨上下文窗口交接。
- Anthropic: Harness design for long-running application development（2026-03-24）：planner / generator / evaluator 三角色、context reset、harness 简化和组件过期问题。

额外只加入几篇高相关、高含金量的 2026 文章：

- OpenAI: Unrolling the Codex agent loop（2026-01-23）：解释 Codex runtime harness 的核心循环、工具调用、上下文增长和终止状态。
- Anthropic: Demystifying evals for AI agents（2026-01-09）：明确评估 agent 时评的是 model + harness，并区分 evaluation harness 与 agent harness。

- LangChain: Improving Deep Agents with harness engineering（2026-02-17）：同一模型不变，只改 system prompt、tools、middleware、tracing 和 self-verification，让 coding agent 在 Terminal Bench 2.0 上从 Top 30 进到 Top 5。
- Thoughtworks / Martin Fowler: Harness engineering for coding agent users（2026-04-02）：把 coding-agent user harness 拆成 feedforward guides 和 feedback sensors，并区分 deterministic controls 与 inferential controls。
- Cursor: Continually improving our agent harness（2026-04-30）：把 harness 当成持续迭代的产品系统，用离线评估、线上指标、工具错误分类、模型定制和 mid-chat model switching 改善 agent 行为。

2026 扩展参考

这些文章不作为课程主轴，但在设计特定 harness 模块时很有借鉴价值。只保留文章正文直接涉及 agent loop、工具执行、上下文管理、验证、沙箱、控制层、回归治理等机制的材料；纯 agent 产品、平台发布、团队实践或 benchmark 不放进这里。

- OpenAI: Unlocking the Codex harness: how we built the App Server（2026-02-04）：把 harness 抽象成 App Server 协议，覆盖 thread lifecycle、resume、fork、diff 和客户端集成。
- OpenAI Developers: Run long horizon tasks with Codex（2026-02-23）：长时任务中的 durable project memory、milestone validation 和 done-when 例子。
- OpenAI: The next evolution of the Agents SDK（2026-04-15）：model-native harness、sandbox execution、文件与命令执行能力。
- OpenAI: An open-source spec for Codex orchestration: Symphony（2026-04-27）：把 issue tracker / Linear 变成多 agent 控制平面。
- Anthropic: Building a C compiler with a team of parallel Claudes（2026-02-05）：并行 agent 团队、任务锁、git 同步、容器隔离和自主循环。
- Anthropic: Scaling Managed Agents: Decoupling the brain from the hands（2026-04-08）：meta-harness 视角，把 session、harness、sandbox 拆成可替换接口。
- Anthropic: An update on recent Claude Code quality reports（2026-04-23）：reasoning effort、context pruning、system prompt 都属于 harness 变更，且需要回归治理。
- LangChain: Context Management for Deep Agents（2026-01-28）：filesystem offloading、tool-call truncation、summarization 和 targeted evals 组成的 context-

management harness。

- LangChain: Tuning Deep Agents to Work Well with Different Models（2026-04-29）：用 model-specific harness profiles 调整 prompt、tool names、middleware 和 subagent 配置。
- LangChain: Continual learning for AI agents（2026-04-05）：把 agent 改进拆成 model、harness、context 三层，并把 traces 作为改进信号。
- Microsoft: Agent Harness in Agent Framework（2026-03-12）：shell/filesystem harness、approval flow、hosted shell、context compaction。
- Google: Announcing ADK for Java 1.0.0（2026-03-30）：插件、event compaction、HITL、session/memory service、A2A 等可复用 harness primitives。
- GitHub: Automate repository tasks with GitHub Agentic Workflows（2026-02-13）：把 GitHub Actions 变成 agentic workflow runner，包含 safe outputs、sandboxing、permissions、review。
- AWS: AI agents in enterprises: Best practices with Amazon Bedrock AgentCore（2026-02-03）：Runtime、Memory、Gateway、Identity/Policy、Observability、Evaluations 的企业级 harness 分层。
- Stripe: Minions: Stripe's one-shot, end-to-end coding agents（2026-02-09）和 Part 2（2026-02-19）：devbox 隔离、custom agent harness、blueprints 状态机、规则文件、MCP tool curation、安全控制、pre-push/CI 反馈循环。
- Cognition: What We Learned Building Cloud Agents（2026-04-23）：云端 agent runtime 的 VM 隔离、session snapshot/resume、orchestration、governance、audit logging 和 integrations。
- Cognition: Multi-Agents: What's Actually Working（2026-04-22）：generator-verifier loop、clean-context reviewer、smart-friend routing、manager-child coordination 和跨 agent 通信边界。
- Addy Osmani: Loop Engineering（2026-06-07）：从 `/goal` 到六原语的完整 loop engineering 框架——automations、worktrees、skills、connectors、sub-agents、external state，以及四种沉默成本。
- Replit: Decision-Time Guidance: Keeping Replit Agent Reliable（2026-01-20，2026-01-23 更新）：用轻量分类器在关键决策点注入短指令，而不是把所有规则塞进系统提示词。
- Vercel: How we made v0 an effective coding agent（2026-01-07）：动态系统提示、streaming rewrite layer 和 deterministic/model-driven autofixers。

- Vercel: Introducing deepsec（2026-05-04）：面向安全扫描的 coding-agent harness，包含 scan、investigate、revalidate、enrich、export、plugin 和 refusal-checker。
- Sourcegraph: CodeScaleBench（2026-03-03）：偏 eval/tooling harness，包含 MCP tool adoption、tool-use transcripts、benchmark QA、verifier/reproducibility gates 和 prompt/preamble 迭代。

严格按时间筛选时，2025-only 的泛参考不进入主列表。原始三篇中的 Anthropic 2025 文章保留，是因为它是本课程方法的基础来源。

- ETH Zurich: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?: 仓库上下文文件的实证研究：任务成功率、推理成本，以及保持要求精简的建议。见摘要和结论。
- On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents: gpt-5.2-codex; 10 repos / 124 PR tasks; Table 1.

推荐阅读顺序

1. `method-map.md`
2. `initializer-agent-playbook.md`
3. `coding-agent-startup-flow.md`
4. `prompt-calibration.md`
5. OpenAI Harness engineering
6. Anthropic Effective harnesses
7. Anthropic Harness design for long-running application development
8. OpenAI Codex agent loop
9. Anthropic agent evals
0. LangChain Improving Deep Agents
1. Thoughtworks / Martin Fowler Harness engineering for coding agent users
2. Cursor Continually improving our agent harness

初始化代理操作手册

这个手册用于仓库里的第一轮重要会话，也就是正式开始增量开发之前的初始化阶段。

目标

先搭出稳定的工作面，让后续会话不必重新推导启动命令、当前状态和任务边界。

必需产出

初始化阶段至少应该留下这些工件：

- 一个根指令文件，例如 `AGENTS.md` 或 `CLAUDE.md`
- 一个机器可读的功能面，例如 `feature_list.json`
- 一个持久进度工件，例如 `claude-progress.md`
- 一个标准启动辅助脚本，例如 `init.sh`
- 一个记录基础脚手架状态的初始安全提交

检查清单

1. 定义标准启动路径
2. 定义标准验证路径
3. 建立进度日志并写下初始状态
4. 把工作拆成功能，并给出状态字段
5. 建立第一个干净的 baseline commit

成功标准

一个完全没有前文聊天上下文的新会话，应该能回答：

- 这个仓库是做什么的
- 怎么启动
- 怎么验证
- 还有什么没做完
- 下一步最佳动作是什么

方法对照表

这个表把最常见的长时 coding-agent 失败模式，对应到最先该补的工件或规则。

失败模式	实际表现	首要修复	主要工件
新会话摸黑	新会话花大量时间重新摸索状态和启动方式	让仓库成为唯一事实来源	`claude-progress.md`
范围蔓延	一次启动多个功能，最后没有一个完整收尾	限制当前活跃范围	`feature_list.json`
提前宣布完成	代码改了就说“完成了”，但没有可运行证据	把完成绑定到验证证据	`clean-state-checklist.md`
启动脆弱	每轮会话都要重新学怎么启动项目	统一启动与验证路径	`init.sh`
交接薄弱	下一轮看不出哪里可用、哪里坏了、接下来做什么	明确写交接摘要	`session-handoff.md`
评审主观	质量判断依赖个人记忆和感觉	用固定维度做评分	`evaluator-rubric.md`

使用原则

优先补最能直接消除当前失败模式的那个工件，不要一出问题就把更多规则堆进一个超长 instruction 文件里。

Prompt 校准

根指令文件应该定义工作框架，而不是把所有动作写死。

应该留在根文件里的内容

- 仓库用途和范围
- 启动路径
- 验证路径
- 不可违反的约束
- 必需的状态工件
- 会话结束规则

应该移出根文件的内容

- 过长的历史边角案例
- 只属于某个子系统的细节实现说明
- 更适合贴在代码附近的局部架构笔记
- 只对单一模块成立的示例

工作原则

根文件的职责是让新会话快速建立方向感。如果它开始变成“过去每次失败都往里加一句”的堆叠场，就应该把细节拆到更小、更明确的文档里，再从根文件跳转过去。

OpenAI 高级资源包

这个文件夹把 OpenAI 那篇《Harness engineering: leveraging Codex in an agent-first world》里更完整、更高阶的仓库组织方式整理成可以直接参考和复制的模板。

当最小 harness 已经不够用，而你的仓库开始需要下面这些能力时，就该看这一套：

- 一个简短、负责路由的 `AGENTS.md`
- 放在仓库里的“唯一事实来源”文档体系
- 活跃执行计划与已完成计划的分层管理
- 明确的产品、可靠性、安全、前端治理文件
- 按产品领域和架构层持续更新的质量评分
- 面向模型阅读的参考材料目录
- 针对架构、知识沉淀、运行验证的标准 SOP

包含的目录骨架

`repo-template/` 里提供了一套可直接复制的起步 结构，核心布局如下：

```text
AGENTS.md
ARCHITECTURE.md
docs/
├── design-docs/
│   ├── index.md
│   └── core-beliefs.md
├── exec-plans/
│   ├── active/
│   ├── completed/
│   └── tech-debt-tracker.md
├── generated/
│   └── db-schema.md
├── product-specs/
│   ├── index.md
│   └── new-user-onboarding.md
├── references/
│   ├── design-system-reference-llms.txt
│   ├── nixpacks-llms.txt
│   └── uv-llms.txt
├── DESIGN.md
├── FRONTEND.md
├── PLANS.md
├── PRODUCT_SENSE.md
├── QUALITY_SCORE.md
├── RELIABILITY.md
└── SECURITY.md
```

怎么用

1. 如果仓库还小，先用最小资源包。
2. 一旦进入多模块、多轮会话、长期演化阶段，就把 `repo-template/` 里的骨架复制到你的仓库。
3. 保持 `AGENTS.md` 很短，把深层规则拆到 `docs/` 里。
4. 把质量文档、可靠性文档、执行计划当成日常开发的一部分，而不是事后补写。
5. 把生成物和外部参考材料明确收进仓库，避免 agent 依赖聊天上下文或人的记忆。

SOP 资料库

`sops/` 把文章里的几张关键图，整理成可以逐步执行的标准流程：

- 分层领域架构搭建 SOP
- 把不可见知识编码进仓库的 SOP
- 本地可观测性栈与反馈回路 SOP
- 用 Chrome DevTools 做 UI 验证闭环的 SOP

设计原则

- 短入口，深链接
- 仓库就是唯一事实来源
- 机械约束优先于口头约定
- 计划、质量和技术债都和代码一起版本化
- 清理与 harness 简化是常规工作，不是临时救火

这套资源包是有倾向性的模板，不是必须逐字照抄。最好的用法是把它当成一套高质量起点，再按你的项目改造。

AGENTS.md

这个仓库面向长时运行的 coding-agent 工作流。保持这个文件简短，把它当成"唯一事实来源"文档的入口和路由层，而不是一个不断膨胀的大说明书。

开工流程

改代码前先做这些事：

1. 用 `pwd` 确认仓库根目录。
2. 读取 `ARCHITECTURE.md`，理解当前系统地图和硬性依赖规则。
3. 读取 `docs/QUALITY_SCORE.md`，先知道最弱的产品领域和架构层。
4. 读取 `docs/PLANS.md`，再打开当前要执行的 active plan。
5. 读取相关的 `docs/product-specs/` 规格文档。
6. 跑这个仓库约定的 bootstrap 与验证路径。
7. 如果基础验证先失败，先修 baseline，再加新范围。

路由地图

- `ARCHITECTURE.md`：领域地图、分层模型、依赖规则
- `docs/design-docs/index.md`：设计决策与核心信念
- `docs/product-specs/index.md`：当前产品行为与验收目标
- `docs/PLANS.md`：计划生命周期与执行计划规则
- `docs/QUALITY_SCORE.md`：产品领域与架构层健康度
- `docs/RELIABILITY.md`：运行信号、benchmark、重启要求
- `docs/SECURITY.md`：密钥、沙箱、数据和外部动作规则
- `docs/FRONTEND.md`：UI 约束、设计系统规则、可访问性检查

工作约定

- 一次只围绕一个有边界的计划或功能切片工作。
- 不能只靠读代码就宣布完成，必须有可运行证据。
- 只要改了行为，就同步更新对应的产品、计划或可靠性文档。
- 如果某类 review feedback 反复出现，把它升级成机械规则、检查或 linter，不要一直在聊天里重复解释。
- 生成物放进 `docs/generated/`，外部 reference 放进 `docs/references/`。
- 需要更多细节时，优先补小而新的文档，而不是继续把这个文件写长。

完成定义

一个改动只有在以下条件都满足时才算完成：

- 目标行为已实现
- 要求的验证真的跑过
- 证据已经挂到相关 plan 或质量文档里
- 受影响的文档仍然是最新的
- 仓库能按标准启动路径干净重启

收尾

结束会话前：

1. 更新当前 active execution plan。
2. 如果产品领域或架构层有明显变化，更新 `docs/QUALITY_SCORE.md`。
3. 如果有延期处理的债务，记到 `docs/exec-plans/tech-debt-tracker.md`。
4. 已完成的计划及时移到 `docs/exec-plans/completed/`。
5. 保证仓库可重启，并留下清晰的下一步动作。

ARCHITECTURE.md

这份文件是系统的顶层地图。它应该保持简短，只提供最关键的结构信息，并把更深的内容指向其他文档。

系统形态

- 产品：`[替换成产品名]`
- 主用户流程：`[替换成核心流程]`
- 运行面：`[desktop / web / cli / services / workers]`
- 产品行为真相来源：`docs/product-specs/`

领域地图

领域	负责什么	主要入口	对应规格
`[domain-a]`	`[职责]`	`[模块 / 路由 / 命令]`	`[spec path]`
`[domain-b]`	`[职责]`	`[模块 / 路由 / 命令]`	`[spec path]`

分层模型

用固定方向的分层模型，避免 agent 临场发明架构：

`Types → Config → Repo → Service → Runtime → UI`

跨领域关注点应该通过明确的 provider 或 adapter 边界进入，而不是直接跨层穿透。

硬性依赖规则

- 低层不能依赖高层。
- UI 不能绕过 runtime 或 service 契约。
- 数据访问必须通过 repo 或等价 adapter 进入。
- 共享 util 必须保持通用，不能慢慢堆成领域逻辑垃圾桶。
- 新依赖要在对应 plan 或 design doc 里说明理由。

横切接口

关注点	允许的边界	备注
日志与 tracing	`[provider / utility path]`	`[只允许结构化日志，不允许临时 console]`
Auth	`[provider path]`	`[token/session 规则]`
外部 API	`[client 或 provider path]`	`[限流 / 重试原则]`
Feature flags	`[flag boundary]`	`[归属]`

当前热点

- `[最难安全修改的区域]`
- `[边界最弱或测试最脆的区域]`

变更检查

当你修改了会影响架构的代码：

1. 如果领域地图或允许边界变了，就更新这份文件。

2. 如果背后的设计理由变了，就更新 `docs/design-docs/` 里的相关文档。
3. 如果规则应该机械执行，就补一个可执行检查。

核心信念

- 仓库就是 agent 的唯一事实来源。
- `AGENTS.md` 是路由器，不是百科全书。
- 验证证据比自信更重要。
- 一次做好一个有边界的任务，胜过同时开很多半成品。
- 反复出现的人类反馈要升级成可复用的 harness 规则。
- 清理与简化都是交付的一部分，不是事后补救。
- 如果 agent 不能在仓库里发现某个事实，就把这个事实视为运行上不存在。

设计文档索引

把这份索引当成设计历史的可发现地图。

Accepted

- `core-beliefs.md`：agent-first 的运行信念与持久项目规范

Proposed

- `[在这里添加新的设计文档路径]`

Deprecated

- `[把被替代或过期的设计文档移到这里，并给出替代链接]`

维护规则

- 每份设计文档都应该有 owner 或更新触发条件。
- 过期文档要么删除，要么明确标成 deprecated，不要让它悄悄漂移。
- active execution plan 要链接自己依赖的设计文档。

DESIGN.md

这份文件是设计文档入口。保持简短，把更细的内容路由到 `docs/design-docs/` 里的具体文件。

目的

记录那些应该跨越单次聊天、单个 sprint、单个 reviewer 记忆而持续存在的产品与系统设计决策。

什么时候先看它

- 你需要理解当前的设计哲学
- 你准备引入新模式
- 你要判断哪些设计已经定了，哪些还在开放状态

核心设计文档

- `docs/design-docs/index.md`：accepted / proposed / deprecated 文档索引
- `docs/design-docs/core-beliefs.md`：项目级 agent-first 核心信念

设计规则

- 设计文档要小而新。
- 一个决策领域尽量对应一份文档。
- 某个改动依赖设计文档时，要在 plan 和 spec 里显式链接它。
- 如果某条设计规则已经变成运行上的硬要求，就把它升级成自动化检查或更新到 `ARCHITECTURE.md`。

当前执行计划

这个文件夹里每一份 markdown 都代表一份当前仍在执行的计划。

建议文件名格式：

- `YYYY-MM-DD-short-topic.md`

每份 active plan 都应该足够新，让一个没有聊天上下文的新 agent 也能只靠仓库继续推进。

已完成计划

已完成的计划移动到这里，不要直接删掉。完成计划也是仓库记忆面的一部分，后续 agent 需要靠它理解代码为什么会变成现在这样。

技术债跟踪

这份文件只记录那些真实存在、已经确认、并且被有意识延后的技术债。

日期	区域	债务	为什么延后	风险	下次触发点
YYYY-MM-DD	`[area]`	`[debt]`	`[reason]`	`[risk]`	`[when to revisit]`

FRONTEND.md

这份文件定义稳定的前端预期，避免 agent 每次都临场发明一套 UI 模式。

UI 原则

- 先保证清晰，再追求新鲜感。
- 交互流程要可发现、可重走、可重启。
- 优先沉淀少量可复用组件，而不是到处长一次性变体。
- 可访问性检查属于正常验证，不是最后的美化工作。

护栏

- 把设计系统或组件库参考材料放进 `docs/references/`。
- 显式记录关键用户状态：empty、loading、success、error、retry。
- 在不同流程里保持文案、键盘行为、视觉层级一致。
- 修复 UI bug 后，顺手补上对应验证步骤。

验证要求

- 为关键用户旅程留下证据。
- 把浏览器或运行时验证步骤写进相关 plan。
- 如果视觉回归常见，就标准化截图或 DOM 检查。

数据库结构

这个目录用来存放那些生成出来但又值得 agent 直接阅读的工件，避免它每次都去源码里反推。

来源

- 生成命令或来源：`[command or source path]`
- 最近刷新时间：`YYYY-MM-DD`

备注

- 生成部分不要手改。
- 底层 schema 一变，就重新生成这份文件。

PLANS.md

这份文件定义执行计划如何创建、更新、完成和归档。

什么时候必须有 plan

当工作满足以下任一条件时，就创建 execution plan：

- 会跨越多个会话
- 会同时影响多个子系统
- 验证或上线风险不小
- 存在需要显式记录的开放决策

计划放哪

- `docs/exec-plans/active/`：当前正在驱动工作的计划
- `docs/exec-plans/completed/`：已完成但仍然要保留上下文的计划
- `docs/exec-plans/tech-debt-tracker.md`：延期处理的债务与 follow-up

最少要包含的部分

- 目标
- 范围与明确不做什么
- 验证路径
- 风险与 blocker
- 进度日志
- 开放决策

运行规则

- 一份 active plan 同一时间应该只有一个清晰的当前步骤。
- plan 要随着工作推进而更新，不要把它当成静态散文。
- 只要某个决策改变了实现方向，就记到 plan 里。
- 已完成的计划要移到 `completed/`，让后续 agent 仍然能发现历史上下文。

PRODUCT_SENSE.md

这份文件记录那些代码本身并不能可靠表达出来的产品判断。

产品核心

- 主要用户：`[替换]`
- 要完成的任务：`[替换]`
- 最想解决的核心痛点：`[替换]`
- 可接受质量门槛：`[替换]`

产品规则

- 优先保证用户可感知的可靠性，而不是一味加功能。
- 行为有歧义时，把它当成 spec 缺口，不要默认允许猜。
- 只要实现改变了用户看见或信任的东西，就更新对应 spec。
- 具体流程写在 product spec 里，跨流程的产品优先级写在这里。

禁区模式

- 隐蔽的破坏性操作
- 没有用户反馈的静默失败
- 用户可见状态没有明确真相来源
- 不能用一句话解释清楚的功能

产品规格索引

这个目录用来放当前仍然有效的用户可见行为规格。

当前有效规格

- `new-user-onboarding.md`

规则

- spec 重点描述用户可见行为和验收标准。
- 如果实现和 spec 不一致，要在同一轮会话里更新其中一方。
- 保持这份索引是最新的，让新 agent 能快速发现产品范围。

新用户引导

目标

描述一个新用户第一次进入产品时应该经历的流程。

起始条件

- [流程开始前的状态]

用户流程

1. [步骤一]
2. [步骤二]
3. [步骤三]

验收标准

- [用户可观察到的结果]
- [用户可观察到的结果]
- [用户可观察到的结果]

失败状态

- [可恢复错误与用户反馈]
- [被阻塞状态与兜底方案]

QUALITY_SCORE.md

这份文档用来跟踪仓库是在变强还是变弱。

评级标准

- `A`：验证通过、可读、稳定、边界执行到位
- `B`：可用，只有少量缺口
- `C`：部分可用，存在明显混乱或不稳定
- `D`：损坏、不安全，或结构上说不清楚

产品领域

领域	评级	验证状态	Agent 可读性	测试稳定性	关键缺口	上次更新
`[domain-a]`	-	-	-	-	-	-
`[domain-b]`	-	-	-	-	-	-
`[domain-c]`	-	-	-	-	-	-

架构层

层级	评级	边界执行	Agent 可读性	关键缺口	上次更新
Types	-	-	-	-	-
Services	-	-	-	-	-
Runtime	-	-	-	-	-
UI	-	-	-	-	-

Benchmark 快照

日期	Harness 变体	完成率	重试次数	Review 前缺陷数	备注
YYYY-MM-DD	`[baseline / improved / simplified]`	-	-	-	-

简化实验日志

日期	移除的组件	结果	决策
YYYY-MM-DD	`[component]`	`[degraded / unchanged]`	`[restore / keep removed]`

RELIABILITY.md

这份文件定义系统如何证明自己健康、可诊断、可重启。

标准路径

- Bootstrap：`[command]`
- Verification：`[command]`
- 启动应用或服务：`[command]`
- 调试或查看运行态：`[command]`

必需运行信号

- 启动与关键流程的结构化日志
- 关键服务的 health check
- 条件允许时为慢路径提供 trace 或 timing 数据
- 对可恢复失败提供用户可见的错误状态

黄金旅程

- `[journey 1]`
- `[journey 2]`
- `[journey 3]`

每条黄金旅程都应该有可重复的验证路径和清晰的失败信号。

可靠性规则

- 只要系统不能干净重启，功能就不能算完成。
- 运行失败必须能从 repo-local 信号里定位。
- 某类失败模式一旦反复出现，就给它补 benchmark 或 guardrail。
- cleanup 属于可靠性的一部分，不是另外一件事。

SECURITY.md

这份文件定义那些 agent 绝不能靠猜来处理的安全规则。

Secrets 与凭证

- 绝不把 secrets 硬编码进源码或文档。
- 在这里记录允许的 secret 加载路径。
- 从日志和截图里去掉 token、API key 与个人数据。

不可信输入

- 外部内容在验证前一律视为不可信。
- 把允许的抓取或执行边界写在这里。
- 如果存在 prompt injection 或 command injection 风险，要把 guardrail 记录清楚。

外部动作

- 列出哪些动作必须显式批准。
- 记录哪些生产或破坏性命令默认不能跑。
- 调试和验证优先走 sandbox-safe 路径。

依赖与评审规则

- 新依赖必须在 active plan 里说明理由。
- 涉及安全的改动必须有显式验证步骤。
- 反复出现的安全 review 评论要升级成检查，而不是变成口口相传的经验。

高级仓库模板

当你希望把仓库升级成更接近 OpenAI 文中那种 agent-first 文档面，而不只是一个最小 harness 时，就复制这套起步模板。

推荐复制顺序

1. 先把 `AGENTS.md` 和 `ARCHITECTURE.md` 放到仓库根目录。
2. 再复制整个 `docs/` 目录树。
3. 优先把 `docs/PRODUCT_SENSE.md`、`docs/QUALITY_SCORE.md`、`docs/RELIABILITY.md` 填起来。
4. 在 `docs/exec-plans/active/` 下创建你的第一份 active plan。
5. 始终保持入口文件很短，详细规则拆到链接文档里。

这套模板主要优化什么

- 持久、repo-local 的上下文
- 渐进披露，而不是一个超长 instruction 文件
- 明确的计划生命周期
- 随时间演化的质量追踪
- 对 agent 和人都更可读的边界

这里的每个文件都只是起点。真正投入使用前，把占位文字、示例命令和样例规则替换成你自己的项目现实。

SOP：Chrome DevTools 验证闭环

当 UI 改动必须经过真实交互、截图、DOM 状态和 console 输出来判断，而不是只靠读代码时，就用这份 SOP。

目标

把 UI 验证变成 agent 可以反复执行的交互闭环，直到这条用户旅程变干净。

核心闭环

1. 选定目标页面或 app 实例。
2. 清掉旧的 console 噪声。
3. 记录 BEFORE 状态。
4. 触发 UI 路径。
5. 观察交互过程中的 runtime events。
6. 记录 AFTER 状态。
7. 修复问题，必要时重启 app。
8. 反复重跑验证，直到这条旅程 clean。

必需输入

- 稳定的启动命令
- 可重复的 UI journey
- 能抓 DOM、console 或截图的方式
- 明确“什么算 clean”的规则

执行 SOP

1. 把目标旅程写进 active plan。
2. 用可观察行为定义成功：文本出现、按钮可点、错误消失、console 干净、请求成功。
3. 交互前先拍初始状态。
4. 一次只触发一条路径。
5. 记录 runtime event、DOM 变化和可见输出。
6. 如果失败，只修最小负责层，然后重启。
7. 重跑同一路径，对比 BEFORE/AFTER 证据。

Clean 标准

- 目标可见状态已经出现
- 没有意外错误
- console 噪声已理解或清理
- 重跑同一路径结果一致

需要同步更新的仓库工件

- 当前 active execution plan
- 如果它变成黄金旅程，更新 `docs/RELIABILITY.md`
- 如果可见行为改变了，更新 product spec

SOP：把不可见知识编码进仓库

当重要上下文还散落在 Google Docs、聊天记录、ticket 或人的脑子里时，就用这份 SOP。

目标

让那些 agent 以前看不见的知识，变成 codebase 里可发现、可读取、可执行的事实。

触发信号

- agent 总在追问系统到底怎么运作。
- 人开始说“这个是 Slack 里定过的”或“按某某上周说的做”。
- review 里经常引用仓库里根本没写下来的产品、安全或架构规则。
- 新会话总在重复那些本该已经稳定下来的发现工作。

执行 SOP

1. 先列出所有不可见知识来源：外部文档、聊天、默认团队规则、口头决策。
2. 对每条知识判断：它属于架构、产品行为、安全策略、可靠性要求、执行上下文，还是参考材料？
3. 按类别编码到对应仓库工件：
   - 架构 → `ARCHITECTURE.md`
   - 产品行为 → `docs/product-specs/`
   - 设计理由 → `docs/design-docs/`
   - 执行状态 → `docs/exec-plans/`
   - 外部参考材料 → `docs/references/`
   - 质量或可靠性要求 → `docs/QUALITY_SCORE.md` 或 `docs/RELIABILITY.md`
4. 把模糊表达改写成运行上真正有用的表达。

5. 删除或废弃旧副本，保证仓库里有一个可发现的真相来源。

好的编码规则

- 为“可发现”而写，不是为“写得很全”而写。
- 文件尽量短，名字尽量明确。
- 相关工件要互相链接。
- 记录耐久规则，不要把会议流水账原封不动塞进来。
- 决策形成的同一轮会话里，就把仓库更新掉。

完成定义

- 一个全新 agent 不问人也能找到相关规则。
- 同一个事实不会散落在多个互相打架的文件里。
- 新工件放在离它所治理的代码或流程最近的地方。

OpenAI 高级 SOP

这些 SOP 把文章里的工作方式翻成可以直接参考、执行、改造的操作手册。

包含哪些 SOP

- `layered-domain-architecture.md`： 搭建显式分层与跨领域边界
- `encode-knowledge-into-repo.md`： 把聊天、外部文档、脑内知识变成 repo-local 文档
- `observability-feedback-loop.md`： 给 agent 提供 logs、metrics、traces 与可重复的调试闭环
- `chrome-devtools-validation-loop.md`： 用浏览器自动化和快照反复验证 UI，直到 clean

怎么用

1. 先选和你当前瓶颈最匹配的 SOP。
2. 按检查清单补齐缺失工件或工具。
3. 把得到的规则编码回 `repo-template/` 里的文档。
4. 把反复出现的 review 评论升级成检查、脚本或 guardrail。

这些 SOP 不是让你机械照抄，而是为了让 harness 更可读、更可执行、更可复用。

SOP：分层领域架构

当 agent 反复跨层乱连、在不同层重复逻辑、或者几轮会话后代码越来越难审时，就用这份 SOP。

目标

把领域边界写清楚、立起来、能执行，让 agent 在高速度下也不至于悄悄把结构做烂。

目标模型

在一个业务领域内部，优先使用这条单向流：

```
Types → Config → Repo → Service → Runtime → UI
```

跨领域关注点通过明确的 provider 或 adapter 进入。共享 utils 保持在领域之外，不能慢慢长成业务逻辑垃圾场。

建设检查清单

- 在 `ARCHITECTURE.md` 里列清当前 domains。
- 在 `ARCHITECTURE.md` 里写清允许的依赖方向。
- 记录 auth、telemetry、外部 API 等 cross-cutting interface。
- 为当前最难处理的边界违规写一条短说明。
- 决定哪些规则应该升级成 lint、test 或 script。

执行 SOP

1. 先给代码库画 domain map，再谈实现风格。
2. 对每个 domain，明确允许的 layer sequence。

3. 找出所有横切关注点，并把它们改走 provider 或 adapter。
4. 把含糊的 shared logic 重新放回所属 domain，或抽成真正通用的 utils。
5. 把规则写进 `ARCHITECTURE.md`。
6. 先为代价最高的一类违规补一个可执行 guardrail。
7. 改完后更新质量评分。

完成定义

- 一个全新的 agent 能判断某个改动应该落在哪一层。
- UI 不再直接连 repo 或外部副作用。
- 横切能力都有明确入口。
- 至少一条重要边界已经被机械执行。

需要同步更新的仓库工件

- `ARCHITECTURE.md`
- `docs/QUALITY_SCORE.md`
- 如果设计理由变了，更新 `docs/design-docs/`
- `docs/PLANS.md` 或当前 active execution plan

SOP：可观测性反馈闭环

当调试太慢、agent 总在没证据的情况下宣布成功、或者运行时行为比代码本身还难看懂时，就用这份 SOP。

目标

给 agent 一套本地闭环，让它可以基于 logs、metrics、traces 和可重复 workload 来判断系统，而不是只靠看代码猜。

最小可用栈

- 应用输出结构化日志
- 条件允许时输出 metrics 和 traces
- 本地采集或 fan-out 层
- 可查询 logs / metrics / traces 的接口
- 每次改动后都能重跑的 workload 或 user journey

执行 SOP

1. 先定义最重要的黄金运行旅程。
2. 给启动流程和关键路径补结构化日志。
3. 在合适位置补 latency、failure count、queue depth 之类的 metrics。
4. 为慢路径或多阶段流程补 traces 或 timing 标记。
5. 让这些信号能从本地开发环境查询到。
6. 给 agent 一条可以反复重跑的 workload 或场景。
7. 强制执行这条闭环：query → correlate → reason → implement → restart → rerun → verify。

调试会话检查清单

- 到底哪里失败了?
- 哪条信号能证明它失败?
- 失败归属哪一层?
- 修复后哪条信号发生了变化?
- App 是否能干净重启?
- 同一 workload 重跑后是否通过?

完成定义

- agent 能用运行证据解释失败模式。
- 每次改动后都能重跑同一 workload。
- restart 与 rerun 已经成为正常任务循环的一部分。
- 可靠性信号已经记录到 `docs/RELIABILITY.md`。